Mobile System Design

For Android SDE-3 Interviews

The Complete Edition

35 system designs · 3 parts · easy → hard

Part 1 — Foundations & First Mobile Pieces (12)
Parking Lot → Elevator → Chess → Library → ATM → Vending Machine → Snake & Ladder → Tic-Tac-Toe → Splitwise → LRU Cache → Image Loader → Pagination

Part 2 — Mobile Apps Core (12)
Push Notifications → Analytics SDK → Offline-First To-Do → News Feed → 1-1 Chat → BookMyShow → E-commerce Home → Stock Ticker → Tinder → Stories → Google Pay → Slack/Teams

Part 3 — Hard Mobile Systems (11)
Instagram Feed → WhatsApp (full) → Uber Driver → Google Photos → Netflix → Spotify → Food Delivery → Dropbox sync → Google Maps → YouTube → Twitch live streaming

How to use this book

Read in order, but skip when needed.

Each chapter is self-contained. The difficulty grows, but it is fine to jump to a famous app (Instagram, WhatsApp, Uber) if that is what your next interview will probe.

Don't memorise — internalise the 7-section recipe.

Every senior interviewer can spot a candidate who has rote-learned. The recipe in this book is the skeleton of every real mobile system design — once you have it in your bones, you can answer questions you have never seen.

Talk through the clarifying questions out loud.

Section 2 of each chapter lists the questions you should ask the interviewer. The biggest mistake candidates make is jumping into design before scoping. Five minutes of clarification saves you forty minutes of building the wrong thing.

Treat the code as reference, not as answers.

You will not write working code at the whiteboard. The code blocks exist so you have a concrete Kotlin idiom to point at, the function names to drop into conversation, and the corner cases (thread-safety, lifecycle) made visible.

Read the cross-questions twice.

These are the follow-ups senior interviewers ask after you've drawn the HLD. Knowing how to respond is half the interview. The answers in section 6 are short on purpose — they're what you should say, not write.

> ■ ***Tip:*** *Treat the interview as a conversation, not a presentation. Your interviewer wants to see your thinking, not your slides. Pause, draw, and ask before you commit.*

Table of Contents

Part 1 — Foundations & First Mobile Pieces

1. Design a Parking Lot — **Easy**

The most classic LLD question — assigns spots to vehicles, charges tickets, handles entry and exit.

2. Design an Elevator (Lift) System — **Easy**

Schedule one or more elevators efficiently across N floors. Tests state machines and scheduling algorithms.

3. Design a Chess Game — **Easy**

Model a chess board, pieces, moves, turns, and termination — the canonical polymorphism test.

4. Design a Library Management System — **Easy**

Members borrow books, librarians issue and return them. A pure object-modelling question.

5. Design an ATM — **Easy**

A state-machine over a card session: insert → authenticate → choose → dispense → eject. Concurrency-safe cash drawer.

6. Design a Vending Machine — **Easy**

A miniature ATM. Tests the State pattern more directly than any other classic LLD.

7. Design Snake & Ladder — **Easy**

A simple board-game LLD — perfect for warming up. Tests turn order, board state, and small extensions.

8. Design Tic-Tac-Toe — **Easy**

The smallest LLD that still rewards good modelling — and the win-detection algorithm is a nice optimisation chat.

9. Design Splitwise — **Easy**

Friends share expenses; the app computes who owes whom. Tests graph thinking and split strategies.

10. Design an LRU Cache — **Easy**

The most asked data-structure design question — O(1) get and put, with optional thread-safety and TTL.

11. Design an Image Loading Library (like Glide / Coil) — **Medium**

Build a Glide-clone — fetch, decode, cache, recycle bitmaps, and stay smooth in a scrolling RecyclerView.

12. Design Infinite-Scroll Pagination (mobile feed) — **Medium**

How a feed loads forever without lagging — cursor pagination, prefetch, deduplication, scroll restoration.

Part 2 — Mobile Apps Core

13. Design a Push Notification System (FCM-based) — **Medium**

How a notification lands on the user's phone — from server trigger through FCM to a click that opens the right screen.

14. Design a Mobile Analytics SDK (like Firebase / Amplitude) — **Medium**

Track events, batch them, persist offline, upload reliably — without slowing the host app or draining battery.

15. Design an Offline-First To-Do App with Server Sync — **Medium**

The canonical offline-first design — Room as source of truth, sync queue, last-write-wins or smarter conflict resolution.

16. Design a News Feed (Twitter-style mobile) — **Medium**

Mobile-side of a feed: API design, pagination, caching, optimistic likes, real-time updates, modular feed items.

17. Design a 1-to-1 Chat (basic WhatsApp) **Medium**

Two users exchange messages in real time; offline messages sync on reconnect; ticks for sent/delivered/read.

18. Design a Movie Ticket Booking App (BookMyShow) **Medium**

Browse → pick seats → lock for 5 minutes → pay → confirm. The seat-locking race condition is the core challenge.

19. Design an E-commerce App's Home Page (Amazon mobile) **Medium**

A long scroll of widgets — carousels, banners, product rows, ads. Backend-driven UI is the production answer.

20. Design a Stock Ticker Screen (Real-time Quotes) **Medium**

Hundreds of tickers stream updates per second. UI must feel alive without burning the CPU or battery.

21. Design a Tinder-style Swipe App **Medium**

Swipe cards left/right, never wait for the next profile to load, never see the same profile twice.

22. Design Instagram Stories (mobile) **Medium**

Ephemeral 24-hour stories — tap to skip, swipe to next user, prefetch the next, auto-track 'seen'.

23. Design a Mobile Payment App (Google Pay / Paytm) **Medium**

P2P money transfer — secure flow, idempotency, network resilience, biometric authorisation.

24. Design a Slack/Teams Chat (channels + threads) **Medium**

Multi-channel team chat with threads, mentions, search, and presence — the 1-1 chat extended to a group context.

Part 3 — Hard Mobile Systems

25. Design the Full Instagram Feed (mobile + ranking + media) **Hard**

Ranked feed of friends' posts (image, carousel, reel). Backend ranks server-side, mobile renders mixed-media smoothly.

26. Design Full WhatsApp (Groups, E2E Encryption, Multi-Device) **Hard**

The hard parts of WhatsApp: end-to-end encryption with the Signal protocol, groups, and same-account on multiple devices.

27. Design the Uber Driver App (Location-based services) **Hard**

Live location streaming with low battery cost, ride matching, offline acceptance, navigation.

28. Design Google Photos (Upload, Sync, Gallery) **Hard**

Continuous background upload of every new photo, deduplication, fast gallery, on-device search.

29. Design Netflix Mobile (Player, ABR, Offline Downloads) **Hard**

ExoPlayer-based video player with adaptive bitrate, content protection (Widevine DRM), and offline downloads.

30. Design Spotify Mobile (Streaming + Offline + Playlists) **Hard**

Audio streaming with gapless playback, smart caching, offline downloads, background playback through screen lock.

31. Design a Food Delivery App (Swiggy / Zomato) **Hard**

Three apps in one: customer, restaurant, delivery partner — all coordinated by an order state machine.

32. Design Dropbox / Drive File Sync (mobile) **Hard**

Two-way sync between device and cloud — efficient chunked uploads, conflict resolution, selective sync.

33. Design Google Maps Mobile (Tiles, Search, Navigation) **Hard**

Render map tiles, search POIs, compute driving routes with live re-routing, work offline.

34. Design YouTube Mobile (Watch + Recommendations) **Hard**

Video-first app with recommendations, mini-player, comments, picture-in-picture — Netflix patterns + UGC complexity.

35. Design a Live Streaming App (Twitch mobile) **Hard**

Sub-3-second-latency live video, parallel chat with thousands of viewers, hearts & reactions, follower notifications.

Part 1

Foundations & First Mobile Pieces

Chapters 1 — 12

Chapter 1: Design a Parking Lot

Difficulty: Easy | **Tags: LLD OOP State Strategy**

The most classic LLD question — assigns spots to vehicles, charges tickets, handles entry and exit.

1. What's the Problem?

Build a parking lot system that lets vehicles enter, get a ticket, park in an available slot, and pay when they leave. The lot has multiple floors, multiple spot types (bike, car, truck, EV) and several entry and exit gates. The system tracks occupancy, computes the fee, and prints/scans the ticket.

Scale to assume

- 10–100 floors per lot, 50–500 spots per floor (medium-sized mall / airport).
- Concurrency: 5–10 entries / exits per second at peak — needs atomic spot assignment.
- In-memory simulation is fine for an interview; persistence is bonus.

2. How Should You Think? (Clarifying Questions)

This question looks small but the interviewer is testing whether you can spot extension points before writing code. Vehicle types, slot types, pricing models, and gates all change over time, so plan for that.

Functional clarifications to ask

- How many vehicle types — bike / car / truck / EV — and can we add more later?
- Are there multiple entry / exit gates? Single ticket dispenser per gate or shared?
- Pricing: flat per hour, slab-based (first 1h free, next 3h cheaper), or vehicle-specific?
- Do we support reservations, VIP / handicap spots, or just first-available?
- Payment: cash, card, UPI, prepaid wallet — and where is it taken (exit gate vs app)?
- Should we support monthly passes / season tickets?

Non-functional clarifications

- Concurrency: two cars arriving at different gates must not claim the same spot.
- Availability: even if the database is down, the gate should let known passes through.
- Extensibility: a new vehicle type should not require changing existing classes (Open–Closed).
- Latency: a barrier should open in under 1 second.

Why ask these? (the signal)

Asking 'how many vehicle types' shows you anticipate inheritance vs strategy. Asking about pricing shows you'll use the Strategy pattern instead of an if-else chain. Asking about gates shows you understand concurrency is a real concern, not theoretical.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Five top-level actors interact: **EntryGate**, **ExitGate**, **ParkingLot** (singleton), **SpotAssigner**, and **FeeCalculator**. EntryGate calls ParkingLot to reserve a Spot atomically and issues a Ticket. ExitGate looks up the Ticket, asks FeeCalculator for the amount, takes payment via a PaymentProcessor, and frees the Spot.

Major components

- ParkingLot — singleton, owns all floors and spots.
- Floor — owns a list of Spots grouped by type.
- Spot — has id, type, status (FREE/OCCUPIED/RESERVED), assigned Vehicle.

- Vehicle — abstract; subclasses Bike, Car, Truck, EVCar.
- Ticket — id, entryTime, spotId, vehicleNumber.
- SpotAssigner — Strategy: NearestSpot, RandomSpot, ByFloorSpot.
- FeeCalculator — Strategy: HourlyFee, FlatFee, SlabFee.
- PaymentProcessor — Strategy: Cash, Card, UPI.

Low-Level Design (LLD)

Use the Strategy pattern for SpotAssigner, FeeCalculator and PaymentProcessor so each can vary without touching ParkingLot. Use Factory for Vehicle creation, Singleton for ParkingLot, and synchronise the spot allocation method to make it thread-safe. Keep Spot.status as an enum; never use boolean flags — you will need RESERVED later.

4. How to Code & Explain (Win the Interviewer)

Open with: *"I'll model this as the lot owning floors, floors owning spots, and use Strategy for the parts that vary — pricing, spot allocation, payment. I'll keep concurrency in mind because two cars at different gates can race for the same spot."* That single sentence gives the interviewer the four signals they want: OOP modelling, design patterns, extensibility, and concurrency awareness.

Draw the class diagram in 4 minutes max. Then walk through entry flow, exit flow, and one edge case (lot full). Resist the urge to over-model — no Address class, no Receipt class. Add only what your requirements need.

When the interviewer says *"now add EV charging spots"*, smile — your Strategy and inheritance setup already supports it. Add ChargingSpot extends Spot, and an EVSpotAssigner that prefers ChargingSpot for EVCar. No existing code changes — that's the win.

> ■ ***Tip:*** *Always mention* ***Open/Closed Principle*** *and* ***Strategy Pattern*** *by name. These two phrases separate junior LLDs from senior ones.*

5. Actual Code (Reference Snippets)

Core domain in Kotlin — Strategy + Singleton + thread-safe park()/unpark()

Kotlin

```
enum class VehicleType { BIKE, CAR, TRUCK, EV }
enum class SpotStatus { FREE, OCCUPIED, RESERVED }

abstract class Vehicle(val number: String, val type: VehicleType)
class Car(number: String) : Vehicle(number, VehicleType.CAR)
class Bike(number: String) : Vehicle(number, VehicleType.BIKE)

data class Spot(
    val id: String,
    val floor: Int,
    val type: VehicleType,
    var status: SpotStatus = SpotStatus.FREE,
    var vehicle: Vehicle? = null
)

interface FeeStrategy { fun calculate(durationMin: Long, v: Vehicle): Double }
class HourlyFee(private val ratePerHour: Map<VehicleType, Double>) : FeeStrategy {
    override fun calculate(durationMin: Long, v: Vehicle): Double {
        val hours = Math.ceil(durationMin / 60.0)
        return hours * (ratePerHour[v.type] ?: 50.0)
    }
}

interface SpotAssigner { fun assign(spots: List<Spot>, v: Vehicle): Spot? }
class NearestSpotAssigner : SpotAssigner {
    override fun assign(spots: List<Spot>, v: Vehicle): Spot? =
```

```
        spots.firstOrNull { it.type == v.type && it.status == SpotStatus.FREE }
}

class ParkingLot private constructor(
    private val spots: MutableList<Spot>,
    private val assigner: SpotAssigner,
    private val fee: FeeStrategy
) {
    private val lock = Any()
    private val tickets = mutableMapOf<String, Ticket>()

    fun park(v: Vehicle): Ticket? = synchronized(lock) {
        val spot = assigner.assign(spots, v) ?: return null
        spot.status = SpotStatus.OCCUPIED; spot.vehicle = v
        val t = Ticket(java.util.UUID.randomUUID().toString(),
                       System.currentTimeMillis(), spot.id, v.number)
        tickets[t.id] = t; return t
    }

    fun unpark(ticketId: String): Double = synchronized(lock) {
        val t = tickets.remove(ticketId) ?: throw IllegalStateException("Bad ticket")
        val spot = spots.first { it.id == t.spotId }
        val dur = (System.currentTimeMillis() - t.entryTime) / 60_000
        spot.status = SpotStatus.FREE; spot.vehicle = null
        return fee.calculate(dur, /* vehicle */ Bike(t.vehicleNumber))
    }
    companion object {
        @Volatile private var inst: ParkingLot? = null
        fun init(s: List<Spot>, a: SpotAssigner, f: FeeStrategy): ParkingLot =
            inst ?: synchronized(this) {
                inst ?: ParkingLot(s.toMutableList(), a, f).also { inst = it }
            }
    }
}
data class Ticket(val id: String, val entryTime: Long,
                  val spotId: String, val vehicleNumber: String)
```

6. Cross-Questions & How to Answer Them

Q: How would you handle two cars arriving at the same instant at different gates?

Synchronise the assign + status-flip block, or use a per-floor lock to reduce contention. In a real backend, use an atomic database update (UPDATE spot SET status='OCCUPIED' WHERE id=? AND status='FREE') and check rowsAffected. Mention that synchronized works for an in-memory simulation but is the bottleneck you'd remove first.

Q: Now add reservations — a user reserves a spot via the app 30 min before arrival.

Add SpotStatus.RESERVED and a Reservation entity with userId, spotId, validFrom, validUntil. A scheduled worker flips expired reservations back to FREE. The assigner skips RESERVED. No change to the public API of park() — that's the strength of the design.

Q: What if the database is down?

Two layers: (a) the in-memory cache of free spots stays authoritative for the next minute, so gates still open; (b) tickets are written to a local file/queue and replayed when the DB comes back. Read availability is fine to be stale; what must never fail is a paid user being denied exit.

Q: Why Strategy and not just polymorphism on Vehicle for pricing?

Pricing changes for reasons unrelated to the vehicle — happy hours, weekend rates, partner discounts. Coupling pricing to Vehicle would force a vehicle change on every promotion. Strategy keeps the axes of change orthogonal.

7. What Better? Alternative Approaches

For a real product: the assigner becomes a query against an indexed table; concurrency goes from synchronized to `SELECT ... FOR UPDATE SKIP LOCKED` on PostgreSQL, or a Redis Lua script.

For massive scale (airports, malls): shard by floor — each floor is its own micro-service with its own lock domain. Reduces contention by 10–100×.

Pricing: move FeeStrategy to a Rules Engine (e.g. Drools) so the business team can change rules without a redeploy.

Observability: emit events ('spot.occupied', 'spot.freed') to Kafka — that gives you analytics, real-time occupancy dashboards, and an audit trail for billing disputes for free.

Key takeaways

- Strategy pattern for the three things that vary: assign, price, pay.
- Singleton for the lot, not for spots — singletons should be wide, not deep.
- Thread-safety is real; mention it even if you don't fully implement it.
- Status is an enum with three states, not a boolean.

Chapter 2: Design an Elevator (Lift) System

Difficulty: Easy | **Tags: LLD OOP State Machine Scheduling**

Schedule one or more elevators efficiently across N floors. Tests state machines and scheduling algorithms.

1. What's the Problem?

Design a control system for a building with multiple elevators and many floors. Users press an up/down button on a floor (external request) or a floor button inside the cabin (internal request). The system must decide which elevator answers which request, move efficiently, and respect direction so users do not get whiplashed.

Scale to assume

- 2–6 elevators in most buildings; up to 100 floors in skyscrapers.
- External + internal requests arrive concurrently — the dispatcher needs a thread-safe queue.
- An interview wants the simulation; a real system needs millisecond response and safety.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Single elevator or a bank of elevators? (Single = easy; bank = scheduling problem.)
- Are there express elevators, service-only, or VIP overrides?
- Door behaviour: auto-close after N seconds, obstacle sensor, 'door open' button?
- Emergency / fire mode — all elevators return to ground?
- Capacity limit — does the cabin refuse to move when overloaded?
- Out-of-service / maintenance state?

Non-functional clarifications

- Safety: doors must never open between floors.
- Fairness: long-waiting users must eventually be served (no starvation).
- Throughput: minimise total wait time across all users — pick a good scheduling algorithm.
- Responsiveness: pressing a button must give visual feedback in < 200 ms.

Why ask these? (the signal)

The 'bank' question is the most important — it changes the problem from a state machine to a scheduling problem. Always ask first.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Each Elevator is a state machine with states {IDLE, MOVING_UP, MOVING_DOWN, DOOR_OPEN, MAINTENANCE}. An **ElevatorController** (the brain) holds all elevators and a request queue. When an external request comes in, the controller asks each elevator for a 'cost' (distance + direction match) and dispatches the cheapest. Internal requests are local to the elevator that received them.

Major components

- Elevator — id, currentFloor, direction, state, requestQueue.
- Button — abstract; HallButton (with floor + direction), CabinButton (with floor).
- Request — sourceFloor, destFloor, direction, timestamp.
- ElevatorController — dispatcher, owns all elevators, runs the scheduling loop.
- Scheduler (strategy) — FCFS, SCAN/elevator-algorithm, LOOK, Nearest-Car (NC).

- Display + Sensor — for the realism layer.

Low-Level Design (LLD)

Use State pattern for the elevator (IdleState, MovingUpState, etc.) so the transition logic is explicit and testable. Use Strategy for the scheduler — start with SCAN (the classic 'elevator algorithm') and mention LOOK as an upgrade. Use Observer so floor displays automatically update when an elevator's currentFloor changes.

4. How to Code & Explain (Win the Interviewer)

Open with: *"This is fundamentally a scheduling problem disguised as a state machine. I'll model each elevator with a State pattern and a Scheduler strategy at the controller level. The classic SCAN algorithm is a good baseline — sweep up, then sweep down, servicing all requests on the way."*

Show one execution trace: 'Elevator at floor 3 going up. Requests: 5↑, 7↑, 2↓. SCAN serves 5, 7, then reverses to 2. FCFS would have gone 5, 2, 7 — much worse.' Concrete examples beat abstract talk.

Mention safety code at the end: *"In real elevators, the door-open check is a hardware interlock, not just software. I'd never trust a software flag for safety-critical operations."* Interviewers love this maturity.

> ■ ***Tip:*** *Convert real time to* `tick()` *in your simulation. You control time; you avoid Thread.sleep and tests run fast. Every senior LLD answer for elevator does this.*

5. Actual Code (Reference Snippets)

Kotlin model — SCAN scheduling baked into the queues, cost-based dispatch for the bank

Kotlin

```kotlin
enum class Direction { UP, DOWN, IDLE }
enum class ElevatorState { IDLE, MOVING, DOOR_OPEN, MAINTENANCE }

data class Request(val source: Int, val dest: Int, val dir: Direction,
                   val time: Long = System.currentTimeMillis())

class Elevator(val id: Int, private val maxFloor: Int) {
    var currentFloor = 0; private set
    var state = ElevatorState.IDLE; private set
    var direction = Direction.IDLE; private set
    val upQueue: java.util.TreeSet<Int> = sortedSetOf()
    val downQueue: java.util.TreeSet<Int> = sortedSetOf(Comparator.reverseOrder())

    fun addStop(floor: Int) {
        if (floor > currentFloor) upQueue.add(floor) else downQueue.add(floor)
    }

    /** One simulation tick: move at most one floor and open door if needed. */
    fun step() {
        when (direction) {
            Direction.UP -> {
                if (upQueue.isEmpty()) { direction = Direction.IDLE; return }
                currentFloor++
                if (currentFloor in upQueue) { upQueue.remove(currentFloor); openDoor() }
                if (upQueue.isEmpty() && downQueue.isNotEmpty()) direction = Direction.DOWN
            }
            Direction.DOWN -> {
                if (downQueue.isEmpty()) { direction = Direction.IDLE; return }
                currentFloor--
                if (currentFloor in downQueue) { downQueue.remove(currentFloor); openDoor() }
                if (downQueue.isEmpty() && upQueue.isNotEmpty()) direction = Direction.UP
            }
            Direction.IDLE -> {
                direction = when {
                    upQueue.isNotEmpty()   -> Direction.UP
                    downQueue.isNotEmpty() -> Direction.DOWN
```

```
                    else                    -> Direction.IDLE
                }
            }
        }
    }
    private fun openDoor() { state = ElevatorState.DOOR_OPEN /* ... close after N ticks */ }
    fun cost(req: Request): Int =                            // smaller = better match
        Math.abs(currentFloor - req.source) +
        (if (direction != Direction.IDLE && direction != req.dir) 1_000 else 0)
}

class ElevatorController(private val elevators: List<Elevator>) {
    fun handleHallCall(req: Request) =
        elevators.minByOrNull { it.cost(req) }?.addStop(req.source)
    fun handleCabinCall(elevatorId: Int, dest: Int) =
        elevators.first { it.id == elevatorId }.addStop(dest)
    fun tick() = elevators.forEach { it.step() }
}
```

6. Cross-Questions & How to Answer Them

Q: Why two queues (up and down) instead of one PriorityQueue?

SCAN serves all up-requests in ascending order while moving up, then all down-requests in descending order while moving down. Two ordered sets (TreeSet) give you exactly that with O(log n) inserts and removeFirst. A single priority queue cannot express 'direction matters'.

Q: How would you reduce average wait time in a bank of elevators during morning rush hour?

Detect the pattern (lots of hall calls on floor 0 going up) and use 'zoning' — assign elevators to floor ranges. Or use 'destination dispatch' where users key in the destination floor at the hall and the system groups passengers, so each elevator stops fewer times. Mention that this is what modern Schindler/Otis systems actually do.

Q: What happens if an elevator fails mid-trip?

Move its pending stops to other elevators (re-dispatch), mark the elevator MAINTENANCE, and alert ops. The simulation supports this because every Request lives in the controller's log, not only in the elevator.

Q: How do you avoid starvation of users on extreme floors?

Add a max-age check to each Request: if the oldest request has waited > threshold, the scheduler boosts its priority by penalising other elevators' cost functions for ignoring it. Or, use LOOK with explicit age-based tie-breaking.

7. What Better? Alternative Approaches

Algorithm upgrade: LOOK is SCAN that turns around when there are no further requests in the current direction — uses less energy.

ML-based dispatch: learn building traffic patterns (morning up, evening down) and pre-position idle elevators on likely floors.

Hardware-friendly: represent the state machine in code that mirrors a PLC ladder logic so the controller is auditable for safety regulators.

Visual / mobile: for an Android-based building app, expose a small REST/MQTT API so phones can 'pre-call' elevators when the user approaches the lobby (use BLE beacons or NFC).

Key takeaways

- State pattern for the elevator, Strategy pattern for the dispatcher.
- Use tick() so you control time and tests are deterministic.
- SCAN is the textbook starting algorithm; mention LOOK as the upgrade.
- Two queues (up + down) is cleaner than one priority queue.

Chapter 3: Design a Chess Game

Difficulty: Easy | **Tags: LLD OOP Polymorphism Game**

Model a chess board, pieces, moves, turns, and termination — the canonical polymorphism test.

1. What's the Problem?

Implement the game of chess: an 8×8 board, 32 pieces (6 types), turn-based play, valid-move rules for each piece, check / checkmate / stalemate detection, castling, en-passant, and pawn promotion. The interviewer rarely expects all of these in 45 minutes — they want you to model cleanly and add the special rules as time allows.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Standard chess only or chess variants (Chess960, three-check)?
- Castling, en passant, pawn promotion — in scope?
- Two-player local, online multiplayer, or human vs computer?
- Move time clock — blitz / classical / no clock?
- Undo / takeback support?
- Resign and draw offers?

Non-functional clarifications

- Move validation must be correct — no illegal moves accepted.
- Latency: in-memory move validation should be sub-millisecond.
- Extensibility: a new variant should not rewrite the base classes.
- Testability: every piece's move logic should be unit-testable in isolation.

Why ask these? (the signal)

Asking about variants and castling tells the interviewer you know inheritance is about reuse, not just hierarchy. It also bounds the scope.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Three top-level objects: **Game** (the conductor — keeps players, current turn, history), **Board** (8×8 grid of Squares), and **Piece** (abstract — King, Queen, Rook, Bishop, Knight, Pawn). Each Piece implements `validMoves(Board, Square): List<Square>`. Game.move(from, to) validates, applies, switches turn, and checks for terminal state.

Major components

- Game — players, currentTurn, board, moveHistory, status.
- Board — squares: Array<Array<Square>>, helper methods isInCheck, isCheckmate.
- Square — row, col, piece.
- Piece — abstract; subclasses for each type. Color enum WHITE/BLACK.
- Move — from, to, piece, captured, special flags (castling, enpassant, promotion).
- Player — color, name, isHuman.

Low-Level Design (LLD)

Polymorphism is the star here — Piece.validMoves is overridden by every subclass. King knows about castling, Pawn knows about double-push and en-passant. Use a Move object instead of two coordinates so you can carry

metadata. Keep board mutation inside Board.applyMove so the Game only orchestrates. Use the Memento pattern for undo.

4. How to Code & Explain (Win the Interviewer)

Open with: *"I'll use polymorphism so each piece owns its own movement rules. The Board doesn't know how a Knight moves — the Knight does. That's the Single Responsibility Principle in action."*

After modelling, walk through a single move trace end-to-end: 'White plays Nf3. Game.move(g1, f3). Board looks up the piece, asks Knight.validMoves, checks f3 is in the list, applies, switches turn, checks if Black king is in check — no — record in history.'

Drop the bonus: *"For check detection, I'll compute it lazily — after a move I generate the opponent's threatened squares and check if the king's square is in that set. For checkmate, additionally check the king has no escape and no piece can block or capture the threat."* Showing you know the algorithm — not just the rule — is the senior signal.

> ■ ***Tip:*** *Implement the King last. Pawn and Knight have simple, isolated rules — get them working, get your scaffolding right, then tackle the King with castling.*

5. Actual Code (Reference Snippets)

Kotlin model — abstract Piece with polymorphic validMoves; Move object for undo & special rules

Kotlin

```kotlin
enum class Color { WHITE, BLACK }
abstract class Piece(val color: Color) {
    abstract fun validMoves(board: Board, from: Square): List<Square>
    fun isOpponent(other: Piece?) = other != null && other.color != color
}

class Square(val row: Int, val col: Int, var piece: Piece? = null) {
    val inBounds get() = row in 0..7 && col in 0..7
}

class Knight(color: Color) : Piece(color) {
    private val offsets = listOf(2 to 1, 1 to 2, -1 to 2, -2 to 1,
                                 -2 to -1, -1 to -2, 1 to -2, 2 to -1)
    override fun validMoves(board: Board, from: Square): List<Square> =
        offsets.mapNotNull { (dr, dc) ->
            board.squareOrNull(from.row + dr, from.col + dc)
                ?.takeIf { it.piece == null || it.piece!!.color != color }
        }
}

class Rook(color: Color) : Piece(color) {
    override fun validMoves(board: Board, from: Square): List<Square> {
        val res = mutableListOf<Square>()
        for ((dr, dc) in listOf(1 to 0, -1 to 0, 0 to 1, 0 to -1)) {
            var r = from.row + dr; var c = from.col + dc
            while (true) {
                val s = board.squareOrNull(r, c) ?: break
                if (s.piece == null) { res.add(s) }
                else { if (s.piece!!.color != color) res.add(s); break }
                r += dr; c += dc
            }
        }
        return res
    }
}

data class Move(val from: Square, val to: Square, val piece: Piece,
                val captured: Piece? = null,
```

```
                val isCastle: Boolean = false,
                val isEnPassant: Boolean = false,
                val promotion: Piece? = null)

class Board {
    private val grid = Array(8) { r -> Array(8) { c -> Square(r, c) } }
    fun squareOrNull(r: Int, c: Int): Square? =
        if (r in 0..7 && c in 0..7) grid[r][c] else null
    fun applyMove(m: Move) {
        m.from.piece = null
        m.to.piece = m.promotion ?: m.piece
    }
}

class Game(val white: Player, val black: Player) {
    val board = Board()
    private val history = ArrayDeque<Move>()
    private var turn = Color.WHITE
    fun move(from: Square, to: Square): Boolean {
        val piece = from.piece ?: return false
        if (piece.color != turn) return false
        if (to !in piece.validMoves(board, from)) return false
        val move = Move(from, to, piece, captured = to.piece)
        board.applyMove(move); history.addLast(move)
        turn = if (turn == Color.WHITE) Color.BLACK else Color.WHITE
        return true
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Where would you put castling logic — in King or in Game?

Move-generation belongs to King.validMoves (the king is the piece that moves two squares), but the consequence (also moving the rook) belongs to Board.applyMove because it touches two squares. Split responsibility: King produces a Move with isCastle=true; Board sees the flag and moves the rook too.

Q: How do you detect check?

After every move, for the side that just moved, generate the opponent's union of validMoves; if it includes the opponent's king square, the opponent is in check. Checkmate = in check AND every legal move for the opponent still leaves them in check (try each, undo, repeat).

Q: Won't undo for checkmate detection be slow?

Polynomial but cheap — at most ~30 pieces × ~28 candidate moves = ~840 boards. With bitboards you'd do it in microseconds. For an interview that runs at human speed, the clarity of try-undo beats premature optimisation.

Q: How would you support Chess960 (random back rank)?

Game has an initial position factory. Replace the standard layout with the variant's. King's castling rule needs a small tweak because rooks aren't on a1/h1 — store castling rights as a pair of rook squares per side instead of hard-coding.

7. What Better? Alternative Approaches

Bitboards — represent the board as 64-bit integers; one bitboard per piece type per color. Move generation becomes bitwise ops. 100×+ faster — useful when adding an AI.

Online play: wrap Game in a Server class; sync via WebSocket; use a Move oracle on the server (never trust the client) so cheating is impossible.

Engine integration: expose UCI (Universal Chess Interface) so any engine (Stockfish) can play your AI side.

Persistence: store games in PGN (Portable Game Notation) — a human-readable text format that's also the standard for chess databases.

Key takeaways

- Polymorphism: each Piece owns its movement.
- Move object > coordinate pair — needed for undo and special rules.
- Compute check by generating opponent threats; checkmate by try-undo.
- Bitboards are the right next step when performance matters.

Chapter 4: Design a Library Management System

Difficulty: Easy | **Tags: LLD OOP CRUD**

Members borrow books, librarians issue and return them. A pure object-modelling question.

1. What's the Problem?

A library has Books (multiple copies of each title), Members, and Librarians. Members can search by title, author, or subject, borrow up to N books at a time for M weeks, and pay a fine if they return late. Librarians can add or remove books, register members, and process issues / returns.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Multiple copies per title? (Yes — usually 'BookItem' distinct from 'Book'.)
- Borrow limit per member? Loan duration? Fine rate?
- Reservation: can a member hold a book that's currently loaned?
- Categories / subjects, search facets?
- Member roles — student, faculty, public — with different limits?
- Inter-library loans, e-books?

Non-functional clarifications

- Two members can't borrow the same physical copy at the same time.
- Search must be fast — index by title, author, ISBN, subject.
- Data durability — losing a member's loan history is bad.
- Audit trail for compliance.

Why ask these? (the signal)

The 'multiple copies' question is the crux: it forces the Book vs BookItem split which is the senior insight.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Two clean domain layers. The **catalogue** answers 'is this title available, who's the author' — that's Book + Author + Subject. The **circulation** answers 'who has copy #347, when is it due' — that's BookItem + Loan + Member + Fine. Keeping the two separated makes search (a read-heavy operation) and lending (a write-heavy one) independently scalable later.

Major components

- Book — title, ISBN, authors, subjects, publisher. The conceptual title.
- BookItem — barcode, status (AVAILABLE/LOANED/LOST/RESERVED), location.
- Member — id, name, contact, role, currentLoans.
- Loan — bookItemId, memberId, issueDate, dueDate, returnedDate.
- Librarian — issueBook(), returnBook(), addBook(), registerMember().
- SearchService — by title / author / ISBN / subject.

Low-Level Design (LLD)

Apply the Repository pattern around each entity (BookRepository, LoanRepository) so the rest of the code is agnostic of storage. Use a Strategy for FineCalculator (per day, per week, free for students).

4. How to Code & Explain (Win the Interviewer)

Open with the **Book vs BookItem** distinction — that one move signals you understand domain modelling. *"A title is a concept; a physical copy is a resource. They have different lifecycles, so they're different classes."*

Walk through one issue flow: *"Member scans card and book. Librarian calls issueBook(memberId, barcode). System checks the member's loan limit, the item's status, creates a Loan, flips the item to LOANED, sets dueDate."*

Don't forget the fine: *"On return, if today > dueDate, FineCalculator.calculate(daysLate, member.role) produces a fine, attached to the loan. The member can't borrow again until cleared."*

> ■ ***Tip:*** *Mention that you would put the loan limit, duration, and fine rate in a* ***Policy*** *object — not hard-code them. Configurable policies are a senior signal.*

5. Actual Code (Reference Snippets)

Kotlin — Book vs BookItem split, configurable LoanPolicy, simple issue/return

Kotlin

```kotlin
enum class ItemStatus { AVAILABLE, LOANED, RESERVED, LOST }

data class Book(val isbn: String, val title: String,
                val authors: List<String>, val subjects: List<String>)

data class BookItem(val barcode: String, val isbn: String,
                    var status: ItemStatus = ItemStatus.AVAILABLE,
                    val location: String)

data class Member(val id: String, val name: String, val role: Role,
                  val activeLoans: MutableList<Loan> = mutableListOf()) {
    fun canBorrow(policy: LoanPolicy) = activeLoans.size < policy.maxBooks(role)
}
enum class Role { STUDENT, FACULTY, PUBLIC }

data class Loan(val itemBarcode: String, val memberId: String,
                val issueDate: java.time.LocalDate,
                val dueDate: java.time.LocalDate,
                var returnedDate: java.time.LocalDate? = null) {
    fun isOverdue(today: java.time.LocalDate) =
        returnedDate == null && today.isAfter(dueDate)
}

data class LoanPolicy(val perRole: Map<Role, Pair<Int, Int>>) {
    fun maxBooks(r: Role) = perRole[r]?.first ?: 2
    fun durationWeeks(r: Role) = perRole[r]?.second ?: 2
}

class Library(private val policy: LoanPolicy) {
    private val books = mutableMapOf<String, Book>()
    private val items = mutableMapOf<String, BookItem>()
    private val members = mutableMapOf<String, Member>()
    private val loans = mutableListOf<Loan>()

    fun issue(memberId: String, barcode: String,
              today: java.time.LocalDate = java.time.LocalDate.now()): Loan {
        val m = members[memberId] ?: error("Unknown member")
        val it = items[barcode] ?: error("Unknown item")
        require(it.status == ItemStatus.AVAILABLE) { "Not available" }
        require(m.canBorrow(policy)) { "Loan limit reached" }
        val due = today.plusWeeks(policy.durationWeeks(m.role).toLong())
        val loan = Loan(barcode, memberId, today, due)
        it.status = ItemStatus.LOANED; m.activeLoans.add(loan); loans.add(loan)
        return loan
```

```
    }

    fun returnItem(barcode: String, fineCalc: (Long) -> Double,
                   today: java.time.LocalDate = java.time.LocalDate.now()): Double {
        val it = items[barcode] ?: error("Unknown item")
        val loan = loans.first { it.itemBarcode == barcode && it.returnedDate == null }
        loan.returnedDate = today
        it.status = ItemStatus.AVAILABLE
        members[loan.memberId]?.activeLoans?.remove(loan)
        val daysLate = java.time.temporal.ChronoUnit.DAYS
            .between(loan.dueDate, today).coerceAtLeast(0)
        return fineCalc(daysLate)
    }
}
```

6. Cross-Questions & How to Answer Them

Q: How would you handle search efficiently?

Don't scan the books list. Keep three in-memory indexes (Map<String, Set<BookId>> by title/author/subject) updated on add/remove. For real scale, plug Elasticsearch behind the same SearchService interface.

Q: How do you prevent the same copy being issued twice in a multi-librarian system?

Pessimistic lock on the BookItem row during issue, or use an optimistic version check. In the simulation, synchronize on items[barcode]. Always mention which approach you'd take at scale and why.

Q: Should a Book have a status field?

No — Book is the title (a concept). Status belongs to BookItem (the physical thing). Conflating them means you can't represent the fact that one library has 3 copies of 'Effective Java' with 2 loaned and 1 available.

Q: How does reservation interact with the loan flow?

Add ItemStatus.RESERVED and a Reservation entity. On return, if a reservation exists, the item goes to RESERVED for the reserver, not AVAILABLE. Add a TTL — the reserver has X hours to collect before the reservation expires.

7. What Better? Alternative Approaches

Move to a database: the in-memory maps become tables, the Repository interface stays. The rest of the code doesn't notice.

Add events: emit book.issued / book.returned events so analytics, late-fee emails, and reservation-fulfillment notifications come for free.

Member-facing app: wrap with a REST layer; Android app reads catalogue and personal loans; payments for fines via gateway.

E-books / DRM: a DigitalBookItem subclass with a download URL and expiry. Same Loan model, different fulfilment.

Key takeaways

- Book vs BookItem is the senior insight.
- Policy objects beat hard-coded constants.
- Repository pattern keeps storage swappable.
- Events make analytics and notifications easy.

Chapter 5: Design an ATM

Difficulty: Easy | **Tags: LLD State Finance**

A state-machine over a card session: insert → authenticate → choose → dispense → eject. Concurrency-safe cash drawer.

1. What's the Problem?

Model an ATM. A user inserts a card, enters a PIN, chooses between balance inquiry, withdrawal, deposit, or transfer, and the machine talks to a Bank service for authentication and balance. The cash drawer holds notes in fixed denominations and must dispense the right combination.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Operations supported — balance, withdraw, deposit, transfer, mini-statement?
- Multi-currency? Multi-denomination?
- Card types — debit, credit, foreign? PIN tries before block?
- Receipts: print, email, none?
- Idle timeout when the user walks away?

Non-functional clarifications

- Cash drawer is a shared resource — two parallel withdrawals can't double-spend.
- Auditability — every transaction logged immutably.
- Availability — if network is down, can the machine do offline balance withdrawal up to a small cap?
- Security — PIN encrypted, never stored on the device.

Why ask these? (the signal)

Asking about denominations and PIN attempts gives you the obvious extension points and the security cred.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

The ATM is a state machine. States: IDLE, CARD_INSERTED, AUTHENTICATED, CHOOSING, PROCESSING, EJECTING. Transitions are driven by user events (card-in, pin-entered, choice, ok, cancel) or system events (auth-ok, auth-failed, dispense-done). Each transaction is a Command (Withdraw, Deposit, Transfer) submitted to the BankService.

Major components

- ATM — owns CardReader, Keypad, Screen, CashDispenser, DepositSlot, Printer.
- Session — current card, account, state, transaction history.
- BankService — authenticate(cardNumber, pinHash), getBalance, debit, credit, transfer.
- CashDispenser — denominations stock, computeDispenseCombination(amount).
- Transaction — abstract; subclasses for each operation.

Low-Level Design (LLD)

State pattern for the session. Command pattern for transactions — easy to add new ones (FundTransfer, BillPay). The cash dispenser solves a small change-making problem: greedy by largest denomination first works for typical ATM denomination sets but you'd note it can fail in pathological cases — DP is safer.

4. How to Code & Explain (Win the Interviewer)

Open with: *"I'll model the ATM as a state machine and each user request as a Command sent to the bank. This separates 'what the user is doing' from 'what the bank is doing about it' — critical because the bank service is remote and unreliable."*

Demonstrate the dispenser: *"To dispense ■1450 from {500, 200, 100, 50}: greedy gives me 500+500+200+200+50 = 5 notes. With a 200 stock-out the algorithm should fall back to DP to find an alternative."*

Bring up timeouts: *"If the user goes to AUTHENTICATED and doesn't act for 30 seconds, I auto-eject the card. State transitions are time-bounded."*

> ■ ***Tip:*** *Talk about* ***idempotency*** *— if the network call to the bank fails after debiting, the retry must not double-debit. Use a transaction ID generated on the ATM and check on the bank side. This is the senior signal in any payment LLD.*

5. Actual Code (Reference Snippets)

ATM with explicit states, idempotent bank.debit via txnId, two-phase dispense (plan → commit)

Kotlin

```
enum class AtmState { IDLE, CARD_IN, AUTHENTICATED, PROCESSING, EJECTING, OUT_OF_SERVICE }

interface BankService {
    fun authenticate(card: String, pin: String): Boolean
    fun balance(card: String): Long
    fun debit(card: String, amount: Long, txnId: String): Boolean
    fun credit(card: String, amount: Long, txnId: String): Boolean
}

class CashDispenser(private val stock: MutableMap<Int, Int>) {  // denom -> count
    private val lock = Any()
    /** Returns map of denom→count if possible, else null. */
    fun plan(amount: Int): Map<Int, Int>? = synchronized(lock) {
        val result = mutableMapOf<Int, Int>()
        var remaining = amount
        for (denom in stock.keys.sortedDescending()) {
            val available = stock[denom] ?: 0
            val needed = remaining / denom
            val used = minOf(needed, available)
            if (used > 0) { result[denom] = used; remaining -= used * denom }
        }
        return if (remaining == 0) result else null      // not enough notes
    }
    fun commit(plan: Map<Int, Int>) = synchronized(lock) {
        plan.forEach { (denom, count) -> stock[denom] = stock[denom]!! - count }
    }
}

class Atm(private val bank: BankService, private val dispenser: CashDispenser) {
    var state = AtmState.IDLE; private set
    private var card: String? = null

    fun insertCard(c: String) { card = c; state = AtmState.CARD_IN }
    fun enterPin(pin: String): Boolean {
        check(state == AtmState.CARD_IN)
        return if (bank.authenticate(card!!, pin)) { state = AtmState.AUTHENTICATED; true }
        else { ejectCard(); false }
    }
    fun withdraw(amount: Int): Boolean {
        check(state == AtmState.AUTHENTICATED); state = AtmState.PROCESSING
        val plan = dispenser.plan(amount) ?: run { state = AtmState.AUTHENTICATED; return false }
        val txnId = java.util.UUID.randomUUID().toString()
        if (!bank.debit(card!!, amount.toLong(), txnId)) {
            state = AtmState.AUTHENTICATED; return false
        }
```

```
        dispenser.commit(plan)         // commit only after bank confirms
        ejectCard(); return true
    }
    fun ejectCard() { card = null; state = AtmState.IDLE }
}
```

6. Cross-Questions & How to Answer Them

Q: Why do you commit on the dispenser only after the bank debit succeeds?

Order matters. If you dispense first and the bank then fails, you've handed out cash for free. By committing dispenser stock after the bank confirms, the only race window is 'bank debited but ATM didn't physically dispense' — which is caught by the reconciliation job and refunded.

Q: How do you prevent two parallel withdrawals from double-spending the same notes?

The dispenser's plan + commit are inside a lock or atomic database update. In production, the cash drawer is physical — only one transaction can be processing at a time, so the state machine already enforces serial access. The bank itself must independently enforce its own balance with row locks.

Q: Network goes down between debit and dispense — what happens?

Worst case: bank thinks debit succeeded, ATM crashes, user sees no cash. On startup, the ATM reads its pending-transactions log and asks the bank to reconcile by txnId. If bank says debited and ATM says not dispensed, automatic credit-back.

Q: How do you handle PIN attempts?

Track failed attempts per card on the bank side, not on the ATM (because the user could try multiple ATMs). After 3 failed attempts the card is blocked. The ATM swallows the card and shows 'contact bank'.

7. What Better? Alternative Approaches

Card-less withdrawal: user requests cash in the bank app, gets a 6-digit OTP, types it at the ATM. The 'card' is replaced by the OTP.

Biometric auth: fingerprint reader replaces PIN; same state machine.

EMV chip + crypto: instead of trusting the magnetic stripe, the chip does a mutual auth with the bank — more secure.

Observability: heart-beat every 30 seconds with cash levels — bank ops can pre-emptively refill before stock-out.

Key takeaways

- ATM = state machine + Command pattern.
- Idempotent transaction IDs are non-negotiable.
- Order of operations matters: bank-debit before cash-commit, not after.
- Plan-then-commit pattern protects shared resources.

Chapter 6: Design a Vending Machine

Difficulty: Easy | **Tags: LLD State Inventory**

A miniature ATM. Tests the State pattern more directly than any other classic LLD.

1. What's the Problem?

Build a vending machine. The user inserts coins, chooses a product, the machine dispenses the product and returns change. Inventory of products and of coins is finite.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Coin denominations accepted? Notes?
- Refund button (cancel before selecting)?
- Product capacity per slot, sold-out behaviour?
- Multiple of same product purchase?
- Card payment too?

Non-functional clarifications

- Concurrency between the coin slot, the product release, and the change return.
- Restock workflow — admin mode.
- Recover after power loss without losing user's inserted money.

Why ask these? (the signal)

The change-making sub-problem is the only tricky algorithmic bit — flag it early.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

States: IDLE, HAS_MONEY, DISPENSING, OUT_OF_STOCK, SERVICE. Transitions on insertCoin, selectProduct, cancel, restock, powerOn/Off. The **Inventory** tracks products and coins; the **ChangeMaker** computes change from the coin inventory.

Major components

- VendingMachine — current state, currentBalance, currentSelection.
- Product — id, name, price, stock.
- Inventory — productMap, coinMap.
- ChangeMaker — strategy: Greedy, DP.
- PaymentGateway — abstract; CoinPayment, CardPayment.

Low-Level Design (LLD)

Use the State pattern formally — each state is a class with handle(event) methods. Cleaner than a giant switch. ChangeMaker is a Strategy because greedy can fail with weird denomination sets. Inventory is a separate service so the same machine class can be tested with a fake.

4. How to Code & Explain (Win the Interviewer)

Open with: *"I'll use the State pattern explicitly — IdleState, HasMoneyState, DispensingState. Each handles only its valid events; everything else throws. That guarantees illegal transitions are impossible by construction."*

Mention the change-making nuance — for {1, 5, 10, 25} greedy works; for {1, 3, 4} greedy fails for amount 6 (4+1+1=3 coins vs 3+3=2 coins). Show you know when the textbook answer is wrong.

Wrap up: *"If the user inserts coins and then unplug happens, on power-on the machine reads its log, finds the unconsumed balance, and refunds at the coin slot."*

> ■ ***Tip:*** *Talk about coin jams. Mention that you'd have a sensor + retry + alert path. Hardware realism impresses.*

5. Actual Code (Reference Snippets)

Sealed-interface State pattern in Kotlin — illegal transitions throw at runtime

Kotlin

```kotlin
sealed interface State {
    fun insertCoin(c: Int, m: VendingMachine): State = error("Not allowed in $this")
    fun select(productId: String, m: VendingMachine): State = error("Not allowed in $this")
    fun cancel(m: VendingMachine): State = error("Not allowed in $this")
}
object Idle : State {
    override fun insertCoin(c: Int, m: VendingMachine): State {
        m.balance += c; return HasMoney
    }
}
object HasMoney : State {
    override fun insertCoin(c: Int, m: VendingMachine): State { m.balance += c; return this }
    override fun cancel(m: VendingMachine): State {
        m.returnCoins(m.balance); m.balance = 0; return Idle
    }
    override fun select(productId: String, m: VendingMachine): State {
        val p = m.inventory.product(productId) ?: error("Unknown")
        require(p.stock > 0) { "Out of stock" }
        require(m.balance >= p.price) { "Need more coins" }
        val change = m.balance - p.price
        val plan = m.changeMaker.plan(change, m.inventory.coinStock())
            ?: run { m.returnCoins(m.balance); m.balance = 0
                     return Idle.also { println("Cannot give change") } }
        p.stock -= 1; m.inventory.removeCoins(plan); m.dispense(p); m.returnCoins(plan)
        m.balance = 0; return Idle
    }
}

class VendingMachine(val inventory: Inventory, val changeMaker: ChangeMaker) {
    var state: State = Idle
    var balance = 0
    fun insertCoin(c: Int) { state = state.insertCoin(c, this) }
    fun select(id: String) { state = state.select(id, this) }
    fun cancel() { state = state.cancel(this) }
    fun returnCoins(amount: Int) { /* drop coins */ }
    fun returnCoins(plan: Map<Int, Int>) { /* drop specific coins */ }
    fun dispense(p: Product) { /* release product */ }
}
```

6. Cross-Questions & How to Answer Them

Q: Sealed interface vs enum for the state — why?

Enum forces a single 'when' statement everywhere. Sealed interface lets each state have its own method bodies — each state knows what it allows. Adding a new state doesn't touch existing ones (Open/Closed).

Q: What if the machine cannot make change?

Two strategies: (a) refuse the sale and refund (safer), (b) round down and charge less (kinder to user but you lose money). Production machines display 'EXACT CHANGE ONLY' when their coin inventory is too low for typical change amounts. Surface it on the UI.

Q: How would you support card payment alongside coins?

Make Payment a strategy: CoinPayment.add(coin), CardPayment.charge(amount). The state still says 'do I have enough?' — the strategy answers.

Q: A coin gets stuck in the slot mid-transaction. What does software do?

Sensor fires a jam event; state moves to SERVICE; current balance is logged so admin can refund the user. UI shows 'Machine is out of service'.

7. What Better? Alternative Approaches

Cloud-connected: machine reports inventory & sales hourly; route trucks restock the right machines first.

Dynamic pricing: raise the soda price on hot days. Trivial change if pricing is a strategy.

Phone tap-to-pay: add an NFC reader and a small embedded auth flow — replaces the coin path.

Anti-theft: on shock detection, lock the machine and alert ops.

Key takeaways

- State pattern is the right tool when the question literally says 'states'.
- Greedy change-making works for standard denoms, fails on adversarial ones — mention DP.
- Refund-on-error is the safe default.

Chapter 7: Design Snake & Ladder

Difficulty: Easy | **Tags: LLD OOP Game**

A simple board-game LLD — perfect for warming up. Tests turn order, board state, and small extensions.

1. What's the Problem?

Two to four players take turns rolling a die. The first to land exactly on square 100 wins. Snakes pull a player down to a lower square, ladders push them up. Rolling a 6 typically gives an extra turn.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- How many players (2–4 standard)?
- Board size (10×10 standard, configurable?)
- Number of dice — one die or two?
- Extra turn on 6 — yes or no?
- Multiple games in parallel?
- Win on exact 100 (overshoot bounces back) or any 100+?

Non-functional clarifications

- Simulation should be deterministic with a seeded RNG for testing.
- Game state must be serialisable for save/resume.

Why ask these? (the signal)

Asking about the die count and extra turn shows you know small rule changes are common — they're separate strategies.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Game orchestrates: board, players, turn order. **Board** knows the snakes and ladders. **Die** generates rolls. On Game.play() it loops until someone wins. A single 'turn' rolls the die, moves the player, applies snake/ladder, checks win.

Major components

- Game — players, board, dice, currentTurn, winner.
- Board — size, snakes: Map<Int,Int>, ladders: Map<Int,Int>.
- Player — id, name, position.
- Die — sides, roll() (seeded RNG injected).
- RuleSet — strategy: ClassicRules, BlitzRules (two dice), KidRules (no overshoot).

Low-Level Design (LLD)

Use Strategy for the RuleSet so 'extra turn on 6', 'two dice', 'overshoot rule' are pluggable. Use Observer to notify any UI / logger after every move. Keep the Die behind an interface so tests can inject a fake.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is small but real interview gold for time management — I'll model the must-haves in 5 minutes and use the rest to add rule variants via Strategy."*

Demonstrate testability: *"With a FakeDie returning [4, 6, 2] I can write a deterministic test for: roll 4, lands on a ladder at 6 → 26, etc."*

Mention concurrency only if asked — usually a single thread is fine. If multi-game, each Game is independent and lockless.

> ■ ***Tip:*** *Always inject the Die (or any RNG) — never instantiate it inside Game. This single habit makes every LLD testable.*

5. Actual Code (Reference Snippets)

Snake & Ladder with injectable Die and pluggable RuleSet

Kotlin

```
interface Die { fun roll(): Int }
class StandardDie(private val random: java.util.Random = java.util.Random()) : Die {
    override fun roll() = random.nextInt(6) + 1
}

data class Player(val id: String, val name: String, var position: Int = 0)

class Board(val size: Int = 100,
            private val snakes: Map<Int, Int>,
            private val ladders: Map<Int, Int>) {
    /** Apply snake/ladder and return new position. */
    fun finalPosition(p: Int): Int {
        ladders[p]?.let { return it }
        snakes[p]?.let { return it }
        return p
    }
}

interface RuleSet {
    fun extraTurn(roll: Int): Boolean
    fun applyMove(current: Int, roll: Int, board: Board): Int
}
class ClassicRules : RuleSet {
    override fun extraTurn(roll: Int) = roll == 6
    override fun applyMove(current: Int, roll: Int, board: Board): Int {
        val candidate = current + roll
        if (candidate > board.size) return current   // can't overshoot
        return board.finalPosition(candidate)
    }
}

class Game(val players: List<Player>, val board: Board,
           val die: Die, val rules: RuleSet) {
    fun play(): Player {
        var idx = 0
        while (true) {
            val p = players[idx]
            do {
                val r = die.roll()
                p.position = rules.applyMove(p.position, r, board)
                if (p.position == board.size) return p
            } while (rules.extraTurn(r))
            idx = (idx + 1) % players.size
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: How would you support save / resume?

Serialise Game state: list of (playerId, position) and currentTurn. JSON is fine. On resume, reconstruct Game with the same Board + Rules.

Q: How would you make it multiplayer online?

Game runs on a server. Players send roll commands over WebSocket; server enforces turn order; broadcasts state diffs. The Game class doesn't change — only the transport does.

Q: What if a snake's head is on a ladder's bottom — infinite loop?

Detect cycles when building the Board — finalPosition should be applied at most once per move, not transitively. Spec the rule explicitly.

Q: Add a third die rule that lets the player choose to use a 6 immediately or save it for later.

A new RuleSet implementation with a Player state extension (savedSixes: Int). Public interfaces of Game and Board don't change. That's the strength of the Strategy split.

7. What Better? Alternative Approaches

Online persistence: store games in Redis with TTL — friends resume up to a week later.

Animation: emit a 'move' event per square so the UI can animate the token rather than teleport.

Tournament mode: a wrapper class manages brackets and feeds winners into the next match.

Key takeaways

- Always inject RNG.
- Use Strategy for rule variants — interviewers love adding rules mid-conversation.
- Game.play() should be unit-testable with a fixed dice sequence.

Chapter 8: Design Tic-Tac-Toe

Difficulty: Easy | **Tags: LLD Game Win Detection**

The smallest LLD that still rewards good modelling — and the win-detection algorithm is a nice optimisation chat.

1. What's the Problem?

Two players alternately place X and O on a 3×3 (or N×N) board. First to align N in a row, column, or diagonal wins. Draw if board fills without a winner.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Standard 3×3 or generalised to N×N with K in a row?
- Human vs human, vs simple AI, or both?
- Undo last move?
- Multiple games per match (best of 3)?

Non-functional clarifications

- Win detection should be O(N) per move, not $O(N^2)$.
- Game state serialisable.

Why ask these? (the signal)

Win-detection efficiency is the only place to score senior points — start the conversation on it.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Game owns a Board and two Players. After each Board.place(row, col, symbol), call Board.checkWinAt(row, col) which returns the winning line or null. Keep row/col/diag/anti-diag counters per side so detection is O(1) amortised.

Major components

- Game — players, board, currentTurn, status (IN_PROGRESS, WIN_X, WIN_O, DRAW).
- Board — size N, K-in-a-row, grid, rowCount[N], colCount[N], diagCount, antiDiagCount.
- Player — symbol, name.

Low-Level Design (LLD)

Two tricks: (1) keep per-row, per-column, per-diagonal counters and increment / decrement on each move; check whether any counter hits K. (2) If undo needed, also keep a move stack.

4. How to Code & Explain (Win the Interviewer)

Open with the O(1) trick: *"Naive scan after every move is $O(N^2)$. With four counter arrays (rows, cols, diag, antidiag) keyed by player, every move is O(1) and a counter equal to K means win."*

Generalise: *"For N×N with K-in-a-row, I'd use a map keyed by (row, col-row, col+row) to count consecutive marks of the current symbol; if any hits K, that's the winning line."*

> ■ ***Tip:*** *If asked for an AI: mention* ***minimax*** *with alpha-beta pruning — 3×3 is small enough that it solves perfectly in microseconds.*

5. Actual Code (Reference Snippets)

O(1) win detection via per-row/col/diag counters

Kotlin

```
enum class Symbol { X, O, EMPTY }
data class Player(val name: String, val symbol: Symbol)

class Board(val n: Int = 3, val k: Int = 3) {
    private val grid = Array(n) { Array(n) { Symbol.EMPTY } }
    // counters keyed by symbol
    private val rows = mapOf(Symbol.X to IntArray(n), Symbol.O to IntArray(n))
    private val cols = mapOf(Symbol.X to IntArray(n), Symbol.O to IntArray(n))
    private val diag = mutableMapOf(Symbol.X to 0, Symbol.O to 0)
    private val anti = mutableMapOf(Symbol.X to 0, Symbol.O to 0)
    var moves = 0; private set

    fun place(r: Int, c: Int, s: Symbol): Boolean {
        require(grid[r][c] == Symbol.EMPTY) { "Occupied" }
        grid[r][c] = s; moves++
        rows[s]!![r]++; cols[s]!![c]++
        if (r == c) diag[s] = diag[s]!! + 1
        if (r + c == n - 1) anti[s] = anti[s]!! + 1
        return rows[s]!![r] == k || cols[s]!![c] == k ||
               diag[s] == k || anti[s] == k
    }
    val isDraw get() = moves == n * n
}

class Game(val px: Player, val po: Player, n: Int = 3) {
    private val board = Board(n)
    private var turn = px
    fun move(r: Int, c: Int): String {
        val won = board.place(r, c, turn.symbol)
        if (won) return "${turn.name} wins"
        if (board.isDraw) return "Draw"
        turn = if (turn == px) po else px
        return "Continue"
    }
}
```

6. Cross-Questions & How to Answer Them

Q: How would you generalise to gomoku (15×15, 5 in a row)?

The counter trick still works but you need 'consecutive' rather than 'total': maintain longest run per (row, col, diag). On place, check the 4 directions for runs through (r,c) — still O(N) per move.

Q: How would you add an undo?

Move stack; on undo, pop the last move, set grid[r][c] = EMPTY, and decrement the relevant counters. Symmetry of operations.

Q: Best AI for 3×3?

Minimax: the game tree has fewer than 9! = 362,880 nodes; pruning brings it well under a millisecond. A perfect player can guarantee at-least-draw.

Q: How would you make it multiplayer online with bots?

Same Game class on a server; a Player interface implemented by Human (waits for socket input) or Bot (computes minimax). The Game doesn't care.

7. What Better? Alternative Approaches

Add MCTS for boards too big for minimax — useful for N=10+.

Web/mobile UI with Compose; emit 'move' events to allow spectators.

Key takeaways

- O(1) win detection via counters is the senior trick.
- Strategy for AI lets you swap implementations without touching Game.
- Generalise to N,K early in the discussion to avoid hard-coding 3.

Chapter 9: Design Splitwise

Difficulty: Easy | **Tags: LLD Domain Modeling Algorithms**

Friends share expenses; the app computes who owes whom. Tests graph thinking and split strategies.

1. What's the Problem?

Build a Splitwise clone. Users add expenses to a group — 'Alice paid ■600 for lunch, split equally among 3' or 'unequally' or 'by share/percentage'. The app keeps a running tally of who owes whom and simplifies the debts.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Split types — equal, exact, percentage, share — all needed?
- Multiple currencies?
- Groups vs friend-pairs vs both?
- Simplify debts (minimise number of transactions)?
- Settle-up partial payments?
- Comments / activity feed on each expense?

Non-functional clarifications

- Consistency: total paid == total owed across all involved users for every expense.
- Concurrency: two members editing the same expense simultaneously.
- Audit: every change is logged.

Why ask these? (the signal)

Asking about split types tells the interviewer you need a Strategy. Asking about simplification flags the algorithmic part.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Three core entities: **User**, **Expense** (amount, paidBy, splits), **BalanceSheet** (per-user, per-counterparty net balance). Adding an expense updates the balance sheet for everyone involved. A **Simplifier** rebalances the graph to minimise transactions.

Major components

- User — id, name.
- Expense — amount, paidBy, currency, splitStrategy, participants, createdAt.
- SplitStrategy — EqualSplit, ExactSplit, PercentSplit, ShareSplit.
- BalanceSheet — Map<UserId, Map<UserId, Double>>. balances[A][B] = how much A owes B.
- ExpenseManager — addExpense, getBalances, simplify.

Low-Level Design (LLD)

Strategy pattern for SplitStrategy. The BalanceSheet uses a sparse map of (debtor → creditor → amount). Simplification (optional but a great talking point) is a min-cash-flow problem — a known greedy algorithm runs in $O(N^2 \log N)$.

4. How to Code & Explain (Win the Interviewer)

Open with the invariant: *"Every expense produces splits that sum exactly to the expense amount. I enforce this in the SplitStrategy.validate step so corrupt data can't enter the system."*

Walk an example: *"Alice paid ₹600, split equally among Alice, Bob, Charlie → each owes ₹200. Bob and Charlie now owe Alice ₹200 each. balances[Bob][Alice] += 200. Simplification later collapses pairs that owe each other."*

Mention simplification: *"If A→B = 100, B→C = 100, the net is A→C = 100. The simplifier reduces the graph; each user has either a single creditor or a single debtor."*

> ■ ***Tip:*** *Validate that splits sum to the expense amount (within a 1-paisa tolerance for percentage rounding). Bad splits are the #1 production bug.*

5. Actual Code (Reference Snippets)

Splitwise core — pluggable SplitStrategy, BalanceSheet that nets immediately

Kotlin

```kotlin
enum class SplitType { EQUAL, EXACT, PERCENT, SHARE }

data class Split(val userId: String, val amount: Double)

interface SplitStrategy {
    fun compute(total: Double, participants: List<String>, params: List<Double> = emptyList()): List<Split>
}
object EqualSplit : SplitStrategy {
    override fun compute(total: Double, participants: List<String>, params: List<Double>): List<Split> {
        val share = total / participants.size
        return participants.map { Split(it, share) }
    }
}
object PercentSplit : SplitStrategy {
    override fun compute(total: Double, participants: List<String>, params: List<Double>): List<Split> {
        require(params.size == participants.size && Math.abs(params.sum() - 100.0) < 0.01)
        return participants.zip(params) { u, p -> Split(u, total * p / 100.0) }
    }
}

data class Expense(val id: String, val amount: Double, val paidBy: String,
                   val splits: List<Split>, val currency: String = "INR")

class BalanceSheet {
    // balances[debtor][creditor] = amount debtor owes creditor
    private val balances = mutableMapOf<String, MutableMap<String, Double>>()
    fun apply(e: Expense) {
        for (s in e.splits) {
            if (s.userId == e.paidBy) continue
            balances.getOrPut(s.userId) { mutableMapOf() }.merge(e.paidBy, s.amount) { a, b -> a + b }
            // also reduce reverse if any
            balances[e.paidBy]?.let { it[s.userId]?.also { rev ->
                val net = rev - s.amount
                if (net <= 0.001) it.remove(s.userId)
                else it[s.userId] = net
            } }
        }
    }
    fun owes(debtor: String, creditor: String) = balances[debtor]?.get(creditor) ?: 0.0
}

class ExpenseManager {
    private val sheet = BalanceSheet()
    fun addExpense(amount: Double, paidBy: String, participants: List<String>,
                   strategy: SplitStrategy, params: List<Double> = emptyList()): Expense {
        val splits = strategy.compute(amount, participants, params)
        require(Math.abs(splits.sumOf { it.amount } - amount) < 0.01) { "Splits don't sum" }
```

```
        val e = Expense(java.util.UUID.randomUUID().toString(), amount, paidBy, splits)
        sheet.apply(e); return e
    }
    fun owed(debtor: String, creditor: String) = sheet.owes(debtor, creditor)
}
```

6. Cross-Questions & How to Answer Them

Q: How does multi-currency work?

Each Expense carries its currency. BalanceSheet either: (a) stores per-currency balances, or (b) converts to a base currency at expense time. (a) is more correct (FX rates change), (b) is simpler. State the trade-off.

Q: What's the simplify-debt algorithm?

Min-cash-flow: compute each user's net balance (creditors positive, debtors negative). Greedy match the largest creditor with the largest debtor, settle the smaller of the two, repeat. $O(N^2 \log N)$. Reduces $\leq N-1$ transactions.

Q: Two users edit the same expense at once — who wins?

Optimistic locking on expense version. If both write, second commit fails and the UI surfaces a merge prompt. Or use CRDTs for true concurrent edits — overkill for this.

Q: How do you handle rounding?

Percentage splits accumulate ■0.01 rounding errors. Either store amounts in paise (Long) to avoid floats entirely, or have one designated user 'absorb' the leftover paisa.

7. What Better? Alternative Approaches

Event sourcing: store an immutable log of operations (addExpense, editExpense, deleteExpense, settleUp). BalanceSheet becomes a projection — auditable and recomputable.

Group analytics: per-month spending by category for each user.

OCR receipts: snap a photo, parse line items, suggest splits.

Recurring expenses: Netflix subscription auto-adds an expense every month — a scheduled job.

Key takeaways

- SplitStrategy is the canonical Strategy use-case.
- Validate splits sum to the total at the boundary.
- Long-paise > Double-rupee for money.
- Simplify debts is min-cash-flow.

Chapter 10: Design an LRU Cache

Difficulty: Easy | **Tags: LLD Data Structure Concurrency**

The most asked data-structure design question — O(1) get and put, with optional thread-safety and TTL.

1. What's the Problem?

Implement a Least-Recently-Used cache: a fixed-capacity key-value store where get and put are both O(1). On capacity overflow, evict the least recently accessed entry. Real-world extensions: TTL expiry, write-back, thread-safety, size-based eviction (not just count).

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Capacity by count or by bytes?
- Expiry TTL per entry?
- Eviction policy — LRU only or pluggable (LFU, FIFO)?
- Synchronous or asynchronous loading on miss?
- Single-thread or thread-safe?

Non-functional clarifications

- Both get and put are O(1).
- Lock contention should be low — read-write lock or striped locking for high throughput.
- Memory bound — must not grow beyond capacity even briefly.

Why ask these? (the signal)

Asking 'by count or bytes' separates juniors from seniors — bytes is what real caches like Glide's BitmapPool do.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Two data structures cooperate: a **HashMap** (key → node) for O(1) lookup, and a **doubly linked list** (most recent at head, least recent at tail) for O(1) reordering and eviction. Every get moves the node to the head; every put adds at the head and evicts the tail if over capacity.

Major components

- Node — key, value, prev, next.
- LinkedHashMap-style doubly linked list with head & tail sentinels.
- Map<K, Node> for O(1) lookup.
- Optional: EvictionPolicy (Strategy), TTL, size accounting.

Low-Level Design (LLD)

The two trickiest things in code are (a) using sentinel head & tail nodes to avoid null checks, and (b) keeping the map and list in lock-step. Always update both. For thread-safety, ReentrantReadWriteLock is the simplest; ConcurrentHashMap + per-node lock is more performant but harder to get right (see Caffeine library).

4. How to Code & Explain (Win the Interviewer)

Open: *"O(1) get + put requires a HashMap and a doubly linked list working together. The map gives lookup; the list gives ordering."* Draw the picture immediately — head <-> A <-> B <-> C <-> tail.

Walk a sequence: *"Capacity 3. put(1,'a'), put(2,'b'), put(3,'c'). get(1) moves 1 to head. put(4,'d') evicts 2 — the least recent."*

Bring up Java's `LinkedHashMap`: *"In production code, I'd extend LinkedHashMap and override removeEldestEntry — that's how Android's LruCache works under the hood."*

> ■ ***Tip:*** *Always use sentinel head & tail nodes. Removes 90% of the null-check bugs juniors write.*

5. Actual Code (Reference Snippets)

Standard hashmap + DLL implementation in Kotlin, synchronized

Kotlin

```
class LruCache<K, V>(private val capacity: Int) {

    private data class Node<K, V>(val key: K, var value: V) {
        var prev: Node<K, V>? = null
        var next: Node<K, V>? = null
    }

    private val map = HashMap<K, Node<K, V>>()
    private val head = Node<K, V>(null as K, null as V).also { /* sentinel */ }
    private val tail = Node<K, V>(null as K, null as V).also { /* sentinel */ }
    init { head.next = tail; tail.prev = head }

    @Synchronized
    fun get(key: K): V? {
        val node = map[key] ?: return null
        moveToHead(node); return node.value
    }

    @Synchronized
    fun put(key: K, value: V) {
        val existing = map[key]
        if (existing != null) {
            existing.value = value; moveToHead(existing); return
        }
        val node = Node(key, value)
        map[key] = node; addAfterHead(node)
        if (map.size > capacity) {
            val lru = tail.prev!!
            removeNode(lru); map.remove(lru.key)
        }
    }

    private fun addAfterHead(n: Node<K, V>) {
        n.prev = head; n.next = head.next
        head.next!!.prev = n; head.next = n
    }
    private fun removeNode(n: Node<K, V>) {
        n.prev!!.next = n.next; n.next!!.prev = n.prev
    }
    private fun moveToHead(n: Node<K, V>) { removeNode(n); addAfterHead(n) }
}
```

6. Cross-Questions & How to Answer Them

Q: How would you make it byte-bounded instead of count-bounded?

Track a total-bytes counter. On put, ask the value for its size (V can be a Sized interface with sizeBytes()). Evict from tail until totalBytes ≤ capacity. This is exactly how Android's LruCache and Glide's BitmapPool work.

Q: Add TTL — entries expire after T seconds.

Two approaches: (a) lazy — on get, check entry's timestamp and treat as miss if expired. Simple, no extra structure. (b) active — a separate thread sweeps periodically. Use lazy by default; active only if you need fast memory release.

Q: Make it thread-safe with high throughput.

Synchronize the whole map is simple but a bottleneck. Better: striped locking (N independent shards keyed by hash) or use Caffeine which uses lock-free read paths + a write-buffer. For an Android in-process cache, ReentrantReadWriteLock is usually fine.

Q: LRU vs LFU vs FIFO — when do you pick which?

LRU is recency-biased — perfect for image caches where recently-viewed images are likely to be viewed again. LFU is frequency-biased — better when there's a long-tail of constantly-hot keys (e.g. user-profile cache). FIFO is dumb but predictable.

7. What Better? Alternative Approaches

Caffeine: for the JVM, just use it. Window-TinyLFU eviction outperforms LRU. Async loader, expiry, size, statistics all built-in.

Two-tier cache: in-memory LRU + disk LRU (Glide pattern). Misses on memory hit disk; misses on disk hit network.

Distributed cache (Redis): when one app instance isn't enough, push the cache to a shared service. Same API, very different operational concerns.

Soft references: Android's BitmapPool allows the GC to reclaim under memory pressure. Mention soft references vs strong references trade-offs.

Key takeaways

- HashMap + DLL is the textbook answer; know it cold.
- Sentinel nodes eliminate edge cases.
- For real Android use, prefer Android's built-in LruCache or Caffeine.
- Byte-bounded > count-bounded for media caches.

Chapter 11: Design an Image Loading Library (like Glide / Coil)

Difficulty: Medium | **Tags: Library Design Caching Bitmaps Concurrency**

Build a Glide-clone — fetch, decode, cache, recycle bitmaps, and stay smooth in a scrolling RecyclerView.

1. What's the Problem?

Design a library that an Android app can call as `ImageLoader.load(url).into(imageView)`. The library fetches the image from network or disk, decodes it to a Bitmap, caches it in memory and disk, handles RecyclerView recycling correctly, and avoids OutOfMemory crashes on low-end devices.

Scale to assume

- A typical Instagram-style feed shows 20+ image-bearing items per screen, each potentially with multiple images.
- Heap budget on a low-end device may be 64–128 MB; a single 4K bitmap is 33 MB. Memory pressure is real.
- Scrolling is the killer test — every frame at 60 fps = 16 ms budget. Decoding on the main thread kills scroll.

2. How Should You Think? (Clarifying Questions)

This is the classic 'design a library' question. The interviewer is checking three things: do you understand the Android Bitmap memory model, can you design a clean public API, and do you know caching is multi-tier?

Functional clarifications to ask

- Sources — URL, file path, content URI, drawable resource — all supported?
- Transformations — resize, crop, blur, rounded corners — pluggable?
- Placeholder while loading, error image on failure?
- GIF, WebP, video thumbnails — in scope?
- Cancel a request when the target view is recycled?
- Prefetching for off-screen items?

Non-functional clarifications

- No OutOfMemoryError on low-RAM devices.
- Decoding always off main thread.
- Smooth 60 fps scroll even with images loading.
- Disk cache survives app restart; memory cache survives screen rotation.

Why ask these? (the signal)

Asking about cancellation reveals you understand RecyclerView's view recycling — it's the most common reason for the wrong image showing up. Asking about bitmap pooling is the senior signal.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Five layers in series: **Request Manager** (owns the lifecycle, attached to Activity/Fragment) → **Engine** (deduplicates in-flight requests, orchestrates) → **Memory Cache** (LRU of bitmaps) → **Disk Cache** (LRU of files) → **Fetcher** (HTTP / file / content URI). A response flows back up, with the decoded bitmap being placed into both caches. A **Bitmap Pool** lives next to the memory cache and recycles bitmap allocations to avoid GC pressure.

Major components

- ImageLoader (entry-point fluent API): load(url).resize().placeholder().into(view).
- Request — url, target (WeakReference<ImageView>), transformations, placeholder.
- RequestManager — bound to lifecycle; cancels requests on onDestroy().
- Engine — dedup by cache-key; serialises work to a thread pool.
- MemoryCache — LruCache<Key, Bitmap>, size-byte bounded.
- DiskCache — LRU on disk (DiskLruCache by Jake Wharton).
- BitmapPool — reusable inBitmap targets for BitmapFactory.
- Fetcher — HttpFetcher, FileFetcher, AssetFetcher, ContentResolverFetcher.
- Decoder — Strategy: ImageDecoder, BitmapFactory, GifDecoder, SvgDecoder.
- Transformation pipeline — list of Bitmap → Bitmap functions.

Low-Level Design (LLD)

The cache key is the most important detail. Naive: just the URL. Senior: URL + target size + transformations. Two ImageViews of different sizes loading the same URL should produce two different cached bitmaps so you don't waste memory scaling up a 100×100 placeholder to fill 1000×1000.

Cancellation via target tag. Set the request as a tag on the ImageView. When the view is recycled, the new request reads the old tag and cancels. This is the standard trick to avoid 'wrong image flashed for one frame'.

WeakReference to the ImageView. Holding a strong reference leaks the activity on rotation. Always weak.

BitmapPool works by passing `BitmapFactory.Options.inBitmap` with a recyclable Bitmap of compatible size. The decoder reuses the allocation. Cuts garbage collection by 70–90% in image-heavy scrolling.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is multi-tier caching with a clean fluent API on top, plus the Android-specific work to play nicely with RecyclerView and Bitmap memory."* Draw the 5-layer pipeline immediately.

Justify each layer: *"Memory cache is fastest but volatile and small. Disk cache survives restart and is much larger but slower. Network is last and slowest. We always try them in order."*

Senior signal: *"The cache key must include the requested size and transformations — otherwise we either return wrong data or waste memory storing the same bitmap at different scales. This bug is in many home-grown loaders."*

Demo cancellation: *"Each ImageView holds a tag pointing to its current Request. When RecyclerView rebinds the view, we first read the tag and call request.cancel() on it. The previous decode never reaches setImageBitmap."*

> ■ ***Tip:*** *Mention **inBitmap** and **BitmapPool** by name — this single phrase tells the interviewer you've actually built or debugged a Glide-like library.*

5. Actual Code (Reference Snippets)

Fluent API + 4-tier load pipeline + BitmapPool reuse. Cancellation via view tag holds the Job.

Kotlin

```kotlin
// Public fluent API
class ImageRequest internal constructor(val url: String) {
    var targetWidth: Int = 0; var targetHeight: Int = 0
    var transformations: List<Transformation> = emptyList()
    var placeholder: Int? = null; var error: Int? = null

    fun resize(w: Int, h: Int) = apply { targetWidth = w; targetHeight = h }
    fun transform(t: Transformation) = apply { transformations = transformations + t }
    fun placeholder(id: Int) = apply { placeholder = id }
    fun into(view: android.widget.ImageView) = ImageLoader.enqueue(this, view)
}
```

```
object ImageLoader {
    private val memCache = MemoryCache(maxBytes = (Runtime.getRuntime().maxMemory() / 8).toInt())
    private val diskCache = DiskCache(dir = /* app cache dir */ java.io.File("/cache"), 50L * 1024 * 1024)
    private val bitmapPool = BitmapPool()
    private val engine = Engine(memCache, diskCache, bitmapPool)

    fun load(url: String) = ImageRequest(url)

    internal fun enqueue(req: ImageRequest, view: android.widget.ImageView) {
        // Cancel any previous request bound to this view
        (view.getTag(R.id.image_request_tag) as? Job)?.cancel()
        val viewRef = java.lang.ref.WeakReference(view)

        val job = engine.scope.launch {
            req.placeholder?.let { viewRef.get()?.setImageResource(it) }
            val key = CacheKey(req.url, req.targetWidth, req.targetHeight, req.transformations)
            try {
                val bmp = engine.load(req, key)
                // Verify the view still wants this request
                val v = viewRef.get() ?: return@launch
                if (v.getTag(R.id.image_request_tag) === coroutineContext[Job]) {
                    v.setImageBitmap(bmp)
                }
            } catch (t: Throwable) {
                req.error?.let { viewRef.get()?.setImageResource(it) }
            }
        }
        view.setTag(R.id.image_request_tag, job)
    }
}

class Engine(private val mem: MemoryCache, private val disk: DiskCache,
             private val pool: BitmapPool) {
    val scope = kotlinx.coroutines.CoroutineScope(
        kotlinx.coroutines.Dispatchers.IO +
        kotlinx.coroutines.SupervisorJob()
    )
    private val inFlight = mutableMapOf<CacheKey, kotlinx.coroutines.Deferred<android.graphics.Bitmap>>()
    private val lock = Any()

    suspend fun load(req: ImageRequest, key: CacheKey): android.graphics.Bitmap {
        mem.get(key)?.let { return it }                     // 1) Memory
        val existing = synchronized(lock) { inFlight[key] }  // 2) Dedup
        if (existing != null) return existing.await()

        val task = scope.async {
            disk.get(key)                                   // 3) Disk
                ?.let { decode(it, req, pool) }
                ?: run {
                    val bytes = HttpFetcher.fetch(req.url)  // 4) Network
                    disk.put(key, bytes)
                    decode(bytes, req, pool)
                }
                .also { mem.put(key, it) }
        }
        synchronized(lock) { inFlight[key] = task }
        try { return task.await() }
        finally { synchronized(lock) { inFlight.remove(key) } }
    }
}

fun decode(bytes: ByteArray, req: ImageRequest, pool: BitmapPool): android.graphics.Bitmap {
    val opts = android.graphics.BitmapFactory.Options().apply {
        inJustDecodeBounds = true
    }
    android.graphics.BitmapFactory.decodeByteArray(bytes, 0, bytes.size, opts)
```

```
    opts.inSampleSize = calcSampleSize(opts, req.targetWidth, req.targetHeight)
    opts.inJustDecodeBounds = false
    opts.inMutable = true
    opts.inBitmap = pool.getReusable(opts)                  // recycle alloc
    return android.graphics.BitmapFactory.decodeByteArray(bytes, 0, bytes.size, opts)
        .let { req.transformations.fold(it) { b, t -> t.apply(b) } }
}
```

6. Cross-Questions & How to Answer Them

Q: Why do you cache by URL + size + transformations instead of just URL?

Two ImageViews of different sizes need different decoded bitmaps. Caching only by URL means either (a) returning a too-small bitmap that looks blurry when scaled up, or (b) wasting heap by keeping a full-resolution bitmap when a thumbnail would do. Glide does it this way for the same reason.

Q: How do you avoid OutOfMemory on a heavy feed?

Three lines of defense. (1) inSampleSize during decode so we never decode bigger than the target view. (2) Byte-bounded LruCache sized as 1/8 of the heap. (3) BitmapPool with inBitmap so even the brief allocation peaks reuse old buffers. On top of that, request cancellation when views recycle prevents bitmaps for off-screen items.

Q: RecyclerView shows the wrong image for one frame — what's the cause and fix?

The view was recycled and asked to load a new URL, but the previous request's onResponse is still pending. When it returns, it sets the old image on the new view. Fix: tag the view with the current request token; before setImageBitmap, verify the view's tag still matches the token. If not, drop the result.

Q: Why not just use OkHttp's disk cache?

OkHttp caches HTTP responses, not decoded Bitmaps. After decoding, you'd re-decode on every disk hit, which is expensive. Glide's disk cache stores decoded + transformed bitmaps so the second view of the same image at the same size skips decode entirely.

Q: What about animated GIFs or WebP?

Decoder becomes a Strategy. The Engine sniffs the bytes header (GIF89a, RIFF…WEBP) and picks GifDecoder or WebpDecoder. Animated formats produce a Drawable instead of a Bitmap; the View accepts both via setImageDrawable.

7. What Better? Alternative Approaches

Use ImageDecoder (API 28+) instead of BitmapFactory — it's faster, supports more formats, and natively supports animated WebP.

Hardware bitmaps via `Config.HARDWARE` — pixel data lives on the GPU, slashing main-memory usage. Trade-off: cannot be transformed in software.

Preloading — the RecyclerView prefetcher (or a custom ScrollListener) can warm the cache for the next 5 items before they bind.

Trim memory — listen to ComponentCallbacks2.onTrimMemory and shrink the bitmap pool / memory cache under TRIM_MEMORY_RUNNING_CRITICAL.

Skip the library: for a single static image, plain BitmapFactory.decodeResource is fine. Don't pull in Glide for one image.

Key takeaways

- Cache key = URL + size + transformations. Not just URL.
- BitmapPool + inBitmap = 70–90% less GC pressure.
- Cancel via view-tag when the view recycles.
- 4-tier load: Memory → InFlight dedup → Disk → Network.

Chapter 12: Design Infinite-Scroll Pagination (mobile feed)

Difficulty: Medium | **Tags: Pagination RecyclerView Networking UX**

How a feed loads forever without lagging — cursor pagination, prefetch, deduplication, scroll restoration.

1. What's the Problem?

A user opens a feed (Twitter, Instagram, LinkedIn). It shows 20 items. As they scroll, more load automatically. Items are displayed buttery-smooth, no duplicates, no jumps when new content arrives, and a back-press from the detail screen returns them to exactly the same scroll position. Design the client side end-to-end.

Scale to assume

- A power user scrolls 500–1000 items per session. The list cannot hold all of them in memory.
- Each item is a complex card with images and text. Initial paint must be under 200 ms.
- Network can be flaky; the feed must degrade gracefully.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Sort order — chronological, ranked, mixed?
- Pull-to-refresh? New-content banner when fresh items arrive at top?
- Pagination model — offset/limit, cursor/since-id, page-token?
- Offline reads — show cached feed when network is down?
- Scroll restoration after navigating into and back out of an item?
- Optimistic UI for like / comment without waiting for the server?

Non-functional clarifications

- 60 fps scroll on mid-range devices.
- Tens of thousands of items eventually scrolled, but bounded memory usage.
- Resilient to slow/lossy network — show loaders, retry, surface errors.
- No duplicate items even when a refresh races with a load-more.

Why ask these? (the signal)

Asking 'offset or cursor' is the single best question. Offset is wrong for a constantly-changing feed and produces duplicates. Cursor is the right answer.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

On the client: **UI (RecyclerView/LazyColumn)** → **ViewModel (StateFlow<FeedState>)** → **PagingSource / Repository** → **Local DB (Room)** ← → **Remote API (cursor paginated)**. The Repository writes new pages into Room; the UI observes Room via Flow so it's always in sync. Single source of truth: Room. The screen never sees a network response directly.

Major components

- FeedRepository — loadInitial / loadAfter(cursor); writes to Room.
- RemoteMediator (Paging 3) — bridges remote pages into local DB.
- Room PagingSource — UI reads from DB; reactive on inserts.

- ViewModel — exposes PagingData<FeedItem> as Flow.
- Compose LazyColumn or RecyclerView.Adapter with PagingDataAdapter.
- Prefetch policy — load page N+1 when scroll reaches 80% of page N.

Low-Level Design (LLD)

Cursor pagination, not offset. The server returns `{ items: [...], nextCursor: 'abc123' }`. The client passes nextCursor back. Offset pagination shows duplicates when new items arrive — never use it for live feeds.

Single source of truth: Room. The UI binds to a query on Room. The Repository's job is to keep Room fresh. This decouples scrolling (always cheap, local) from networking (sometimes slow).

RemoteMediator — Jetpack Paging 3's integration point. Implements load() with LoadType.REFRESH / APPEND / PREPEND. Persists the cursor between sessions so resuming the app restores position.

Deduplication — the FeedItem id is the primary key in Room. Inserts with conflict strategy REPLACE prevent duplicates even if a refresh re-fetches an item from a different page.

Scroll restoration — Paging 3 + RecyclerView handle this if you preserve the LayoutManager state via Bundle in onSaveInstanceState.

4. How to Code & Explain (Win the Interviewer)

Open: *"I'll use Jetpack Paging 3 with Room as the source of truth and a cursor-based API. This pattern gives infinite scroll, offline support, retry, scroll restoration, and deduplication essentially for free."*

Justify cursor: *"Offset pagination breaks on a live feed — if 3 new items arrive between page 1 and page 2, items 18–20 of page 1 reappear as 21–23 of page 2. Cursors anchor to specific items, immune to insertion at the top."*

Walk one scroll: *"User scrolls. PagingDataAdapter peeks at upcoming items. When position reaches 80%, it triggers RemoteMediator.load(APPEND). RemoteMediator hits API, gets next page + cursor, inserts into Room. Room's PagingSource emits, UI binds new items. The user sees seamless scroll."*

Sneak in performance: *"Each row's onBindViewHolder must be cheap — no DB calls, no heavy parsing. Pre-process when writing to Room, not when reading on the UI thread."*

> ■ ***Tip:*** *Bring up the* ***'new posts above' problem*** *— when new items arrive while the user is scrolled mid-feed, never insert above their visible window silently. Show a 'New posts' banner; let them tap it.*

5. Actual Code (Reference Snippets)

Paging 3 + Room + cursor pagination — the canonical Android way

Kotlin

```kotlin
@Entity(tableName = "feed_item")
data class FeedItem(
    @PrimaryKey val id: String,
    val authorId: String, val text: String, val imageUrl: String?,
    val createdAt: Long, val pageKey: Long
)
@Entity(tableName = "remote_keys")
data class RemoteKey(@PrimaryKey val feedKey: String, val nextCursor: String?)

@Dao interface FeedDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(items: List<FeedItem>)
    @Query("SELECT * FROM feed_item ORDER BY createdAt DESC")
    fun pagingSource(): PagingSource<Int, FeedItem>
    @Query("DELETE FROM feed_item") suspend fun clear()
}

@OptIn(ExperimentalPagingApi::class)
```

```
class FeedRemoteMediator(
    private val api: FeedApi, private val db: AppDatabase
) : RemoteMediator<Int, FeedItem>() {

    override suspend fun load(loadType: LoadType, state: PagingState<Int, FeedItem>):
            MediatorResult = try {
        val cursor: String? = when (loadType) {
            LoadType.REFRESH -> null
            LoadType.PREPEND -> return MediatorResult.Success(endOfPaginationReached = true)
            LoadType.APPEND  -> db.remoteKeysDao().get("feed")?.nextCursor
                ?: return MediatorResult.Success(endOfPaginationReached = true)
        }
        val response = api.feed(cursor = cursor, limit = state.config.pageSize)
        db.withTransaction {
            if (loadType == LoadType.REFRESH) db.feedDao().clear()
            db.remoteKeysDao().insert(RemoteKey("feed", response.nextCursor))
            db.feedDao().insertAll(response.items)
        }
        MediatorResult.Success(endOfPaginationReached = response.nextCursor == null)
    } catch (e: Exception) { MediatorResult.Error(e) }
}

class FeedRepository(private val api: FeedApi, private val db: AppDatabase) {
    @OptIn(ExperimentalPagingApi::class)
    fun feedPager(): Flow<PagingData<FeedItem>> = Pager(
        config = PagingConfig(pageSize = 20, prefetchDistance = 5,
                              enablePlaceholders = false, initialLoadSize = 40),
        remoteMediator = FeedRemoteMediator(api, db),
        pagingSourceFactory = { db.feedDao().pagingSource() }
    ).flow
}

// ViewModel
class FeedViewModel(repo: FeedRepository) : ViewModel() {
    val items: Flow<PagingData<FeedItem>> =
        repo.feedPager().cachedIn(viewModelScope)
}

// UI (Compose)
@Composable
fun FeedScreen(vm: FeedViewModel) {
    val items = vm.items.collectAsLazyPagingItems()
    LazyColumn {
        items(items.itemCount, key = { items[it]?.id ?: it }) { i ->
            items[i]?.let { FeedItemCard(it) }
        }
        item { if (items.loadState.append is LoadState.Loading) CircularProgressIndicator() }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why is offset pagination wrong for a live feed?

Imagine the feed is ordered DESC by createdAt. You load page 1 (LIMIT 20 OFFSET 0). While reading it, three new posts arrive. When you request page 2 (LIMIT 20 OFFSET 20), positions shift: items that were 18–20 are now 21–23. You see them twice. Cursor (WHERE createdAt < lastSeenCreatedAt) anchors to a specific point and is immune to inserts above.

Q: How do you handle pull-to-refresh without losing the user's scroll position?

Refresh hits the top of the feed and gets the latest cursor; new items are inserted to Room above. Don't auto-scroll the list. Show a small 'N new posts' banner. When the user taps it, smoothScrollToPosition(0). This is what Twitter does.

Q: How do you survive a process death mid-scroll?

Paging 3 + Room give this for free. The DB persists; the cursor is stored in RemoteKey. On rehydrate, the same PagingSource reopens at the same items. RecyclerView state via savedInstanceState preserves scroll offset.

Q: What if a user dwells on item 5000?

PagingData keeps recently-accessed pages in memory and evicts old ones (maxSize in PagingConfig). Room still has all items if you want to scroll back, but RAM is bounded. Critical for low-end devices.

Q: Items show ranked, not chronological — does cursor pagination still work?

Yes, but the cursor needs to encode the ranking signal. Common pattern: cursor = base64({rankScore, id}). The server's WHERE clause becomes WHERE (rankScore, id) < (lastScore, lastId). Or the server keeps the ranking session-scoped and issues an opaque token.

7. What Better? Alternative Approaches

Skeleton placeholders instead of spinners — perceived performance jumps by ~25%.

Prefetch images of the next 5 items as soon as their rows enter the prefetch window. Combine with Glide.preload().

Reactive merging: if the feed is also receiving real-time pushes (likes, comments updates), let WebSocket events update Room — the UI re-emits automatically. Single source of truth wins.

Decouple from Paging 3: for a custom feed (e.g. ML-personalised), implement a custom PagingSource directly; same pattern, more control.

Key takeaways

- Cursor pagination — never offset.
- Single source of truth = Room. The UI observes the DB, not the network.
- Paging 3 + RemoteMediator gives offline, retry, restoration almost for free.
- Banner-on-new instead of auto-insert preserves user context.

Part 2

Mobile Apps Core

Chapters 13 — 24

Chapter 13: Design a Push Notification System (FCM-based)

Difficulty: Medium | **Tags: Push FCM Notifications Background work**

How a notification lands on the user's phone — from server trigger through FCM to a click that opens the right screen.

1. What's the Problem?

Design the end-to-end notification path for a mobile app. The backend has an event (new message, breaking news, promotion). The user's phone must receive the notification within seconds, show it correctly, and deep-link to the right screen on tap — even if the app was killed.

Scale to assume

- 100M+ users; peak burst of millions of notifications per minute on breaking news.
- Delivery latency target: 95th percentile under 3 seconds.
- Android battery / Doze mode: device may be asleep — only FCM high-priority can wake it.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Categories — transactional (OTP), promotional (offer), social (like, comment), system (update)?
- Per-user preferences — opt-out, quiet hours, channel selection?
- Rich content — images, action buttons, expandable layout?
- Deep links — which screen, with what state?
- In-app banner when app is foregrounded (separate UI)?
- Aggregation — '5 people liked your post' instead of 5 separate pushes?

Non-functional clarifications

- Best-effort delivery (FCM doesn't guarantee); design for it.
- Low battery impact — high-priority only when needed.
- Survives app being killed (FCM does, in-process WebSocket does not).
- Tap analytics: did user tap? When? On which device?

Why ask these? (the signal)

Asking about high-priority vs normal-priority is the senior signal — most candidates don't know Doze mode delays normal-priority for hours.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Server side: Event Producer → Notification Service (templating + user prefs) → Queue (Kafka) → Dispatcher → FCM HTTP v1 API. **Device side:** FCM-managed connection → FirebaseMessagingService → NotificationCompat builder → System UI. On tap: PendingIntent → DeepLink Activity → target screen with state.

Major components

- Notification Service (server) — owns templates + per-user preferences + opt-outs.
- Token Registry — maps userId → list of FCM tokens (one per device).
- Dispatcher — applies prefs, rate limits, batches, sends to FCM.

- FirebaseMessagingService (Android) — onMessageReceived; refresh token on rotation.
- NotificationChannel (Android 8+) — category, importance, sound, vibration.
- DeepLink router — Activity that parses intent extras and navigates.

Low-Level Design (LLD)

Token lifecycle: FCM tokens rotate. Override onNewToken to push fresh tokens to the server. On logout, delete the token from both FCM and the server.

NotificationChannel is mandatory on Android 8+. Create channels per category (transactional vs promotional) so users can mute one without muting all.

Foreground vs background: If app is in foreground, onMessageReceived fires for the 'data' part; the OS doesn't show the notification for you. Build your own NotificationCompat. If app is killed, only 'notification' payloads auto-show.

Deep linking via PendingIntent. Always set FLAG_UPDATE_CURRENT and an action so taps go to the right Activity even if the app is cold-started.

Aggregation via NotificationCompat.InboxStyle when 5+ notifications share a group key.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is a two-sided problem: server fan-out plus device handling. I'll keep them decoupled with a queue, and use FCM as the wake-up channel since nothing else reliably wakes an Android device in Doze."*

Justify high-priority: *"For transactional notifications I'd set priority=high and content_available=true so the device wakes; for promotional, normal-priority so Doze can batch them — saves battery."*

Walk one click: *"Tap → PendingIntent fires → DeepLinkActivity reads the intent's data Uri → uses NavController.navigate. If the app was cold-started, we set the back stack so Up takes the user to Home, not out of the app."*

> ■ ***Tip:*** *Bring up* ***'silent push'*** *(data-only payload). The OS does not show anything; the app runs in the background to sync. This is what mail apps use to refresh inboxes. Senior signal.*

5. Actual Code (Reference Snippets)

FCM service handler with deep-link PendingIntent and per-category channels

Kotlin

```
class MyMessagingService : FirebaseMessagingService() {

    override fun onNewToken(token: String) {
        // FCM rotates tokens; sync to server
        TokenSyncWorker.enqueue(applicationContext, token)
    }

    override fun onMessageReceived(msg: RemoteMessage) {
        // Two paths: notification payload (display) or data payload (silent / custom)
        val data = msg.data
        val type = data["type"] ?: "default"
        when (type) {
            "silent_sync" -> SyncWorker.enqueue(applicationContext)
            else -> showNotification(data)
        }
    }

    private fun showNotification(data: Map<String, String>) {
        val title = data["title"] ?: return
        val body = data["body"] ?: ""
        val deepLink = data["deep_link"] ?: "app://home"
```

```
        val channelId = data["category"] ?: "default"

        val pending = PendingIntent.getActivity(
            this, deepLink.hashCode(),
            Intent(Intent.ACTION_VIEW, Uri.parse(deepLink))
                .setPackage(packageName)
                .addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP),
            PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
        )

        val notif = NotificationCompat.Builder(this, channelId)
            .setSmallIcon(R.drawable.ic_notification)
            .setContentTitle(title)
            .setContentText(body)
            .setStyle(NotificationCompat.BigTextStyle().bigText(body))
            .setContentIntent(pending)
            .setAutoCancel(true)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .build()
        NotificationManagerCompat.from(this).notify(data["id"]?.hashCode() ?: 0, notif)
    }
}

// Application.onCreate: create channels (idempotent)
fun createChannels(ctx: Context) {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return
    val mgr = ctx.getSystemService(NotificationManager::class.java)
    listOf(
        Channel("transactional", "Account & Security", IMPORTANCE_HIGH),
        Channel("social",        "Likes, Comments",    IMPORTANCE_DEFAULT),
        Channel("promotional",   "Offers",             IMPORTANCE_LOW),
    ).forEach { c ->
        mgr.createNotificationChannel(
            NotificationChannel(c.id, c.name, c.imp))
    }
}
data class Channel(val id: String, val name: String, val imp: Int)
```

6. Cross-Questions & How to Answer Them

Q: Why FCM and not your own WebSocket?

A WebSocket dies the moment the app is killed (which Android does aggressively under memory pressure or in Doze). FCM has a single OS-level connection that survives, and is the only way to wake the device. WebSocket is fine for in-app real time while the app is foregrounded — many apps use both.

Q: How do you ensure the same user with 3 devices doesn't miss notifications?

Register every device's FCM token under the userId. The server fans out to all tokens. Track 'last seen' per device; deactivate tokens that haven't pinged in 30+ days to avoid wasted sends.

Q: How would you de-duplicate when multiple servers race to send the same notification?

Generate a notification UUID at the producer; the dispatcher uses it as an idempotency key in a short-TTL Redis SETNX. Second sender sees the key exists and skips.

Q: How do you avoid notification fatigue?

Server-side rate limiter per user (max N social pushes/hour). Aggregation: '5 new likes' instead of 5 pushes — use the same group_key and InboxStyle. Respect user-configured quiet hours.

Q: FCM sometimes silently drops messages. What do you do?

It's best-effort. For critical messages (OTP), have a fallback: SMS via Twilio after N seconds of no ack. For chat, design the in-app sync to fetch missed messages when the app opens — never rely solely on push to deliver state.

7. What Better? Alternative Approaches

Server-side: SNS, OneSignal, Airship — managed providers do the rate-limiting, retry, and analytics for you.

Rich notifications with images (BigPictureStyle), inline reply, custom layouts (RemoteViews).

Notification trampolines deprecation: Android 12+ blocks Service/Broadcast intents from notifications. Always use Activity intents.

Predictive batching: ML model decides 'is now a good time to notify this user'. Most opens cluster in commute hours.

Push tokens leak risk: never log them; rotate on logout; treat as PII.

Key takeaways

- FCM = only reliable wake-up on Android.
- Channels (8.0+) and high-priority (battery) are non-negotiable.
- Always use PendingIntent.FLAG_IMMUTABLE on API 31+.
- Push delivers best-effort — your app must reconcile state on open.

Chapter 14: Design a Mobile Analytics SDK (like Firebase / Amplitude)

Difficulty: Medium | **Tags: SDK Eventing Batching Background work**

Track events, batch them, persist offline, upload reliably — without slowing the host app or draining battery.

1. What's the Problem?

Design an SDK that an app integrates with one line: `Analytics.track('button_click', mapOf('screen' to 'home'))`. Events must reach the server reliably, even after app kills, low connectivity, or device reboots. The SDK must not block the main thread, must minimise battery and data, and must respect user privacy / GDPR.

Scale to assume

- A typical session generates 50–200 events. Power users hit 1000+.
- Networks vary: WiFi is fine, cellular costs the user, edge cases like trains have intermittent connectivity.
- SDK adds <100 ms to app cold start and <1 MB to APK.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Auto-tracked events (screen views, taps, crashes) vs custom events?
- User properties vs event properties?
- Identify / login / logout — does data attach to a user or a device?
- Real-time vs batched delivery? Real-time mode for debug builds?
- Sampling — track 1% of users for some events?
- Privacy: opt-out, data deletion, GDPR right-to-be-forgotten?

Non-functional clarifications

- Track() returns in < 1 ms — no I/O on caller's thread.
- No event loss across app kills / device reboots.
- Battery- and data-conscious: batch, prefer WiFi, exponential backoff.
- Crash-safe: SDK crashes must never bring down the host app.

Why ask these? (the signal)

Asking about offline queue + retry shows you understand the SDK is fundamentally a reliable-pipeline problem, not just an API wrapper.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Three layers. **Public API** (track, identify, setUserProperty) → **Event Queue** in memory + persistent (Room/SQLite) → **Uploader** (WorkManager background job that batches and POSTs). Each layer is independent and testable. The public API hands off via a single-threaded executor to ensure FIFO ordering without blocking the caller.

Major components

- AnalyticsClient — public API singleton, fluent + Kotlin-friendly.
- EventEnricher — adds common props (session id, app version, OS, device) automatically.
- EventQueue — in-mem buffer + Room-backed durable queue.

- Batcher — groups N events or 30s, whichever first, into a batch.
- Uploader (WorkManager) — POST /events; exponential backoff; constraints (network connected).
- ConfigService — remote config: enable/disable events, sample rates.

Low-Level Design (LLD)

Single-thread executor for all SDK work. track() does `executor.submit { … }` and returns instantly. Inside, we enrich the event, append to in-mem ring buffer, periodically flush to Room.

Event schema: id (UUID), name, props, timestamp, sessionId, userId. Schema-stable across versions; new fields are nullable.

Upload strategy: WorkManager OneTimeWorkRequest with NetworkType.CONNECTED constraint. On 5xx, retry with exponential backoff (1s, 2s, 4s…). On 4xx, drop the batch and log — the data is unparseable, retrying won't help.

Crash safety: every public method body wraps in try/catch; uncaught throwables are swallowed and only logged in debug builds. Production apps must never crash because of analytics.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is fundamentally a durable queue inside a mobile app. The API is the easy part — getting events delivered reliably across kills, restarts, and bad networks is the hard part."*

Show the pipeline: *"track() never blocks; we hand off to an executor, enrich, persist to Room. WorkManager periodically wakes, reads a batch, posts, and on success deletes the rows. On failure, it backs off and retries."*

Sneak in privacy: *"Every event passes through a sampling + opt-out gate before reaching the queue. Users can call Analytics.optOut() and we stop persisting new events and discard the queue."*

> ■ ***Tip:*** *Mention* ***WorkManager + ExpeditedWorkRequest*** *for low-latency uploads. On Android 12+, expedited work bypasses Doze. Real Amplitude / Firebase SDKs use exactly this.*

5. Actual Code (Reference Snippets)

Single-threaded executor + Room durable queue + WorkManager uploader

Kotlin

```kotlin
object Analytics {
    private val executor = java.util.concurrent.Executors.newSingleThreadExecutor()
    private val context: Context get() = AnalyticsModule.appContext
    private val db: EventDao get() = AnalyticsModule.db.eventDao()
    @Volatile private var userId: String? = null
    @Volatile private var optedOut = false

    fun track(event: String, props: Map<String, Any?> = emptyMap()) {
        if (optedOut) return
        executor.submit {
            try {
                val e = Event(
                    id = java.util.UUID.randomUUID().toString(),
                    name = event,
                    propsJson = JsonConverter.to(props + commonProps()),
                    userId = userId,
                    timestamp = System.currentTimeMillis()
                )
                db.insert(e)
                maybeScheduleUpload()
            } catch (t: Throwable) {
                Log.w("Analytics", "track failed", t)   // never crash host
            }
        }
    }
```

```
    fun identify(id: String) { executor.submit { userId = id } }
    fun optOut() { optedOut = true; executor.submit { db.deleteAll() } }

    private fun maybeScheduleUpload() {
        val req = OneTimeWorkRequestBuilder<UploadWorker>()
            .setConstraints(Constraints.Builder()
                .setRequiredNetworkType(NetworkType.CONNECTED).build())
            .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 10, TimeUnit.SECONDS)
            .build()
        WorkManager.getInstance(context)
            .enqueueUniqueWork("analytics_upload",
                ExistingWorkPolicy.KEEP, req)
    }

    private fun commonProps() = mapOf(
        "app_version" to BuildConfig.VERSION_NAME,
        "os_version"  to Build.VERSION.RELEASE,
        "device"      to Build.MODEL,
        "session_id"  to SessionTracker.current
    )
}

@Entity(tableName = "events")
data class Event(@PrimaryKey val id: String, val name: String,
                 val propsJson: String, val userId: String?,
                 val timestamp: Long)

class UploadWorker(c: Context, p: WorkerParameters) : CoroutineWorker(c, p) {
    override suspend fun doWork(): Result {
        val dao = AnalyticsModule.db.eventDao()
        while (true) {
            val batch = dao.takeBatch(limit = 100)
            if (batch.isEmpty()) return Result.success()
            val resp = try { AnalyticsApi.upload(batch) }
                       catch (e: IOException) { return Result.retry() }
            when {
                resp.code() in 200..299 -> dao.delete(batch)
                resp.code() in 400..499 -> { dao.delete(batch); /* drop */ }
                else                    -> return Result.retry()
            }
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: How do you guarantee event ordering?

Single-threaded executor for writes ensures FIFO into the queue. The uploader sends batches in id-order. The server, if it cares about order, sorts by client timestamp on ingest. Note client clocks can skew — record server-receive time too.

Q: What if the app is killed mid-track?

If we already wrote to Room, the event survives. If track() returned before the executor ran, the event is lost. For critical events (purchase complete) we expose a synchronous variant that flushes to Room before returning.

Q: How do you handle device clock skew (user changes time)?

Record both client and server times. For events where ordering matters, server time wins. For lifecycle events ('user opened app at 9am'), client time matters; we just accept the noise.

Q: How is sampling implemented?

Hash userId into [0, 100); compare against sample rate from remote config. Deterministic, so the same user is consistently in or out of a cohort. Per-event-name sample rates allow tracking 100% of purchases but 1% of scroll events.

Q: How would you support real-time mode for debug builds?

Skip Room and skip the WorkManager — POST each event immediately on the executor thread. Wrap in BuildConfig.DEBUG so prod doesn't pay the cost.

7. What Better? Alternative Approaches

Protobuf over JSON for the wire format — ~5× smaller for the same data; saves cellular bytes.

Deduplication on the server using the event id (UUID generated on device). Survives client retries.

Funnels precomputed on the server side; the SDK stays dumb, knows only how to enqueue.

Crash + ANR capture as a separate SDK module; not analytics' core but commonly bundled.

Open-source reference: see Mixpanel and Amplitude Android SDKs on GitHub for production-grade code.

Key takeaways

- Durable queue (Room) + WorkManager = the canonical pattern.
- Single-thread executor for ordering, never block caller.
- Crash-safe: every public method in try/catch.
- Schema-stable, additive evolution only.

Chapter 15: Design an Offline-First To-Do App with Server Sync

Difficulty: Medium | **Tags: Offline Sync Conflict resolution Room**

The canonical offline-first design — Room as source of truth, sync queue, last-write-wins or smarter conflict resolution.

1. What's the Problem?

Build a Todoist-style to-do app. The user adds, edits, completes, and deletes tasks; the app shows them instantly even without network; everything syncs to a server in the background; another device sees the changes when it comes online; if the same task is edited on two devices, the conflict is resolved sensibly.

Scale to assume

- User has 100–10,000 tasks. Sync is per-user.
- Most operations are offline-friendly creates/edits; large bulk imports are rare.
- Multi-device — phone + tablet + web.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Create, read, update, delete tasks. Lists / folders / tags?
- Due dates, reminders, recurring tasks?
- Multi-device same-account sync?
- Shared lists with other users?
- Conflict policy: last-write-wins, manual merge, CRDT?
- Soft delete (recover from trash) or hard delete?

Non-functional clarifications

- UI is instantaneous — no spinner waiting for network on any write.
- Sync happens within seconds of going online.
- No data loss even with airplane mode + force-kill.
- Conflict resolution is correct and explainable.

Why ask these? (the signal)

Asking about conflict policy reveals you know real offline-first is harder than a sync button.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Local-first: **UI** ↔ **Room (source of truth)**. Every change writes to Room first, then to a pending-ops queue. A **SyncEngine** (WorkManager) drains the queue against the server, fetches remote changes, and merges. The server is authoritative for IDs and ordering; the client owns the user's intent.

Major components

- Task table — id (server or local UUID), title, done, dueDate, syncStatus, version, updatedAt.
- PendingOp table — opType (CREATE/UPDATE/DELETE), taskId, payload, attempts.
- SyncEngine — pushPending(), pullChanges(since=watermark), merge().
- WorkManager periodic worker — runs every 15 min + on connectivity restored.

- ConflictResolver — strategy: LastWriteWins, FieldLevelMerge, UserPrompt.

Low-Level Design (LLD)

IDs: generate locally as UUIDs so offline-created tasks have a stable id. The server accepts the UUID as the canonical id — no rebinding.

Vector or Lamport clocks: each task carries a version number incremented on every edit. On sync, server's version vs client's version determines the conflict.

Outbox pattern: writes go to (Task table) + (PendingOp table) in the same transaction. SyncEngine reads from PendingOp; deletes ops after server ack.

Pull side: server has a 'changed_since(watermark)' endpoint; client persists the last successful watermark.

Conflict resolution (LWW default): compare updatedAt; newest wins. Refine to field-level merge if needed.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is offline-first, not offline-fallback. Every write hits the local DB synchronously; sync is a separate, eventually-consistent process that the user never waits on."*

Show the write path: *"Tap 'add task' → Repository.add(task) → Room.insert(task with syncStatus=PENDING_CREATE) + PendingOp.insert(create payload). UI observes Room and renders instantly. No network call on this path."*

Show the sync path: *"SyncEngine runs every 15 min or on connectivity. For each PendingOp, POST/PUT/DELETE to server; on 200, mark task SYNCED + delete op. Pull side asks 'what changed since X', merges, advances watermark."*

Conflict trick: *"For LWW, every write stamps updatedAt locally. If server's version is newer than ours when we push, server wins; we re-pull and discard our intent. Most conflict-policy bugs are around not telling the user."*

> ■ ***Tip:*** *Bring up **idempotency keys** on PendingOp. Network retries can otherwise create duplicate tasks. Each op carries a UUID; server SETNX's it.*

5. Actual Code (Reference Snippets)

Outbox pattern: write to (task, pending_op) in one transaction; SyncWorker drains.

Kotlin

```kotlin
enum class SyncStatus { SYNCED, PENDING_CREATE, PENDING_UPDATE, PENDING_DELETE }

@Entity(tableName = "task")
data class Task(
    @PrimaryKey val id: String,          // UUID generated on device
    val title: String,
    val done: Boolean,
    val updatedAt: Long,
    val version: Int = 0,
    val syncStatus: SyncStatus = SyncStatus.PENDING_CREATE
)

@Entity(tableName = "pending_op")
data class PendingOp(
    @PrimaryKey val opId: String,        // UUID — idempotency key
    val taskId: String,
    val opType: String,                  // CREATE / UPDATE / DELETE
    val payloadJson: String,
    val attempts: Int = 0
)

class TaskRepository(private val db: AppDatabase, private val api: TaskApi) {
    suspend fun create(title: String) = db.withTransaction {
```

```
        val task = Task(java.util.UUID.randomUUID().toString(), title, false,
                        System.currentTimeMillis(),
                        syncStatus = SyncStatus.PENDING_CREATE)
        db.taskDao().insert(task)
        db.opDao().insert(PendingOp(
            java.util.UUID.randomUUID().toString(),
            task.id, "CREATE",
            JsonConverter.to(task)))
    }

    suspend fun toggle(id: String) = db.withTransaction {
        val t = db.taskDao().get(id) ?: return@withTransaction
        val updated = t.copy(done = !t.done, version = t.version + 1,
                             updatedAt = System.currentTimeMillis(),
                             syncStatus = SyncStatus.PENDING_UPDATE)
        db.taskDao().update(updated)
        db.opDao().insert(PendingOp(
            java.util.UUID.randomUUID().toString(), id, "UPDATE",
            JsonConverter.to(updated)))
    }
}

class SyncWorker(c: Context, p: WorkerParameters) : CoroutineWorker(c, p) {
    override suspend fun doWork(): Result {
        val db = AppModule.db
        val api = AppModule.api
        // 1) Push pending ops
        for (op in db.opDao().all()) {
            try {
                val result = when (op.opType) {
                    "CREATE" -> api.create(op.opId, /*body*/ op.payloadJson)
                    "UPDATE" -> api.update(op.opId, op.taskId, op.payloadJson)
                    "DELETE" -> api.delete(op.opId, op.taskId)
                    else -> error("unknown op")
                }
                if (result.isSuccessful) {
                    db.opDao().delete(op.opId)
                    db.taskDao().markSynced(op.taskId)
                } else if (result.code() == 409) {
                    // server has newer version; drop op and re-pull
                    db.opDao().delete(op.opId)
                }
            } catch (e: IOException) { return Result.retry() }
        }
        // 2) Pull changes
        val watermark = WatermarkStore.get()
        val changes = api.changedSince(watermark)
        db.withTransaction {
            for (remote in changes.tasks) {
                val local = db.taskDao().get(remote.id)
                if (local == null || remote.updatedAt > local.updatedAt) {
                    db.taskDao().upsert(remote.toLocal())  // LWW: server wins on newer
                }
            }
            WatermarkStore.set(changes.serverNow)
        }
        return Result.success()
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why not just call the server first and update UI on success?

That makes every write wait for a network round trip. On a flaky cellular connection, the user sees 'creating…' for 3 seconds. Offline-first reverses the order: write locally first, sync later. The UI feels instant; the user does

not care (or notice) when sync happens.

Q: How do you resolve concurrent edits on two devices?

Last-write-wins by updatedAt is the simplest answer. For a richer model, field-level merge: title from device A, dueDate from device B — both kept. CRDTs (e.g. Yjs) give automatic conflict-free merge but are heavy. For a to-do app, LWW with a 'show me what changed' UI is usually enough.

Q: Why a separate PendingOp table — can't syncStatus on Task be enough?

If a user creates and deletes a task before sync, you need to record both intentions or nothing — but the Task is gone. PendingOp persists the act, not the state. Also lets you replay in order (FIFO), survive partial failures, and apply idempotency.

Q: What if the server's response is lost — how do you avoid double creates?

Idempotency key (opId UUID) on every op. The server stores recent opIds in Redis with TTL; if a duplicate arrives, it returns the prior result instead of creating again.

Q: How do you handle a 1000-task initial sync?

Paginate the pull endpoint; advance watermark per page. Show a 'syncing…' indicator the first time only. After that, delta-sync is small and silent.

7. What Better? Alternative Approaches

CRDTs (Yjs, Automerge) for offline-collaborative editing — every operation is commutative; no central conflict resolution.

WebSocket push on top of pull-based sync; reduces 'time to consistency' from minutes to seconds.

End-to-end encryption if data is sensitive — server stores opaque blobs, client encrypts before push.

Schemaless Task payload (JSON blob) for easy schema evolution without DB migrations on every client.

Background sync from server-side: Firestore / CouchDB give you most of this for free if you can accept the vendor lock-in.

Key takeaways

- Local DB is source of truth.
- Outbox (PendingOp) table makes sync robust to crashes.
- Idempotency keys are non-negotiable for retries.
- Last-write-wins with timestamps is the simplest sane default.

Chapter 16: Design a News Feed (Twitter-style mobile)

Difficulty: Medium | **Tags: Feed Pagination Caching Real-time**

Mobile-side of a feed: API design, pagination, caching, optimistic likes, real-time updates, modular feed items.

1. What's the Problem?

Design the Twitter-feed module for an Android app. It shows a scroll of tweets, supports pull-to-refresh, infinite scroll, like/comment/retweet with optimistic UI, and shows new tweets banner when fresh content arrives. Each tweet may have text, image, video, poll, or be a retweet of another tweet.

Scale to assume

- Active user opens the app 5–20 times/day; sessions are minutes long.
- A tweet card is heterogeneous — text-only, image, video, poll, retweet — needs polymorphic rendering.
- Feed is heavily read; writes (post, like) are personal.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Feed ordering: chronological reverse, ML-ranked, or both as tabs?
- Tweet types — text, image, video, poll, quote-retweet — all in MVP?
- Likes / comments / retweets visible in real time?
- Pull-to-refresh + 'new tweets' banner pattern?
- Mute / block — applied client side, server side, or both?
- Offline caching — last N tweets readable when offline?

Non-functional clarifications

- 60 fps scroll; cold open under 1 second from cache.
- Image loading must not block scroll.
- Optimistic UI for likes / retweets — never feel network latency.
- Memory-bounded — list of 1000 tweets must not OOM.

Why ask these? (the signal)

Asking about tweet-type polymorphism is the senior signal — feed cards are not one type.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

UI = LazyColumn / RecyclerView with multi-view-type adapter. ViewModel exposes PagingData<FeedItem>. Repository uses Paging 3 + RemoteMediator backed by Room. Real-time updates come via WebSocket → write to Room → UI auto-emits. Optimistic actions write to Room first, queue a server call, reconcile on response.

Major components

- FeedItem (sealed) — Text, Image, Video, Poll, Retweet.
- FeedRepository — pulls pages, writes to Room, exposes Flow<PagingData<FeedItem>>.
- Room — feed_item table (heterogeneous via type discriminator).
- ActionRepository — optimistic like/retweet/comment.
- RealtimeChannel — WebSocket subscription per user.

- ItemRenderer — Strategy by type.
- MediaPlayer pool — for inline video autoplay.

Low-Level Design (LLD)

Heterogeneous feed: sealed class FeedItem; in adapter, getItemViewType returns the discriminator; bind dispatches.

Optimistic like: on tap, increment local counter in Room → POST /like → on 200 do nothing, on failure roll back + show toast.

Inline video: use ExoPlayer instances pooled in a small list (typically 3) and reassigned as items scroll. Never create a new player per row.

Pull-to-refresh doesn't reload the whole feed — it adds new tweets at the top while preserving the user's scroll. Use the cursor as in chapter 12.

Image dedup: same image URL in two retweets should fetch once. Image library (chapter 11) handles this.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is Paging 3 + Room + a sealed-class FeedItem so each tweet type has its own ViewHolder. Optimistic mutations write to Room first; the UI never feels server latency on likes."*

Show the action flow: *"User taps heart. ActionRepository.like(tweetId) → Room.update(localLikes += 1) → enqueue WorkManager job → POST /like. UI emits the new count instantly. If the server replies 409 (already liked), reconcile silently."*

Show real-time: *"WebSocket onMessage → parse tweetEvent → Room.update — auto-flows through PagingData and back to LazyColumn. We don't push UI directly from the socket."*

Bring up video: *"ExoPlayer is expensive. We pool 3 instances and assign to whichever item is currently visible. Off-screen items get their player released to save memory and battery."*

> ■ ***Tip:*** *Mention **DiffUtil** (with stable IDs) or Paging 3's automatic DiffCallback — the wrong implementation causes flicker on every refresh. Stable keys = no flicker.*

5. Actual Code (Reference Snippets)

Multi-view-type adapter + optimistic like with rollback

Kotlin

```kotlin
sealed class FeedItem {
    abstract val id: String
    data class Text(override val id: String, val text: String, val author: String,
                    val likes: Int, val liked: Boolean) : FeedItem()
    data class Image(override val id: String, val text: String, val author: String,
                     val imageUrl: String, val likes: Int, val liked: Boolean) : FeedItem()
    data class Video(override val id: String, val text: String, val author: String,
                     val videoUrl: String, val thumbUrl: String) : FeedItem()
    data class Retweet(override val id: String, val original: FeedItem,
                       val retweetedBy: String) : FeedItem()
}

class FeedAdapter : PagingDataAdapter<FeedItem, RecyclerView.ViewHolder>(DIFF) {
    override fun getItemViewType(pos: Int) = when (peek(pos)) {
        is FeedItem.Text    -> 0
        is FeedItem.Image   -> 1
        is FeedItem.Video   -> 2
        is FeedItem.Retweet -> 3
        null -> 0
    }
    override fun onCreateViewHolder(p: ViewGroup, vt: Int): RecyclerView.ViewHolder =
```

```
        when (vt) {
            0 -> TextVH(/* inflate */)
            1 -> ImageVH(/* inflate */)
            2 -> VideoVH(/* inflate */, AppModule.exoPool)
            else -> RetweetVH(/* inflate */)
        }
    override fun onBindViewHolder(h: RecyclerView.ViewHolder, pos: Int) {
        when (val item = getItem(pos)) {
            is FeedItem.Text    -> (h as TextVH).bind(item)
            is FeedItem.Image   -> (h as ImageVH).bind(item)
            is FeedItem.Video   -> (h as VideoVH).bind(item)
            is FeedItem.Retweet -> (h as RetweetVH).bind(item)
            null -> Unit
        }
    }
    override fun onViewRecycled(h: RecyclerView.ViewHolder) {
        if (h is VideoVH) h.releasePlayer()
    }
    companion object {
        val DIFF = object : DiffUtil.ItemCallback<FeedItem>() {
            override fun areItemsTheSame(a: FeedItem, b: FeedItem) = a.id == b.id
            override fun areContentsTheSame(a: FeedItem, b: FeedItem) = a == b
        }
    }
}

class ActionRepository(private val db: AppDatabase, private val api: FeedApi) {
    suspend fun toggleLike(id: String) {
        val current = db.feedDao().get(id) ?: return
        val optimistic = current.copy(liked = !current.liked,
                                      likes = current.likes + if (current.liked) -1 else 1)
        db.feedDao().update(optimistic)
        try { api.toggleLike(id) }
        catch (e: Exception) {
            db.feedDao().update(current)   // rollback
            // surface a toast via shared Flow
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why a sealed class for FeedItem instead of an interface or open class?

Sealed = exhaustive when expressions. When you add a new type (Poll), the compiler tells you every place that handles feed items needs updating. No silent crashes. Senior insight.

Q: How do you prevent video playback from killing battery?

Pool ExoPlayer instances (3 typically). Play only the centered/visible item; pause when scroll-off-screen. Use HLS so buffer is small. Drop to 480p on cellular (player.setMaxVideoSize).

Q: How does a retweet show without duplicating storage?

Retweet.original is just a reference (or a foreign key in Room). The ViewHolder rebinds the original underneath the 'retweeted by X' header. Single source of truth.

Q: How do you handle a tweet being deleted while it's in someone's feed?

Real-time tweet-deleted event → Room.delete → PagingData re-emits and the item smoothly disappears. If not on real-time, next pull-to-refresh catches it; we tolerate brief inconsistency.

Q: Memory: 1000 tweets with images — won't this crash?

PagingData caps in-memory size (maxSize). Image library memory cache is byte-bounded (~1/8 heap). Room keeps the full list on disk; UI only holds what's near the visible window. Together = safe.

7. What Better? Alternative Approaches

Compose instead of XML — LazyColumn handles heterogeneous items more elegantly via @Composable conditionals.

Pre-fetch images for the next 5 rows when entering Paging's prefetch window.

Sticky headers (date dividers) via Compose's stickyHeader or RecyclerView ItemDecoration.

Saved-for-later — bookmark items via a separate 'bookmark' table; same Paging pattern.

Backend-driven UI (BDUI): server sends a JSON describing each card's layout. Add new tweet types without releasing the app. Used by Airbnb / Spotify.

Key takeaways

- Sealed class for heterogeneous feed types.
- Optimistic write → rollback on failure.
- ExoPlayer pooling for autoplay.
- Paging 3 + Room handles infinite scroll for free.

Chapter 17: Design a 1-to-1 Chat (basic WhatsApp)

Difficulty: Medium | **Tags: Real-time WebSocket Offline Receipts**

Two users exchange messages in real time; offline messages sync on reconnect; ticks for sent/delivered/read.

1. What's the Problem?

Build a 1-to-1 messaging feature: two users connected via the app exchange text messages instantly when both are online; when one is offline, messages are queued on the server and delivered when they come back; UI shows single-tick (sent), double-tick (delivered), blue-tick (read).

Scale to assume

- 10M+ DAU; each user sends 30–50 messages a day.
- Message latency target: 95th percentile under 500 ms within the same region.
- Both clients may be on flaky cellular; reconnections are constant.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Text only, or media (images/audio) too?
- Typing indicator? Read receipts toggleable by user?
- Message persistence — server keeps last N days? Or only undelivered?
- End-to-end encryption (Signal protocol)? — out of scope for this chapter.
- Multi-device sync — same user on two devices?
- Edit/delete messages?

Non-functional clarifications

- Real-time within seconds when both online.
- Reliable when offline → online; no message loss; no duplicates.
- Ordering preserved per chat.
- Battery: idle WebSocket cost on Android needs heartbeat tuning.

Why ask these? (the signal)

Asking about multi-device exposes that messages aren't simply server→client — they fan out per-device.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Client opens a **WebSocket** on app start; the server connection-service holds it. When user sends, message goes over WebSocket to the server; server persists, looks up recipient's active connections, and pushes. If recipient is offline, queue. On reconnect, client pulls all messages since its last ack. Receipts (delivered, read) are special messages with the same plumbing.

Major components

- ConnectionService (server) — holds WebSocket map of userId → socket.
- Message table (server) — id, chatId, senderId, recipientId, content, status, timestamp.
- OfflineQueue — Redis list per userId for undelivered messages.
- Mobile: ChatRepository — sends/receives via WebSocket; persists to Room.
- Mobile: MessageDao — local table mirrors server, plus syncStatus.
- OutboxWorker — re-sends pending messages if WebSocket reconnects.

Low-Level Design (LLD)

Message ID: generated on client (UUID). Server uses it for dedup so a retry never creates a duplicate.

Status state machine: PENDING (in client outbox) → SENT (server got it) → DELIVERED (recipient device acked) → READ (recipient opened chat). Each transition is a separate event with the same messageId.

Reconnect protocol: client sends 'subscribe(lastSeenMessageId)' on connect. Server replays anything newer.

Heartbeat: ping every 30s. If 2 misses, force-reconnect. Tune for battery: 60s is fine for most use cases.

Local order: order by timestamp + sequence number. Server timestamps every message on receipt to handle client clock skew.

4. How to Code & Explain (Win the Interviewer)

Open: *"WebSocket-first design with a durable server-side outbox for offline users. Client persists everything in Room so the UI is offline-readable and order is preserved."*

Walk one message: *"User types and hits send → ChatRepo writes Message(status=PENDING) to Room, UI shows the clock icon. Repo sends over WebSocket. Server replies SENT → status → SENT (single tick). Server pushes to recipient, who replies DELIVERED → double tick. Recipient opens chat → READ → blue tick."*

Show reconnect: *"App was offline 10 minutes. On reconnect, client sends 'sub last=msg_42'. Server replays msg_43, 44, 45. Client persists each. UI emits the new messages. Receipts for previously-sent-but-not-acked update too."*

> ■ ***Tip:*** *Mention **FCM as a fallback**: if the user's WebSocket is dead and a new message arrives, the server pushes via FCM to wake the app and reconnect. WebSocket alone won't survive Android Doze.*

5. Actual Code (Reference Snippets)

WebSocket repo with local Room outbox + status state machine + reconnect/replay

Kotlin

```kotlin
enum class MessageStatus { PENDING, SENT, DELIVERED, READ, FAILED }

@Entity(tableName = "message",
        indices = [Index("chatId", "timestamp")])
data class Message(
    @PrimaryKey val id: String,            // UUID generated on send
    val chatId: String,
    val senderId: String,
    val text: String,
    val timestamp: Long,                   // server time once acked
    val status: MessageStatus,
    val localCreatedAt: Long               // for local ordering before server time
)

class ChatRepository(
    private val ws: WebSocketClient,
    private val db: AppDatabase,
    private val me: String
) {
    init {
        ws.events.onEach { handle(it) }.launchIn(GlobalScope)
        ws.onOpen { ws.send(Subscribe(lastSeen = db.messageDao().lastSeenId())) }
    }

    suspend fun send(chatId: String, text: String) {
        val msg = Message(
            id = java.util.UUID.randomUUID().toString(),
            chatId = chatId, senderId = me, text = text,
            timestamp = 0L, localCreatedAt = System.currentTimeMillis(),
```

```
            status = MessageStatus.PENDING
        )
        db.messageDao().insert(msg)             // UI sees it now
        try { ws.send(SendMsg(msg.id, chatId, text)) }
        catch (e: Exception) { /* will retry on reconnect via outbox */ }
    }

    suspend fun markRead(chatId: String, lastMsgId: String) {
        ws.send(ReadReceipt(chatId, lastMsgId))
    }

    private suspend fun handle(ev: ServerEvent) = when (ev) {
        is ServerEvent.Sent ->
            db.messageDao().updateStatus(ev.id, MessageStatus.SENT, ev.timestamp)
        is ServerEvent.Delivered ->
            db.messageDao().updateStatus(ev.id, MessageStatus.DELIVERED)
        is ServerEvent.Read ->
            db.messageDao().updateStatusUpTo(ev.chatId, ev.lastMsgId, MessageStatus.READ)
        is ServerEvent.NewMessage -> {
            db.messageDao().insertIgnore(ev.toLocal())          // dedup by id
            ws.send(DeliveredAck(ev.id))                         // tell sender
        }
    }
}

// WebSocket reconnection with exponential backoff
class WebSocketClient(private val url: String) {
    private var backoffMs = 1000L
    private var attempt = 0
    fun connectWithBackoff() {
        scope.launch {
            while (true) {
                try { connect(); attempt = 0; backoffMs = 1000L; return@launch }
                catch (e: Exception) {
                    delay(backoffMs)
                    backoffMs = (backoffMs * 2).coerceAtMost(30_000)
                    attempt++
                }
            }
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why client-generated UUID for message id?

Idempotency. If the network drops between send and ack, the client retries with the same id. Server SETNX's the id and returns the original ack. No duplicate message on the recipient.

Q: How do you keep ordering when timestamps from two senders are close?

Server stamps a server-side monotonic timestamp on receipt. Ordering by server timestamp is the truth. Client also keeps a localCreatedAt so the message appears in the right place before the server ack arrives.

Q: How do read receipts work across re-opens?

Read receipts are stored as the highest readMessageId per chat per user. UI uses the recipient's readId vs message id to decide blue-tick. When recipient closes and reopens, readId is unchanged — no double events.

Q: Why not just use FCM for every message?

Latency: FCM is best-effort and can take seconds. Pushes are rate-limited. WebSocket is sub-100ms when alive. We use FCM only to wake the device when the socket is down.

Q: How do you handle a 30-day offline user?

Server keeps a per-user inbox queue (Redis or DB). On reconnect, client subscribes with watermark; server replays. Hard cap at, say, 30 days or 10,000 messages; beyond that the server tells the client 'too many; do a

full chat refresh'.

7. What Better? Alternative Approaches

End-to-end encryption (Signal protocol) — see Chapter 26 for full design.

Multi-device sync — every message fans out to all of the recipient's devices; receipts per-device.

Group chats — fan-out N times; covered in Chapter 26.

Media messages — upload to S3 first; send the URL via WebSocket.

Long-lived connection on Android: pin a foreground service for power users; standard apps rely on FCM wakes.

Key takeaways

- WebSocket for low-latency; FCM for wake-up.
- Client-generated UUID = idempotency for free.
- Status state machine: PENDING → SENT → DELIVERED → READ.
- Outbox + replay = reliable on reconnect.

Chapter 18: Design a Movie Ticket Booking App (BookMyShow)

Difficulty: Medium | **Tags: Seats Locking Payment Concurrency**

Browse → pick seats → lock for 5 minutes → pay → confirm. The seat-locking race condition is the core challenge.

1. What's the Problem?

Users browse movies for their city, pick a show, see a seat map, select seats, get a 5-minute lock, pay, and receive a ticket. Many users hit the same popular show simultaneously, but a single seat can only be sold once. Design the mobile client and the seat-locking model.

Scale to assume

- Popular release: 10,000+ users hit the same show at the same minute on opening day.
- Seats per show: 100–500. Lock window: 5 minutes. Strong consistency on seats — no double-booking.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- City list → movies → shows → seats → payment → ticket.
- Seat categories, prices, accessible seats, couple seats?
- Combo offers (popcorn + ticket)?
- Wallet / coupons / partner discounts?
- Cancel & refund window?
- Group bookings — multi-seat select limit?

Non-functional clarifications

- No double-booked seats. Strong consistency on seat state.
- Sub-200ms seat-map load.
- Tolerate payment-gateway slowness (10–30 second roundtrip).
- Mobile-friendly seat map (zoom, pinch, scroll).

Why ask these? (the signal)

Asking about lock duration vs payment latency reveals you understand the real race.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Mobile: **BrowseScreen** → **SeatMap** → **Payment** → **Ticket**. Server: BookingService applies a **distributed lock** on selected seats (Redis SETNX with 5-min TTL). On payment success → seats flip to BOOKED, locks deleted. On timeout or failure → locks expire automatically, seats become available again.

Major components

- Movie / Show / SeatLayout / Seat in catalogue service.
- BookingService — lock(seats), confirm(bookingId, paymentRef), release(bookingId).
- Redis — keys like 'show:{showId}:seat:{seatId}' with TTL.
- Payment gateway integration — Razorpay / Stripe webhook.
- Mobile: SeatMapView (custom canvas), BookingViewModel.

Low-Level Design (LLD)

Lock semantics: Redis SETNX with PX 300000 (5 min). If SETNX fails for any seat, return 'seat unavailable' and release the ones already locked in the same call.

Atomic batch lock: lock seats in a single Lua script so we don't half-lock. Either all seats lock or none.

Payment happens via gateway; on webhook confirm, mobile polls / WebSocket-subscribes to the booking status. Don't block UI on synchronous payment.

Timer on mobile: count down 5 minutes; show 'time remaining'. On expiry, force user to re-select.

Seat map UI: don't redraw on every state change. Use a canvas with a sparse render only of changed cells. Mention Compose's invalidate optimisations or RecyclerView with grid.

4. How to Code & Explain (Win the Interviewer)

Open: *"The race condition on popular shows is the heart of this design. I'll use Redis distributed locks with TTL — even if the user's app crashes during payment, the seats become free again automatically. No manual rollback."*

Show the booking flow: *"User taps 3 seats. App calls POST /lock with the seat IDs. Server runs a Lua script that SETNXs all 3 keys atomically. If any fail, releases the others. Returns bookingId + 5-min TTL."*

Talk payment: *"User pays. Gateway redirects back. App polls /booking/{id}. Server has received the webhook, flipped seats to BOOKED, deleted the locks. Booking confirmed. If payment fails / times out, locks expire → seats freed."*

Mention the UI gotcha: *"Seat map updates from polling can flicker. We render diffs only — the changed cells — and use the Booking status from server, not the local lock TTL, to avoid optimistic-but-wrong UI."*

> ■ ***Tip:*** *Bring up* ***auto-release on browser-tab-close*** *on web; on mobile, use a foreground service or simply 'do not release on backgrounded' — let the TTL handle it.*

5. Actual Code (Reference Snippets)

Mobile-side booking VM + sketch of atomic batch lock on Redis

Kotlin

```kotlin
// Mobile client — booking flow with countdown timer
class BookingViewModel(private val api: BookingApi) : ViewModel() {

    private val _state = MutableStateFlow<BookingState>(BookingState.Idle)
    val state: StateFlow<BookingState> = _state

    fun lockSeats(showId: String, seats: List<String>) {
        viewModelScope.launch {
            _state.value = BookingState.Locking
            try {
                val r = api.lock(showId, seats)
                _state.value = BookingState.Locked(r.bookingId, r.expiresAt)
                startCountdown(r.expiresAt)
            } catch (e: HttpException) {
                _state.value = if (e.code() == 409)
                    BookingState.SeatUnavailable
                else BookingState.Error(e.message ?: "")
            }
        }
    }

    private fun startCountdown(expiresAt: Long) = viewModelScope.launch {
        while (System.currentTimeMillis() < expiresAt) {
            val rem = expiresAt - System.currentTimeMillis()
            _state.update { (it as? BookingState.Locked)?.copy(remainingMs = rem) ?: it }
            delay(1000)
```

```
        }
        _state.value = BookingState.Expired
    }

    fun pay(bookingId: String) = viewModelScope.launch {
        val token = api.startPayment(bookingId)
        // launch payment UI; on return, poll
        repeat(20) {
            val s = api.bookingStatus(bookingId)
            if (s.confirmed) { _state.value = BookingState.Confirmed(s.ticketId); return@launch }
            delay(1500)
        }
        _state.value = BookingState.PaymentTimeout
    }
}

// Server-side seat lock (pseudo / Lua-ish, for explanation)
// KEYS = seats, ARGV[1] = bookingId, ARGV[2] = ttlMs
val LOCK_LUA = '''
for i, key in ipairs(KEYS) do
    if redis.call("EXISTS", key) == 1 then
        for j = 1, i-1 do redis.call("DEL", KEYS[j]) end
        return -1
    end
    redis.call("SET", key, ARGV[1], "PX", tonumber(ARGV[2]))
end
return 1
'''
```

6. Cross-Questions & How to Answer Them

Q: Why Redis locks and not DB row-level locks?

Two reasons: speed (Redis is sub-ms, DB row locks are slower under contention) and TTL (Redis auto-releases on timeout, DB locks held by a dead session need cleanup). For seat booking, TTL on lock is essentially the timeout for the user to complete payment.

Q: What if Redis dies during a booking?

Redis Sentinel / Cluster gives HA. If a lock entry was lost in failover, two users may briefly both think they have the seat. Mitigation: persist a lock claim to the DB too (write-through). The final 'confirm' step does a DB transaction with a uniqueness constraint on (show, seat); the second commit fails and that user is refunded.

Q: How does the mobile app know the lock has expired?

Client counts down based on the server's expiresAt timestamp returned from /lock. On expiry, UI grays out the 'pay' button. Server is the source of truth — if the user pays right at the boundary and server's TTL has already passed, the confirm fails and we refund.

Q: Two users selected the same single remaining seat — what happens?

Both hit /lock. Redis Lua script is single-threaded; only the first SETNX succeeds. Second user gets 409 and a fresh seat map showing the seat as taken.

Q: What about race after payment but before confirm?

Payment is the only step that can take seconds. Lock TTL must exceed gateway p99 latency (5 min is overkill — most gateways are sub-30s). If gateway timeouts, we extend the TTL on user action ('still paying?') or auto-cancel.

7. What Better? Alternative Approaches

Pre-booking warm-up for blockbuster shows: pre-allocate Redis shards, scale gateway, cache seat layouts on CDN.

Waitlist queue: on seat unavailable, offer to join a waitlist; on cancellation by another user, notify next in line.

Group booking: bundle multiple users' selections into one transaction; one fails, all fail.

Server-driven UI for the seat map — server decides layout, prices, and special offers per show, app just renders.

Anti-bot: rate-limit, CAPTCHA on high-traffic shows, device fingerprinting.

Key takeaways

- Redis locks with TTL beat DB row locks for short-lived holds.
- Atomic batch lock via Lua = no half-locked bookings.
- Mobile countdown UI based on server's expiresAt, not local timer.
- Final DB confirm with UNIQUE constraint is the ultimate safety net.

Chapter 19: Design an E-commerce App's Home Page (Amazon mobile)

Difficulty: Medium | **Tags: BDUI Heterogeneous lists Caching Personalisation**

A long scroll of widgets — carousels, banners, product rows, ads. Backend-driven UI is the production answer.

1. What's the Problem?

Design the home page of a shopping app (Amazon / Flipkart / Myntra). It's a vertical scroll containing many widget types: promo banner, hero carousel, 'recommended for you' product grid, 'recently viewed', category tiles, sponsored products, etc. The mix changes per user, per region, per A/B test, and updates frequently — sometimes without an app release.

Scale to assume

- Tens of millions of users, hundreds of thousands of products.
- Home page is the most-viewed screen — 60+% of session opens.
- Personalisation: top widgets and rows differ per user.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Which widget types — banner, carousel, product grid, video, ads?
- Personalisation — fully ranked or fixed-template?
- A/B testing — different layouts per cohort?
- Add-to-cart from home (quick add)?
- Skeleton state, error state, empty state?
- Offline / cached view of last home?

Non-functional clarifications

- Cold open under 1 second from cache; full home loaded under 2 seconds.
- New widget types can ship without an app release.
- Per-user personalisation; minimal latency for ML ranker.
- 60 fps scroll with mixed widgets.

Why ask these? (the signal)

Asking 'can we ship new widget types without an app release' steers you straight to backend-driven UI — the production answer.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Backend-driven UI (BDUI). Mobile asks `GET /home` and receives a JSON list of **widgets**, each with a `type` and a typed `data` blob. The app has a renderer per widget type that knows how to lay it out. New widget types are added by deploying a new server response + an app build that includes the renderer; until then unknown types are silently skipped.

Major components

- HomeService (server) — composes widgets from feature services (search, recs, banners, ads).
- Widget schema — { id, type, data, version }.

- Mobile WidgetRegistry — Map<String, WidgetRenderer>.
- WidgetRenderer (interface) — bind(parent, data) and prefetch().
- HomeRepository — caches the response in Room with TTL; serves stale-while-revalidate.
- ImpressionTracker — emits analytics events when each widget enters the viewport.

Low-Level Design (LLD)

WidgetRenderer interface: every widget type implements bind, optionalPrefetch, getViewType. The RecyclerView adapter uses widget.type.hashCode() as the view type so layouts are cached and reused.

Stable IDs: widget.id is the stable key for DiffUtil. Same widget moved up/down doesn't rebind.

Forward compatibility: if the app receives an unknown widget type, skip it. Don't crash. Server can also send a placeholder type that the app already supports.

Cache strategy: stale-while-revalidate. Always serve from Room immediately (instant render), kick off a background refresh, swap in the new data.

Impressions: only count after 50% of widget is visible for 1+ second — RecyclerView.OnScrollListener + getGlobalVisibleRect.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is backend-driven UI. The app is a renderer; the server decides the layout. New widget types ship by adding a new renderer in the app once; from then on, the server can compose home pages with arbitrary widget mixes."*

Walk a request: *"App opens. HomeRepository reads cached widgets from Room — instant render, may be slightly stale. Fires GET /home. Response replaces the cache. RecyclerView diffs by stable IDs — only changed widgets re-bind."*

Show extensibility: *"Marketing wants a new 'spin the wheel' game widget. Backend adds a 'gamified_offer' widget type. App build N+1 includes the renderer. Until users update, the server tags their cohort and excludes that widget type. Zero crashes."*

> ■ ***Tip:*** *Mention **Lottie** for hero animations and **Glide preload** for product images on off-screen widgets — performance perception matters more than actual latency.*

5. Actual Code (Reference Snippets)

BDUI scaffold: WidgetRegistry + per-type renderers + ListAdapter

Kotlin

```kotlin
// Generic widget shape
data class Widget(val id: String, val type: String, val dataJson: String)

interface WidgetRenderer<T> {
    val viewType: Int
    fun create(parent: ViewGroup): RecyclerView.ViewHolder
    fun bind(vh: RecyclerView.ViewHolder, data: T)
    fun parse(json: String): T
}

object WidgetRegistry {
    private val byType = mutableMapOf<String, WidgetRenderer<*>>()
    fun register(type: String, r: WidgetRenderer<*>) { byType[type] = r }
    fun rendererFor(type: String): WidgetRenderer<*>? = byType[type]
}

class HomeAdapter : ListAdapter<Widget, RecyclerView.ViewHolder>(DIFF) {
    override fun getItemViewType(pos: Int): Int =
        WidgetRegistry.rendererFor(getItem(pos).type)?.viewType ?: -1
```

```
    override fun onCreateViewHolder(p: ViewGroup, vt: Int): RecyclerView.ViewHolder {
        val r = WidgetRegistry.byType.values.first { it.viewType == vt }
        return r.create(p)
    }
    override fun onBindViewHolder(h: RecyclerView.ViewHolder, pos: Int) {
        val w = getItem(pos)
        val r = WidgetRegistry.rendererFor(w.type) ?: return
        @Suppress("UNCHECKED_CAST")
        (r as WidgetRenderer<Any>).bind(h, r.parse(w.dataJson))
    }
    companion object {
        val DIFF = object : DiffUtil.ItemCallback<Widget>() {
            override fun areItemsTheSame(a: Widget, b: Widget) = a.id == b.id
            override fun areContentsTheSame(a: Widget, b: Widget) = a == b
        }
    }
}

// Banner renderer example
data class BannerData(val imageUrl: String, val deepLink: String)

object BannerRenderer : WidgetRenderer<BannerData> {
    override val viewType = "banner".hashCode()
    override fun create(parent: ViewGroup) = BannerVH(/* inflate banner_item */)
    override fun bind(vh: RecyclerView.ViewHolder, d: BannerData) {
        (vh as BannerVH).bind(d)
    }
    override fun parse(json: String): BannerData =
        Json.decodeFromString(json)
}

// Application setup
fun registerWidgets() {
    WidgetRegistry.register("banner", BannerRenderer)
    WidgetRegistry.register("carousel", CarouselRenderer)
    WidgetRegistry.register("product_grid", ProductGridRenderer)
    // Unknown server types just skip rendering — forward compatible
}
```

6. Cross-Questions & How to Answer Them

Q: How is BDUI different from a server returning a list of products?

Products are just data. BDUI returns layout + data — server decides the order, types, headers, sub-titles, image aspect ratios, even animations. The app becomes a thin renderer; product, marketing, and ML teams can ship UI changes without app releases.

Q: How do you avoid bloating each widget's JSON?

Two strategies: (1) thin widget refs — server returns widget.id + widget.type; app fetches data details via batched API. (2) Compress: gzip the response. Most apps do (2).

Q: How do you A/B test widget order?

Server-side. The home composer reads the user's experiment bucket and produces a different layout. Mobile is unaware. Mobile reports impressions/clicks tagged with the variant id.

Q: How do you make this offline?

Cache the last home JSON in Room. On no network, render from cache with a small 'offline' indicator. Product images are served from Glide's disk cache.

Q: How do you handle a widget whose data is slow (e.g. recommendations)?

Two-phase: server returns the home with the rec-widget marked 'loading'. App shows skeleton. Server pushes the rec data via WebSocket or app polls /rec/{userId}. Replaces the skeleton in place using stable IDs.

7. What Better? Alternative Approaches

Compose-based BDUI — Slot APIs map naturally to widget trees; e.g. Slack's BlockKit, Airbnb's Epoxy.

Diff at the field level, not just the widget level — re-bind only the changed sub-component.

Pre-warming images and product detail pages on widget impression so navigation feels instant.

Edge personalisation: a CDN with edge logic (Cloudflare Workers) personalises the home for cheap, fast responses.

Open-source references: Airbnb Epoxy, Instacart's BDUI talks.

Key takeaways

- BDUI = server owns layout, app owns rendering.
- Forward compatibility: unknown widget types skip silently.
- Stable widget IDs power smooth diffs.
- Stale-while-revalidate keeps cold start instant.

Chapter 20: Design a Stock Ticker Screen (Real-time Quotes)

Difficulty: Medium | **Tags: WebSocket Throttling Real-time UI Background sync**

Hundreds of tickers stream updates per second. UI must feel alive without burning the CPU or battery.

1. What's the Problem?

Build a stock ticker page: 50–200 watchlist stocks each receiving price updates several times per second. Show last price, change, percent change, flashing background on tick. Charts update intraday. Off-screen tickers should not waste CPU.

Scale to assume

- 100–200 active tickers; each may receive 5–20 ticks/sec at peak.
- Bandwidth tight on cellular; payload per tick must be small.
- Mobile battery cost matters — can't render 60 fps animations for off-screen rows.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Tick frequency — every tick, or aggregated at 1Hz / 5Hz?
- Charts intraday with live tick? Or charts on demand only?
- Conditional alerts (e.g. notify when AAPL crosses $200)?
- Pre-market / after-hours data?
- Watchlist editing — add/remove on the fly?

Non-functional clarifications

- UI must not stutter — throttle updates to one render per 16ms per row.
- Battery: pause stream when app backgrounded.
- Network: WebSocket with binary or compact JSON; resume on reconnect.
- Memory: store only current price + minimal history; full chart fetched on demand.

Why ask these? (the signal)

Asking about throttling reveals you understand naive 'render every tick' kills the main thread.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Server pushes ticks over **WebSocket** to a per-user subscription. App keeps an in-memory map `symbol → latest price`. A **throttler/coalescer** emits at most one UI update per row per frame. Only currently-visible rows are bound — RecyclerView's ViewHolder is updated; off-screen rows update the model silently.

Major components

- QuoteStreamRepository — WebSocket; subscribe(symbols); Flow<Tick>.
- QuoteCache — ConcurrentHashMap<String, Quote> latest snapshot.
- Throttler — coalesces ticks; emits at 16ms boundaries.
- RecyclerView Adapter — observes only the visible window; updates via payloads (DiffUtil).
- ChartRepository — pulls on demand; subscribes to ticks while open.
- Lifecycle observer — pauses subscription when app backgrounded.

Low-Level Design (LLD)

Subscription protocol: subscribe([AAPL, MSFT, ...]); server responds with snapshot + then streams deltas. On reconnect, resubscribe.

Coalescing: maintain a 'dirty set' of symbols; every 16ms emit the set to the UI thread; clear it. 100 ticks/sec on AAPL = 6 renders/sec — invisible to the eye, easy on the CPU.

Partial bind: instead of rebinding the whole row, send a payload (e.g. PRICE_CHANGED). The ViewHolder's onBindViewHolder(holder, position, payloads) updates only the price TextView. 10× cheaper.

Visible window: keep a Set of currently-bound symbols and only emit ticks to UI for those. Off-screen, just update the cache.

Flash animation: set tint based on delta sign; use ValueAnimator to fade back. Cancel on next bind.

4. How to Code & Explain (Win the Interviewer)

Open: *"Naive bind-on-every-tick burns the main thread. I coalesce ticks at 16ms — one frame — and use partial-bind payloads so each row updates one TextView, not the whole layout."*

Show the flow: *"WebSocket onMessage → QuoteCache update + dirtySet.add(symbol) → on the next 16ms tick of the throttler, snapshot the dirtySet, send to adapter, clear. Adapter triggers notifyItemChanged(pos, 'PRICE_CHANGED') for each visible symbol."*

Background: *"Lifecycle.onStop → unsubscribe(); WebSocket goes idle; battery saved. Lifecycle.onStart → resubscribe; server replies with current snapshot + streams."*

> ■ ***Tip:*** *Show the **partial-bind** code (RecyclerView's payloads parameter). It's the single most senior trick in real-time UIs and most candidates miss it.*

5. Actual Code (Reference Snippets)

60 Hz throttler + partial-bind via RecyclerView payloads

Kotlin

```kotlin
data class Quote(val symbol: String, val price: Double,
                 val change: Double, val pct: Double, val ts: Long)
data class Tick(val symbol: String, val price: Double, val ts: Long)

class QuoteRepository(private val ws: WebSocketClient) {
    private val cache = java.util.concurrent.ConcurrentHashMap<String, Quote>()
    private val dirty = java.util.concurrent.ConcurrentHashMap.newKeySet<String>()
    private val _flushes = MutableSharedFlow<Set<String>>(extraBufferCapacity = 1)
    val flushes: SharedFlow<Set<String>> = _flushes

    init {
        ws.events
            .onEach { onTick(it) }
            .launchIn(GlobalScope)
        GlobalScope.launch {                        // 60Hz throttler
            while (true) {
                delay(16)
                if (dirty.isNotEmpty()) {
                    val snap = dirty.toSet()
                    dirty.clear()
                    _flushes.tryEmit(snap)
                }
            }
        }
    }

    fun snapshot(symbol: String) = cache[symbol]
```

```
    private fun onTick(t: Tick) {
        val prev = cache[t.symbol]
        val ch = if (prev != null) t.price - prev.price else 0.0
        val pct = if (prev != null && prev.price > 0) ch / prev.price * 100 else 0.0
        cache[t.symbol] = Quote(t.symbol, t.price, ch, pct, t.ts)
        dirty.add(t.symbol)
    }

    fun subscribe(symbols: List<String>) = ws.send(Sub(symbols))
}

class TickerAdapter(private val repo: QuoteRepository) :
    RecyclerView.Adapter<TickerVH>() {
    private val symbols = mutableListOf<String>()
    fun setWatchlist(s: List<String>) {
        symbols.clear(); symbols += s; notifyDataSetChanged()
        repo.subscribe(s)
    }
    override fun onBindViewHolder(h: TickerVH, p: Int) =
        h.bind(repo.snapshot(symbols[p]) ?: Quote(symbols[p], 0.0, 0.0, 0.0, 0))
    override fun onBindViewHolder(h: TickerVH, p: Int, payloads: List<Any>) {
        if (payloads.contains("PRICE_CHANGED")) {
            h.bindPriceOnly(repo.snapshot(symbols[p])!!)
        } else super.onBindViewHolder(h, p, payloads)
    }
    fun onFlush(dirty: Set<String>) {
        symbols.forEachIndexed { i, sym ->
            if (sym in dirty) notifyItemChanged(i, "PRICE_CHANGED")
        }
    }
    override fun getItemCount() = symbols.size
}
```

6. Cross-Questions & How to Answer Them

Q: Why coalesce instead of rendering every tick?

AAPL might tick 20×/sec. 200 stocks × 20 ticks = 4000 renders/sec. The main thread runs at 60 fps = 60 frames/sec. Most renders would be dropped anyway. Coalescing to one-per-frame is provably the maximum useful rate.

Q: How do you avoid flicker when the price changes back and forth?

Flash colour based on the *previous* committed price, not the previous tick. Coalesce direction over the 16ms window; if net change is positive, flash green; if zero net, no flash.

Q: How is this different from chat where every message must render?

Chat messages are unique events that all matter. Stock ticks are values where only the latest matters. The right abstraction is a snapshot, not a log.

Q: How would you handle a 30-second mobile disconnect?

WebSocket reconnect → resubscribe → server sends a fresh snapshot of all subscribed symbols (one big payload), then resumes deltas. Cache replaces stale entries; UI auto-updates via flush.

Q: Memory at 1000 symbols — issue?

1000 Quote objects ~100 KB. Trivial. The concern is the dirty-set; bounded at 1000. We're fine.

7. What Better? Alternative Approaches

Binary protocol (Protobuf, FlatBuffers) — smaller payload per tick, faster parse than JSON.

Server-side aggregation: instead of pushing every tick, push 200ms-aggregated snapshots. Lower bandwidth, ~no UX difference.

Charts: use a sliding-window buffer (last 5 min ticks) and a custom canvas Drawable; avoid GraphView libraries that allocate per-frame.

Backpressure: if the dirty set is consistently growing each frame, lower the subscription rate (drop to 5Hz aggregated).

Key takeaways

- Coalesce ticks to one frame.
- Partial-bind via RecyclerView payloads.
- Update cache always; update UI only for visible rows.
- Pause subscriptions when app is backgrounded.

Chapter 21: Design a Tinder-style Swipe App

Difficulty: Medium | **Tags: Swipe UX Prefetching Matching Image-heavy**

Swipe cards left/right, never wait for the next profile to load, never see the same profile twice.

1. What's the Problem?

Build a swipe-card UI like Tinder. The user swipes right (like) or left (pass). The next card is ready instantly. Behind the scenes the server fetches new candidates, matches are computed when both users like each other, and a match notification is delivered.

Scale to assume

- Users can swipe 100+ cards per session.
- Image-heavy — each profile has 3–6 photos.
- Latency matters: 0 ms between swipe and next card.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Swipe right / left / super-like? Undo (rewind)?
- Filter — distance, age, interests?
- Match notification — push? In-app banner?
- Profile prefetching — how many candidates ahead?
- Daily limit on right-swipes (free tier)?
- Block / report inappropriate users?

Non-functional clarifications

- Next-card render must be 0 ms — never block on network.
- Image prefetch — first photo of next 5 cards already decoded.
- Battery — image cache should not OOM on long sessions.
- Match detection latency under 2 seconds end-to-end.

Why ask these? (the signal)

Asking about prefetching window shows you understand the magic isn't the swipe gesture — it's the invisible network priming.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Client keeps a **card stack** of, say, 10 candidates in memory. On every swipe, pop top, render the next; trigger fetch if remaining < 5. Each swipe is sent to the server (POST /swipe). Server checks the symmetric like and creates a Match if both sides liked. Match arrives via WebSocket / FCM.

Major components

- CandidateRepository — fetch batches of profiles; track seen-set to avoid repeats.
- Stack in ViewModel — current N candidates + next K prefetched.
- Card view (Compose or custom View) — drag/throw gesture.
- SwipeRepository — POST /swipe, optimistic UI.
- MatchRepository — listens for match events; pushes to UI as a banner / dialog.
- ImageLoader (chapter 11) — prefetches the top photo of upcoming cards.

Low-Level Design (LLD)

Card stack rendered as Z-stacked Compose elements: only top is interactive; next 2–3 are visible behind with scale + offset for depth.

Swipe physics: track gesture; commit on threshold (>30% width or velocity > X). Animate off-screen on commit; rebind the next.

Prefetch trigger: when stack size drops below threshold, fire async fetch. Show a loading spinner only if the stack hits empty (rare).

Seen-set: server filters out already-swiped; client also tracks the last batch IDs to avoid duplicates if the server returns a too-broad page.

Match detection: server has a Postgres table likes(from, to). On INSERT, check WHERE from=this.to AND to=this.from. If exists, INSERT match(a, b).

4. How to Code & Explain (Win the Interviewer)

Open: *"The magic of Tinder is invisible prefetching. The user never waits because we always have 5+ candidates queued in memory with their first image decoded."*

Show the swipe: *"User drags. Past threshold, we commit. Local stack pops; the next card slides forward. POST /swipe is fire-and-forget — the user's UI is already past it."*

Show the match: *"WebSocket pushes 'match(alice, bob, matchId)'. App shows a match modal. If app is killed, FCM wakes it; the modal appears on next launch."*

Edge case worth flagging: *"Network blips can lose swipes. Same outbox pattern as offline-first todo — write swipe locally with PENDING, sync later. Without it, you'd undercount likes and miss matches."*

> ■ ***Tip:*** *Talk about the* ***'card boost'*** *business logic — paid users get their profile shown to more people. The mobile app doesn't care; the server feed algorithm does. But mention you'd build the feed algo as a Strategy on server side.*

5. Actual Code (Reference Snippets)

Compose card stack + ViewModel with prefetch on threshold

Kotlin

```kotlin
data class Candidate(val id: String, val name: String, val age: Int,
                     val photos: List<String>, val bio: String)

class SwipeViewModel(private val repo: CandidateRepository,
                     private val swipeRepo: SwipeRepository,
                     private val imageLoader: ImageLoader) : ViewModel() {
    private val _stack = MutableStateFlow<List<Candidate>>(emptyList())
    val stack = _stack.asStateFlow()
    private val seen = mutableSetOf<String>()
    private val prefetchThreshold = 5
    private val batchSize = 10

    init { refill() }

    fun onSwipe(candidate: Candidate, direction: Swipe) {
        // 1) optimistic pop
        _stack.update { it - candidate }
        // 2) record swipe (offline-safe)
        viewModelScope.launch { swipeRepo.record(candidate.id, direction) }
        // 3) prefetch if low
        if (_stack.value.size < prefetchThreshold) refill()
    }
    private fun refill() = viewModelScope.launch {
```

```
        val batch = repo.fetch(batchSize, exclude = seen)
        seen += batch.map { it.id }
        // prefetch first image of each
        batch.forEach { c -> imageLoader.prefetch(c.photos.first()) }
        _stack.update { it + batch }
    }
}

enum class Swipe { LIKE, PASS, SUPER }

// Custom swipe-able Compose card stack
@Composable
fun SwipeStack(stack: List<Candidate>, onSwipe: (Candidate, Swipe) -> Unit) {
    Box {
        // Render top 3, reversed so top is drawn last
        stack.take(3).reversed().forEachIndexed { idx, c ->
            val isTop = idx == stack.take(3).lastIndex
            val offset = remember(c.id) { Animatable(Offset.Zero, Offset.VectorConverter) }
            Card(modifier = Modifier
                .offset { IntOffset(offset.value.x.toInt(), offset.value.y.toInt()) }
                .scale(1f - (stack.take(3).size - 1 - idx) * 0.05f)
                .pointerInput(c.id) {
                    if (!isTop) return@pointerInput
                    detectDragGestures(onDragEnd = {
                        val x = offset.value.x
                        when {
                            x > 300 -> { /* animate off right */; onSwipe(c, Swipe.LIKE) }
                            x < -300 -> { /* animate off left */; onSwipe(c, Swipe.PASS) }
                            else -> { /* snap back */ }
                        }
                    }) { _, d ->
                        scope.launch { offset.snapTo(offset.value + d) }
                    }
                }) {
                CandidateCardContent(c)
            }
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why prefetch image but not the whole profile?

The first photo is what the user sees instantly when the card surfaces. Bio and other photos are revealed on tap and can load lazily. Prefetching all photos for 10 cards would waste cellular bandwidth — many will be left-swiped.

Q: How do you guarantee no profile shown twice?

Server filters by seen-set per user (Redis Set, persists across sessions). Client also tracks locally to be safe — race window if server hasn't received the swipe yet but is also fetching.

Q: What if both users tap right on each other at the same millisecond?

Database INSERT...ON CONFLICT DO NOTHING with the (lower, higher) tuple as the unique key. Single match row created; either insert wins. Then a single match-created event fan-outs to both.

Q: How would you implement 'undo' (rewind)?

Push the last swipe onto a stack in the ViewModel; on undo, send POST /unswipe, pop the swipe, push the candidate back onto the top of the stack. Limit to last 1 or last 3 to avoid abuse.

Q: Match notification arrives but app is killed.

FCM. The match server fires a notification; tapping it deep-links to /matches/{id}. State is fetched on open.

7. What Better? Alternative Approaches

ML re-ranking: the order of cards is not random; server scores candidates by predicted compatibility and shows highest first.

Adaptive prefetch: faster swipers get larger windows. Track swipes per minute and prefetch accordingly.

Photo quality progressive: load thumb first, full quality after card surfaces.

Match game: when a match is made, instant message window opens — bridge to your chat module.

Fraud detection: profiles with super-fast right-swipe-everything patterns get throttled (bot-like behaviour).

Key takeaways

- Prefetch + in-memory stack = zero perceived latency.
- Outbox pattern for swipes — offline-safe.
- Image library does the heavy lifting for photo prefetch.
- Match = symmetric INSERT detection on server.

Chapter 22: Design Instagram Stories (mobile)

Difficulty: Medium | **Tags: Media Auto-play Prefetch Lifecycle**

Ephemeral 24-hour stories — tap to skip, swipe to next user, prefetch the next, auto-track 'seen'.

1. What's the Problem?

Implement Stories: a horizontal list of users with unseen stories at top of the feed. Tap a user → enter story viewer → each story plays for 5s (image) or its video duration, auto-advances → tap right to next within the same user → swipe right to next user. After 24 hours, stories disappear. Stories the user has already seen are dimmed.

Scale to assume

- A heavy user views 100+ stories per session.
- Each story is an image or video; needs CDN with low latency.
- Story content is short-lived but high-traffic on creation.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Story types — image (5s timer), video (own duration), text/canvas?
- Reactions / replies inside the story viewer?
- Mute / hide a user's stories?
- Seen tracking — sync immediately or batched?
- Re-watching — allowed?
- Stickers, polls, mentions inside story?

Non-functional clarifications

- Tap-to-next is 0ms (prefetched).
- Video plays smoothly even on cellular.
- Battery: story viewer must release players when exiting.
- Memory: never decode more than necessary.

Why ask these? (the signal)

Asking about prefetch + player lifecycle reveals you've thought about the smooth-tap-to-next feel.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Story tray at top of feed shows users with unseen stories. Tapping enters the **StoryViewer** — a horizontal pager of users, each user is a vertical/tap pager of their stories. **Prefetcher** primes the next story (and the next user's first story) so taps and swipes never wait. Seen events batched and flushed on exit.

Major components

- Story tray adapter (RecyclerView horizontal).
- StoryViewer (ViewPager2 horizontal across users).
- StoryPager (per user) — vertical pager / overlay with progress bars.
- StoryProgressView — N segments at top, current fills over time.
- Prefetcher — uses image loader + ExoPlayer prepare for next.
- SeenRepository — batched event flush to server.

Low-Level Design (LLD)

Two pagers: horizontal across users; within each user, your own progress controller for stories.

Progress bar: ValueAnimator from 0→100 over story duration; on segment complete, advance.

Player pool: one ExoPlayer for the current video, one being prepared for the next. Three at most.

Prefetch: for the current user's next story; for the next user's first story. Don't prefetch deeper to save data.

Seen tracking: write seen events to a buffer; flush on viewer exit, on every 5 stories, and on app stop.

4. How to Code & Explain (Win the Interviewer)

Open: *"The signature feel is zero-latency next-story. That's all prefetch — we prepare the next image and the next video before the user can tap."*

Walk one story: *"User taps Alice. StoryViewer opens at Alice/story0. Progress bar starts a 5s ValueAnimator. Prefetcher fires for Alice/story1 image and Bob's first story image. On complete, the bar fills, story switches to story1 — image is already in memory cache, swap is instant."*

Player handoff: *"For video stories, we have player A playing story0 and player B preparing story1 in parallel. On segment end, swap A↔B. Player A starts preparing story2. Two players in flight, never more."*

Lifecycle: *"onPause inside the viewer → pause animator + player. onResume → resume. Exit viewer → release both players, flush seen buffer to server."*

> ■ ***Tip:*** *Mention* ***holding-tap-to-pause*** *— the iconic Instagram gesture. Pause both the timer and the player on DOWN; resume on UP. Tiny detail; huge UX signal.*

5. Actual Code (Reference Snippets)

Stories viewer ViewModel with prefetch and seen tracking

Kotlin

```kotlin
data class Story(val id: String, val userId: String, val type: StoryType,
                 val url: String, val durationMs: Long, val createdAt: Long)
enum class StoryType { IMAGE, VIDEO }

class StoryViewerViewModel(
    private val repo: StoryRepository,
    private val imageLoader: ImageLoader,
    private val playerPool: PlayerPool
) : ViewModel() {

    private val _state = MutableStateFlow(ViewerState())
    val state = _state.asStateFlow()

    fun show(users: List<UserStories>, startUser: Int) {
        _state.value = ViewerState(users = users, userIdx = startUser, storyIdx = 0)
        prefetchAround()
    }

    fun nextStory() {
        val cur = _state.value
        val user = cur.users[cur.userIdx]
        if (cur.storyIdx + 1 < user.stories.size) {
            _state.update { it.copy(storyIdx = it.storyIdx + 1) }
        } else nextUser()
        markSeen(user.stories[cur.storyIdx].id)
        prefetchAround()
    }

    fun previousStory() {
        val cur = _state.value
```

```
        if (cur.storyIdx > 0) _state.update { it.copy(storyIdx = it.storyIdx - 1) }
        else previousUser()
    }

    private fun nextUser() {
        val cur = _state.value
        if (cur.userIdx + 1 < cur.users.size)
            _state.update { it.copy(userIdx = it.userIdx + 1, storyIdx = 0) }
        else exit()
    }

    private fun prefetchAround() {
        val cur = _state.value
        val user = cur.users[cur.userIdx]
        // Prefetch next story in current user
        user.stories.getOrNull(cur.storyIdx + 1)?.let { prefetch(it) }
        // First story of next user
        cur.users.getOrNull(cur.userIdx + 1)?.stories?.firstOrNull()?.let { prefetch(it) }
    }

    private fun prefetch(s: Story) {
        when (s.type) {
            StoryType.IMAGE -> imageLoader.prefetch(s.url)
            StoryType.VIDEO -> playerPool.warm(s.url)
        }
    }

    private fun markSeen(storyId: String) {
        viewModelScope.launch { repo.recordSeen(storyId) }
    }
}

// Progress segments
@Composable
fun StoryProgressBar(segments: Int, current: Int, progress: Float) {
    Row(Modifier.fillMaxWidth().padding(8.dp)) {
        repeat(segments) { idx ->
            val fill = when {
                idx < current -> 1f
                idx == current -> progress
                else -> 0f
            }
            LinearProgressIndicator(
                progress = fill,
                modifier = Modifier.weight(1f).padding(horizontal = 2.dp).height(2.dp)
            )
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why two ExoPlayer instances in the pool?

One plays the current video; one is prepared for the next. Single player would have a perceptible buffer time on switch. Three+ would waste memory for no UX gain since the user can only see one at a time.

Q: How is 'seen' batched?

SeenRepository buffers events in memory; flushes on (a) every 5 stories, (b) viewer exit, (c) app onStop. Server tolerates duplicates (idempotent set add).

Q: What about hide-this-user's-stories?

Server-side preference applied at story-tray generation. Client passes a local 'muted' set to its tray filter so changes take effect without a server roundtrip.

Q: How do you handle a 24-hour-old story still in someone's viewer?

Server returns stories newer than now-24h. Edge case: user opened the viewer just before expiry — let them finish current story; next-prefetch filters out the expired one.

Q: Memory: image cache fills up after 100 stories.

Image library is byte-bounded; LRU evicts. Story-specific concern: video files in ExoPlayer cache. Limit pool size; delete cached video on viewer exit if memory pressure (onTrimMemory).

7. What Better? Alternative Approaches

HLS for stories instead of MP4 — smaller initial download, smoother bitrate adaptation. Many apps do this.

Skeleton tray with shimmer while the tray loads.

Story view analytics for creators: who watched, when. Privacy-aware aggregation.

Reactions inside the viewer — write to a 'reaction' table, fan-out to creator's notifications.

Polls / quizzes: interactive overlays that read/write through repo.

Key takeaways

- Prefetch the next story image + warm next player.
- Two-player pool for video handoff.
- Batched 'seen' flush — never one network call per tap.
- Hold-to-pause is the signature interaction; bake it in.

Chapter 23: Design a Mobile Payment App (Google Pay / Paytm)

Difficulty: Medium | **Tags: Payments Security UPI Idempotency**

P2P money transfer — secure flow, idempotency, network resilience, biometric authorisation.

1. What's the Problem?

Build a UPI-style payment app. The user can scan a QR code (or pick a contact), enter an amount, authenticate via biometric/PIN, and money moves between bank accounts. Show transaction history. Handle network failures, retries, and payment status (pending/success/failed) without ever double-charging.

Scale to assume

- 100M+ users; peak 5,000+ transactions per second.
- Strict regulatory compliance — RBI / PCI-DSS / SOC-2.
- Network is unreliable on cellular; failures must be safe.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Payment rails — UPI, card, wallet — all in scope?
- Recipient discovery — phone number, UPI ID, QR scan?
- P2P only or merchant payments too?
- Recurring payments / mandates?
- Transaction history with filters?
- Refund and dispute flow?

Non-functional clarifications

- Idempotency — a retry must never double-debit.
- Strong authentication — biometric or PIN every transaction.
- Sub-3-second p95 end-to-end latency.
- Transaction state must be queryable for audit (immutable log).
- No sensitive data in app logs ever.

Why ask these? (the signal)

Asking about idempotency keys is the #1 senior signal for any payment system.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Mobile: PaymentForm → AuthPrompt (biometric) → Submit. **Server**: PaymentService creates a Transaction with client-supplied `idempotencyKey`, calls the bank API (NPCI for UPI), polls / awaits webhook, updates status. Mobile **polls or subscribes via WebSocket** for status. Final status is one of SUCCESS / FAILED / PENDING. PENDING transactions can be reconciled later.

Major components

- PaymentRepository (mobile) — submit(transferRequest, idempotencyKey).
- Transaction model — id, idempotencyKey, amount, sender, recipient, status, createdAt, updatedAt.
- BiometricPrompt — BiometricPrompt API for auth before submitting.

- Status poller / WebSocket subscriber.
- Server PaymentService — bank API integration, retries, reconciliation.
- TransactionLog (immutable, append-only).

Low-Level Design (LLD)

Idempotency key: generated on the client (UUID) when the user taps Pay. Sent with the request. Server SETNX's it in Redis. If a retry arrives with the same key, server returns the original result, never re-submits to the bank.

Biometric authorisation: BiometricPrompt with CryptoObject (signed by Android Keystore). The signature is sent to server proving the user authorised — never log or store the biometric itself.

PENDING is a real state: bank API may not give immediate confirmation. App shows 'Processing…' with a 'check later' button; server has a reconciler that polls the bank and updates.

Status query: GET /transaction/{idempotencyKey} returns the current state. Used both for retries and for app reopens.

Never trust client amounts: server re-validates against the QR or recipient details. App is presentation.

4. How to Code & Explain (Win the Interviewer)

Open with idempotency loud and clear: *"Every payment carries a client-generated idempotency key. If the network drops or the user hits Pay twice, the server returns the original result — never debits twice. This is non-negotiable in any payment system."*

Walk the success path: *"User scans QR → form filled → tap Pay → BiometricPrompt unlocks a Keystore key → app POSTs payment + signature + idempotencyKey → server checks signature, sends to bank, persists Transaction(PENDING) → bank responds → status flips to SUCCESS → mobile subscribes and updates UI."*

Failure paths: *"App crashes mid-flow → on relaunch, mobile finds open Transaction (PENDING), polls status. If server hadn't received it, app re-submits with the same idempotency key — bank gets one debit. If bank received it, server returns the right state."*

> ■ ***Tip:*** *Bring up* ***'mandatory final state'****: any payment must eventually become SUCCESS or FAILED. PENDING is temporary. Reconciler sweeps PENDING > N minutes and forces a check with the bank.*

5. Actual Code (Reference Snippets)

Idempotent payment with biometric signature, local PENDING tracking, reconcile job

Kotlin

```kotlin
enum class TxnStatus { PENDING, SUCCESS, FAILED }

@Entity(tableName = "transaction_local")
data class TransactionLocal(
    @PrimaryKey val idempotencyKey: String,
    val recipient: String, val amountPaise: Long,
    val status: TxnStatus, val createdAt: Long, val serverId: String?
)

class PaymentRepository(
    private val api: PaymentApi,
    private val db: AppDatabase,
    private val biometric: BiometricController
) {
    suspend fun pay(activity: FragmentActivity,
                    recipientUpi: String, amountPaise: Long): TxnStatus {
        val key = java.util.UUID.randomUUID().toString()
        db.txnDao().insert(TransactionLocal(
            key, recipientUpi, amountPaise, TxnStatus.PENDING,
```

```
            System.currentTimeMillis(), null
        ))
        val signature = biometric.authorise(activity,
            promptText = "Pay ■${amountPaise / 100} to $recipientUpi")
            ?: throw CancellationException("User cancelled")

        return submitWithRetry(key, recipientUpi, amountPaise, signature)
    }

    suspend fun submitWithRetry(key: String, recipient: String,
                                amount: Long, signature: ByteArray): TxnStatus {
        repeat(3) { attempt ->
            try {
                val resp = api.transfer(
                    PayRequest(idempotencyKey = key,
                               recipient = recipient,
                               amountPaise = amount,
                               signature = signature)
                )
                db.txnDao().updateStatus(key, resp.status, resp.id)
                return resp.status
            } catch (e: IOException) {
                if (attempt == 2) {
                    // could not confirm — STILL pending, app must reconcile
                    return TxnStatus.PENDING
                }
                kotlinx.coroutines.delay(1000L * (1 shl attempt))
            }
        }
        return TxnStatus.PENDING
    }

    suspend fun reconcilePending() {
        for (txn in db.txnDao().pending()) {
            try {
                val s = api.status(txn.idempotencyKey)
                db.txnDao().updateStatus(txn.idempotencyKey, s.status, s.id)
            } catch (_: Exception) { /* try again later */ }
        }
    }
}

class BiometricController(private val keystore: KeyStoreHelper) {
    suspend fun authorise(activity: FragmentActivity,
                          promptText: String): ByteArray? = suspendCoroutine { cont ->
        val signature = keystore.signatureForPayment()
        val prompt = BiometricPrompt(activity,
            ContextCompat.getMainExecutor(activity.applicationContext),
            object : BiometricPrompt.AuthenticationCallback() {
                override fun onAuthenticationSucceeded(
                    result: BiometricPrompt.AuthenticationResult) {
                    val sigBytes = result.cryptoObject!!.signature!!
                        .also { it.update(promptText.toByteArray()) }
                        .sign()
                    cont.resume(sigBytes)
                }
                override fun onAuthenticationError(c: Int, msg: CharSequence) =
                    cont.resume(null)
            })
        prompt.authenticate(
            BiometricPrompt.PromptInfo.Builder()
                .setTitle("Authorise payment").setSubtitle(promptText)
                .setNegativeButtonText("Cancel").build(),
            BiometricPrompt.CryptoObject(signature)
        )
    }
```

```
}
```

6. Cross-Questions & How to Answer Them

Q: Why a client-generated idempotency key, not a server-generated one?

Because the client may retry before the server has acknowledged the first request. A server-generated key would be different on the retry — creating a duplicate. Client-generated, sent with every retry, is the only safe pattern.

Q: How does biometric prevent attacks if the device is stolen?

BiometricPrompt with CryptoObject ensures the operation only succeeds if a biometric matches a private key bound to Android Keystore. The key is set as 'setUserAuthenticationRequired(true)'. Steal the device, you still need the fingerprint.

Q: What happens if the server confirms the bank debit but the response is lost?

Client retries with same idempotencyKey. Server returns the original SUCCESS (cached in Redis or DB by key). Client updates local state.

Q: How do you handle two devices of the same user paying at once?

Server's bank-side balance check is the truth. Both transactions are accepted as long as funds are sufficient; if not, one fails with INSUFFICIENT_FUNDS. App must always show 'Available balance' as a hint, not as a guarantee.

Q: What about UPI PIN entry — that's separate from biometric.

On UPI, the PIN is handled by NPCI's SDK in a sandboxed UI inside the app. Your app never sees the PIN. Biometric is an additional convenience for app-level auth; the UPI PIN is the bank's authorisation.

7. What Better? Alternative Approaches

Hardware-backed Keystore (StrongBox on Pixel 3+) — keys live in the secure element, even root can't extract them.

Risk engine on server — anomalies (sudden large amount, new device, foreign IP) trigger step-up auth.

End-to-end encrypted message for transaction details — even server admins can't read amount + recipient.

Wallet abstraction — UPI / card / bank-account-direct all behind a single 'payment source' interface.

Compliance: PCI-DSS means never log card PAN, mask in UI, tokenize before storing.

Key takeaways

- Client-generated idempotency key on every request.
- Biometric via BiometricPrompt + CryptoObject + Keystore.
- PENDING is a real state with a reconciler.
- Never trust client amounts; server re-validates.

Chapter 24: Design a Slack/Teams Chat (channels + threads)

Difficulty: Medium | **Tags: Real-time Channels Threads Search**

Multi-channel team chat with threads, mentions, search, and presence — the 1-1 chat extended to a group context.

1. What's the Problem?

Build a Slack-like app: workspaces have channels, channels have messages, messages can have threaded replies, users mentioned via @ get notified, presence (online/away) shown next to names, full-text search across the workspace, and all of this in real time across thousands of concurrent users in a single workspace.

Scale to assume

- Workspaces from 10 to 100,000 users.
- Channel sizes from 2 to 10,000+ members.
- A power channel may receive 5+ messages per second.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Channels — public, private, DM, multi-party DM?
- Threads — reply count, last reply timestamp?
- Mentions — user, channel (@channel)?
- Reactions (emoji)?
- Search — across messages, files, members?
- Notifications — per-channel preferences?

Non-functional clarifications

- Sub-second message delivery within a workspace.
- Search latency sub-300ms.
- Member list of 10K loads quickly.
- Offline: read recent channels, queue sends.

Why ask these? (the signal)

Asking about channels-with-10K-members vs DM-with-2 shows you understand fan-out is the scaling challenge.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Mobile: WorkspaceShell → ChannelList sidebar + currently-open ChannelScreen → Thread overlay. WebSocket carries messages, presence, typing events. **Server**: ChannelService stores messages; fan-out to all online channel members via their per-user WebSocket. Presence service tracks heartbeats. SearchService runs full-text index (Elasticsearch).

Major components

- ChannelRepository — channels list, joined channels, members per channel.
- MessageRepository — messages per channel; thread replies; reactions.
- PresenceService — online status, last seen.

- SearchRepository — query messages + files + users.
- MentionParser — @ matching with autocomplete.
- NotificationRouter — applies per-channel mute and mention overrides.

Low-Level Design (LLD)

Channel subscription model: app subscribes to currently-open channel + ambient (unread badge updates) for joined channels. Server pushes only relevant events to each user.

Threads: parent message has replyCount and lastReplyAt fields, updated by replies. Reply messages have a parentId. Thread view fetches by parentId.

Mentions: parsed client-side at render; click resolves user. Server stores mentions as a separate index for fast '@me' query.

Search: server-side Elasticsearch with workspace as a routing key. Client renders results with snippets and deep-links into the conversation.

Presence: WebSocket heartbeat every 30s; if no heartbeat for 60s, mark away. Online events are a high-volume topic — many apps push presence in aggregated batches not per-event.

4. How to Code & Explain (Win the Interviewer)

Open: *"The same WebSocket+Room pattern as 1-1 chat, scaled by channels. The mobile side cares about which channel is open so we don't render every message in every channel."*

Show one message: *"User types in #general → POST /channels/{id}/messages with client UUID → server persists, fan-outs to subscribers' sockets. Subscribers' apps insert into Room → channel screen re-emits."*

Thread: *"Thread is just a parentId. Opening the thread fetches /messages?parent=X. The parent's badge updates via an event 'thread.reply'."*

Presence: *"Per-channel presence is too expensive on big channels. We compute it lazily — the channel header shows 'X members online' from a precomputed presence count, not by querying each member."*

> ■ ***Tip:*** *Mention* ***'channel switch perceived latency'*** *— every channel switch must show messages instantly from Room cache; new messages fetched in background, never blocking the UI.*

5. Actual Code (Reference Snippets)

ChannelRepository with per-channel subscriptions and thread reply counter

Kotlin

```kotlin
@Entity(tableName = "channel")
data class Channel(@PrimaryKey val id: String, val workspaceId: String,
                   val name: String, val isPrivate: Boolean,
                   val lastReadId: String?, val unreadCount: Int = 0)

@Entity(tableName = "message",
        indices = [Index("channelId", "createdAt"), Index("parentId")])
data class Message(
    @PrimaryKey val id: String,
    val channelId: String, val parentId: String?,
    val senderId: String, val text: String,
    val createdAt: Long, val status: MessageStatus,
    val replyCount: Int = 0, val lastReplyAt: Long? = null,
    val reactionsJson: String? = null
)

class ChannelViewModel(private val repo: ChannelRepository) : ViewModel() {
    private val openChannelId = MutableStateFlow<String?>(null)

    val messages: Flow<PagingData<Message>> = openChannelId
```

```
        .filterNotNull()
        .flatMapLatest { id -> repo.pagingFor(id) }
        .cachedIn(viewModelScope)

    fun open(channelId: String) {
        openChannelId.value = channelId
        viewModelScope.launch { repo.subscribe(channelId) }
    }

    fun send(text: String) = viewModelScope.launch {
        repo.send(openChannelId.value!!, text)
    }
}

class ChannelRepository(
    private val ws: WebSocketClient,
    private val db: AppDatabase,
    private val api: ChatApi,
    private val me: String
) {
    private val activeSubs = mutableSetOf<String>()

    suspend fun subscribe(channelId: String) {
        if (channelId in activeSubs) return
        activeSubs += channelId
        ws.send(SubscribeChannel(channelId,
                                 sinceId = db.messageDao().lastSeenId(channelId)))
    }

    suspend fun send(channelId: String, text: String) {
        val msg = Message(java.util.UUID.randomUUID().toString(),
                          channelId, null, me, text,
                          System.currentTimeMillis(), MessageStatus.PENDING)
        db.messageDao().insert(msg)
        ws.send(SendChannelMsg(msg.id, channelId, text))
    }

    init {
        ws.events.onEach { ev ->
            when (ev) {
                is ServerEvent.NewMessage -> {
                    db.messageDao().insertIgnore(ev.toLocal())
                    if (ev.parentId != null)
                        db.messageDao().incrementReplyCount(ev.parentId)
                }
                is ServerEvent.MentionedYou ->
                    NotificationsHelper.show(ev.channelId, ev.preview)
                is ServerEvent.Presence ->
                    db.presenceDao().update(ev.userId, ev.online)
            }
        }.launchIn(GlobalScope)
    }
}
```

6. Cross-Questions & How to Answer Them

Q: How would you handle a channel with 10,000 members all online?

Don't broadcast everyone's presence to everyone. Group members into 'pages'; the client sees presence only for currently-visible members. For sending, the server still has to fan out to 10K sockets — usually done via a pub/sub like Redis Pub/Sub or NATS, where one publish broadcasts to all subscribers.

Q: What about offline members joining a big channel — should they get every message they missed?

No. We cap missed-message replay at, say, the last 200 per channel. Beyond that, the user can scroll up which fetches older messages from the server. Keeps reconnect bounded.

Q: How is full-text search done?

Server-side Elasticsearch index keyed by workspaceId. Mobile sends query → server returns ranked hits with snippets and messageIds → client fetches message context for the chosen result.

Q: Why not store messages in SQLite FTS for offline search?

Could, but: (a) index size on a large workspace is big; (b) search expectations are 'across everything' which includes messages the user has never opened. Server-side wins.

Q: How do reactions update without sending the full message?

Reaction added → server emits a small 'reaction.added' event with messageId + emoji + userId. Client updates that one message's reactions field. RecyclerView partial-bind (payload pattern) avoids full re-bind.

7. What Better? Alternative Approaches

Encrypted DMs — see Chapter 26 for the Signal protocol pattern.

Bots / webhooks — a webhook URL produces 'app messages' authored by a bot user.

Voice / video huddles — WebRTC stream attached to a channel; not the focus of this chapter.

Cross-workspace search — federated query across multiple ES indexes.

Compose-aware UI: Slack's text input has rich markdown, mentions, slash-commands — all best implemented as a Compose TextField with custom annotations + autocomplete chips.

Key takeaways

- Channels scale by selective subscription, not global broadcast.
- Threads = parentId + reply-count denormalised on parent.
- Presence aggregated, not per-event.
- Search is server-side ES.

Part 3

Hard Mobile Systems

Chapters 25 — 35

Chapter 25: Design the Full Instagram Feed (mobile + ranking + media)

Difficulty: Hard | **Tags: Feed Ranking Media Scale**

Ranked feed of friends' posts (image, carousel, reel). Backend ranks server-side, mobile renders mixed-media smoothly.

1. What's the Problem?

Design the home feed of Instagram on Android. Posts from people the user follows are shown in a personalised order (not just chronological). Each post may be a single image, a carousel of 2–10 images, or a 60-second video (Reels). Likes, comments, saves are inline. Pull-to-refresh, infinite scroll. Cold open under 1.5 seconds with cached content. Each post records impressions and dwell time.

Scale to assume

- 500M+ DAU, billions of posts.
- Active user views 100+ posts per session.
- Image + video are large; CDN + adaptive bitrate are mandatory.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Ranked feed (ML on server) or chronological — both as tabs?
- Post types — image, carousel, video, ad — heterogeneous list?
- Inline actions — like, comment, save, share, follow author?
- Stories at the top? (covered in Chapter 22)
- Suggested posts inserted between followed accounts?
- Pull-to-refresh — does it re-rank or just fetch newer?

Non-functional clarifications

- Cold open under 1.5s with cached posts visible.
- 60 fps scroll on mid-range with mixed images and videos.
- Bounded memory — feed of 1000+ items must not OOM.
- Network savings — only one video plays at a time.

Why ask these? (the signal)

Asking 'is the feed ranked or chronological' clarifies you understand ranking happens server-side and the mobile is a renderer. Most candidates conflate the two.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Server: FeedService asks FollowGraph for whom user follows, asks PostStore for recent candidates, asks RankingService to score, returns a paged list. **Mobile**: PagingData<FeedItem> with sealed-class items (Photo, Carousel, Video, Ad). Room is single source of truth; RemoteMediator pulls pages. ExoPlayer pool for videos. ImageLoader prefetches next-page first photos.

Major components

- FeedRepository — pulls /feed?cursor=…, writes to Room.
- RankingService (server) — out-of-scope client-side, but mention.

- Sealed FeedItem hierarchy — Photo, Carousel, Video, Ad, Suggested.
- Multi-view-type PagingDataAdapter.
- ExoPlayerPool — 3 instances, assigned to centered visible item.
- ImpressionTracker — emits view-events when 50% visible for 1s+.
- ActionRepository — optimistic like/save/comment.

Low-Level Design (LLD)

Server cursor is opaque (encodes user position + rank scores + experiment bucket). Don't expose it to client logic.

Single video at a time: a CenteredItemDetector returns the most-visible item per scroll frame; only that one plays. Use a debounce (e.g. 200ms after scroll settles) before starting a video — avoid playing each item as the user scrolls past.

Carousel: nested ViewPager2 inside the feed item; each carousel page is an image. Prefetch only the first 2 pages of each carousel; the rest load on swipe.

Like is optimistic with rollback — write Room first, POST /like, on failure roll back + toast.

Impressions: only count 'shown' if 50% in viewport for 1s; emit to analytics. Critical for ad billing — must be defensible.

4. How to Code & Explain (Win the Interviewer)

Open: *"The mobile side is Paging 3 + Room + sealed FeedItem types + ExoPlayer pool. The interesting infrastructure on the server is the ranker — but on the client, the magic is mostly performance: stable diffing, prefetch, partial-bind."*

Walk a scroll: *"User scrolls. PagingDataAdapter peeks ahead; at 80% of the page it triggers RemoteMediator.APPEND. RemoteMediator hits /feed?cursor=X, gets the next 20 posts, inserts into Room, emits. CenteredItemDetector finds the most visible item; if it's a Video, ExoPlayerPool reassigns a player and starts playback after a 200ms settle."*

Like flow: *"Heart tap → Room.update(liked=true, likeCount++) → emit; UI shows immediately. POST /like in WorkManager. If 409 (already liked) we silently accept; if 5xx, retry; if persistent failure, roll back and toast."*

> ■ ***Tip:*** *Bring up* ***'feed rank scrambling'*** *— server adds slight randomisation so the same user opening twice doesn't see identical order. Prevents the feeling of 'I already scrolled this'.*

5. Actual Code (Reference Snippets)

CenteredItemDetector + VideoBinder = single-video autoplay

Kotlin

```kotlin
sealed class FeedItem {
    abstract val id: String
    data class Photo(override val id: String, val author: Author,
                     val imageUrl: String, val caption: String?,
                     val likes: Long, val liked: Boolean) : FeedItem()
    data class Carousel(override val id: String, val author: Author,
                        val imageUrls: List<String>, val caption: String?,
                        val likes: Long, val liked: Boolean) : FeedItem()
    data class Video(override val id: String, val author: Author,
                     val hlsUrl: String, val thumbUrl: String,
                     val caption: String?, val likes: Long, val liked: Boolean) : FeedItem()
    data class Ad(override val id: String, val sponsor: String,
                  val imageUrl: String, val cta: String, val deeplink: String) : FeedItem()
}

// Centered-item detector for autoplay
```

```
class CenteredItemDetector(private val rv: RecyclerView) {
    private val _centered = MutableStateFlow<Int?>(null)
    val centered: StateFlow<Int?> = _centered

    init {
        rv.addOnScrollListener(object : RecyclerView.OnScrollListener() {
            private val handler = Handler(Looper.getMainLooper())
            private val emit = Runnable {
                _centered.value = computeCentered()
            }
            override fun onScrolled(rv: RecyclerView, dx: Int, dy: Int) {
                handler.removeCallbacks(emit)
                handler.postDelayed(emit, 200)   // settle window
            }
        })
    }
    private fun computeCentered(): Int? {
        val lm = rv.layoutManager as LinearLayoutManager
        val first = lm.findFirstVisibleItemPosition()
        val last = lm.findLastVisibleItemPosition()
        val rvCenter = (rv.top + rv.bottom) / 2
        var best: Int? = null; var bestDist = Int.MAX_VALUE
        for (i in first..last) {
            val v = lm.findViewByPosition(i) ?: continue
            val center = (v.top + v.bottom) / 2
            val d = kotlin.math.abs(center - rvCenter)
            if (d < bestDist) { bestDist = d; best = i }
        }
        return best
    }
}

// Bind videos to the centered item
class VideoBinder(private val pool: ExoPlayerPool, private val rv: RecyclerView,
                  private val adapter: FeedAdapter) {
    init {
        val detector = CenteredItemDetector(rv)
        GlobalScope.launch {
            detector.centered.collect { idx ->
                if (idx == null) return@collect
                val item = adapter.peek(idx)
                pool.bindTo(rv.findViewHolderForAdapterPosition(idx), item)
            }
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why does ranking happen server-side?

Three reasons: (1) the ranking model is large and needs candidate data the client doesn't have; (2) experiment-bucket assignment is server-managed; (3) client-side ranking means every app version must include the model — impossible to iterate quickly.

Q: How do you keep the feed responsive when the ranker is slow?

Two-stage: fast 'recent-from-follow-graph' returned immediately; ranked re-order pushed via a follow-up call. App shows the fast feed first, then upgrades if the ranked one arrives within 200ms — else keeps the fast one for this session.

Q: How do impressions defend against fraud (fake views to game ads)?

Server validates: cluster of 1000 impressions from one user in 5 seconds is a bot. Use viewport % + dwell time + click-through ratio as signals. Mobile sends honest events; server filters.

Q: How is 'pull to refresh' different from new posts arriving in real time?

Pull-to-refresh triggers a fresh ranked feed fetch — may re-order top items. Real-time arrivals (push) only invalidate the badge ('new posts'). We never auto-insert above the visible window without the user's tap, to avoid context loss.

Q: Memory: 1000 items in PagingData?

PagingConfig.maxSize bounds in-memory items; old pages evicted; Room keeps the data. Image library byte-bounded; ExoPlayer pool capped at 3 instances. The combination stays under low-RAM limits.

7. What Better? Alternative Approaches

Pre-warming sessions: on app cold open, render cached feed instantly while a /feed?fresh=1 call runs; merge.

Per-item DiffUtil payloads — only re-bind the like-count, not the whole card.

Adaptive video quality — HLS or DASH, low res on cellular, high on WiFi.

Pre-bind suggested posts in known slots so they don't shift content.

On-device personalisation for the 'cold start' user — first session ranks by simple heuristics (most popular among followed) until server signals arrive.

Key takeaways

- Mobile is a renderer; ranking is server-side.
- Sealed types + single-video-at-a-time = smooth mixed media.
- Impressions defended by visibility-percent + dwell time.
- Feed = Paging 3 + Room + RemoteMediator (same as Chapter 12).

Chapter 26: Design Full WhatsApp (Groups, E2E Encryption, Multi-Device)

Difficulty: Hard | **Tags: E2E Signal protocol Groups Multi-device**

The hard parts of WhatsApp: end-to-end encryption with the Signal protocol, groups, and same-account on multiple devices.

1. What's the Problem?

Extend the 1-to-1 chat from Chapter 17 to support: (a) end-to-end encryption — server never sees plaintext; (b) groups of up to 1024 members; (c) the same user logged in on phone + tablet + web, with all devices receiving every message. Plus reliability under all the prior constraints (offline, receipts, reconnect).

Scale to assume

- 2B+ users globally.
- Groups up to 1024 members; broadcast lists larger.
- Multi-device: typically 1 phone + up to 4 companion devices per user.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- 1-1 + group messaging.
- End-to-end encryption — server stores ciphertext.
- Forward secrecy — past messages safe even if a key is compromised.
- Multi-device — every message reaches all of user's devices.
- Group join / leave key rotation.
- Media — encrypted before upload?

Non-functional clarifications

- Server cannot read message contents.
- Key compromise on one device doesn't reveal history.
- Adding a new device is a one-time setup and a key fan-out.
- Group of 1024 — fan-out scalable; not $O(N^2)$ on the wire.

Why ask these? (the signal)

Mentioning 'Signal protocol' by name and being able to sketch X3DH + Double Ratchet is the difference between mid-level and senior here.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Each **device** has its own keypair. Sender encrypts the message **once per recipient device** using a session key derived from the Signal protocol (X3DH for first contact, Double Ratchet for subsequent messages). Server stores opaque ciphertexts in per-device inboxes and routes them. For groups, the sender encrypts once with a **sender key**, then encrypts that sender key individually for each member's device — fan-out happens client-side via per-device session encryption.

Major components

- DeviceKey — identity key + signed prekey + one-time prekeys.
- KeyDirectoryService (server) — stores everyone's public prekey bundles.

- Session (per pair of devices) — X3DH-derived, advanced via Double Ratchet on each message.
- Sender Key (per group) — symmetric ratchet; rotated on membership change.
- Per-device inbox — server stores encrypted blobs.
- Multi-device sync — every send fans out to all of user's own devices too.

Low-Level Design (LLD)

X3DH (initial handshake): Alice grabs Bob's prekey bundle from the server, runs an Elliptic-curve Diffie-Hellman involving 4 keys (identity, signed prekey, one-time prekey, ephemeral). Both sides derive the same root key.

Double Ratchet: every message advances two ratchets — Diffie-Hellman (when a new key is exchanged) and a chain key. This gives forward secrecy (old keys can't decrypt new messages) and post-compromise security (after one compromise, future messages become safe again).

Per-device fan-out: Alice sends to Bob → app looks up Bob's devices → encrypts per device → sends N ciphertexts. Server treats each as a separate routing decision.

Multi-device: when Alice has phone + tablet, sending from phone also encrypts the message to her own tablet's session. Tablet receives a copy. Both sides agree by syncing keypair pairs.

Groups: sender key (a symmetric key) encrypts the message; sender key itself is delivered via the per-pair Double Ratchet to each member. Each member uses the sender key directly to decrypt subsequent messages — cheaper than per-message asymmetric ops.

4. How to Code & Explain (Win the Interviewer)

Open: *"The crux is: every device has its own identity key and the server never sees plaintext. Encryption is done before send, decryption after receive. The server is a routing layer."*

X3DH plainly: *"First time Alice talks to Bob, her app downloads Bob's public prekey bundle, does an ECDH dance with four keys to derive a shared secret, and sends the first message under that secret. Bob's app derives the same secret independently."*

Double Ratchet plainly: *"Every subsequent message advances a one-way hash chain — old keys are unrecoverable. When either side sends, they include a new DH public; the next received message resets the ratchet with new entropy."*

Multi-device: *"Same content, encrypted N times — once per recipient device, once per my own other devices. Server fan-outs."*

Groups: *"For efficiency, we use a sender-key — a symmetric key derived per-sender-per-group. Delivered to each member via their pairwise session. On membership change, rotate the sender key and re-deliver."*

> ■ ***Tip:*** *Reference* ***libsignal-protocol-java*** *(the open-source library WhatsApp uses) — naming it shows you've read past the marketing.*

5. Actual Code (Reference Snippets)

High-level encryption engine + per-recipient fan-out + group sender-key path

Kotlin

```kotlin
// Conceptual model — real impl uses libsignal-protocol-java
class IdentityKeyPair(val publicKey: ECPublicKey, val privateKey: ECPrivateKey)
class PreKeyBundle(val identityKey: ECPublicKey,
                   val signedPreKey: ECPublicKey, val signedPreKeySig: ByteArray,
                   val oneTimePreKey: ECPublicKey?,
                   val registrationId: Int, val deviceId: Int)

class SignalSessionStore {
    private val sessions = mutableMapOf<DeviceAddress, SessionRecord>()
    fun load(addr: DeviceAddress): SessionRecord? = sessions[addr]
```

```
    fun store(addr: DeviceAddress, s: SessionRecord) { sessions[addr] = s }
}
data class DeviceAddress(val userId: String, val deviceId: Int)

class EncryptionEngine(
    private val sessions: SignalSessionStore,
    private val identity: IdentityKeyPair,
    private val keyDir: KeyDirectoryService
) {
    suspend fun encryptFor(recipient: DeviceAddress, plaintext: ByteArray): CipherText {
        val s = sessions.load(recipient) ?: run {
            val bundle = keyDir.fetchBundle(recipient)
            val s = X3DH.handshake(identity, bundle)   // first contact
            sessions.store(recipient, s); s
        }
        val (cipher, advanced) = DoubleRatchet.encrypt(s, plaintext)
        sessions.store(recipient, advanced)
        return cipher
    }

    suspend fun decrypt(sender: DeviceAddress, c: CipherText): ByteArray {
        val s = sessions.load(sender)
            ?: throw IllegalStateException("no session — possible new device")
        val (plain, advanced) = DoubleRatchet.decrypt(s, c)
        sessions.store(sender, advanced)
        return plain
    }
}

class ChatRepositoryE2E(
    private val net: Network, private val engine: EncryptionEngine,
    private val recipients: RecipientResolver, private val me: User
) {
    suspend fun sendTo(userId: String, plaintext: String) {
        val devices = recipients.devicesFor(userId) +
                      recipients.devicesFor(me.id).filter { it.deviceId != me.deviceId }
        val bytes = plaintext.toByteArray()
        for (d in devices) {
            val ct = engine.encryptFor(d, bytes)
            net.send(d, ct)
        }
    }

    suspend fun sendToGroup(groupId: String, plaintext: String) {
        val senderKey = SenderKeyStore.getOrRotate(groupId, me.deviceId)
        val ct = SenderKeyCipher.encrypt(senderKey, plaintext.toByteArray())
        val msg = GroupMessage(groupId, ct, senderKey.version)
        // each member must already have senderKey delivered via pairwise session
        for (m in recipients.members(groupId)) {
            net.send(m, msg)
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why fan out per device on the client?

Because each device has its own session keys. The server can't read plaintext to re-encrypt. Per-device ciphertext is the only model that preserves E2E.

Q: What happens when a user adds a new device?

New device generates its own key pair, registers with the directory. Server notifies other contacts of the new device. Contacts add this device to future sends. There's a 'verify security number' UX so users can confirm they're not under MITM.

Q: Group of 1024 — isn't fan-out expensive?

Each send is ~1KB ciphertext × 1024 devices = ~1MB per message. Heavy but doable. WhatsApp throttles huge groups; the sender-key trick avoids per-message asymmetric crypto — only when sender-key itself rotates is there a fan-out of key material.

Q: How is forward secrecy achieved?

Double Ratchet: every send/receive advances a one-way hash chain on the chain key. Even if today's session key leaks, yesterday's messages cannot be decrypted because yesterday's keys are unrecoverable.

Q: What about backups — Google Drive backup leaks plaintext?

WhatsApp's backup is encrypted with a user-chosen passphrase before being uploaded. Without the passphrase, even Google Drive can't read it. (Old non-encrypted backup option still exists; users choose.)

7. What Better? Alternative Approaches

Sealed sender: server doesn't even learn who's talking to whom — just metadata-minimal routing.

MLS (Messaging Layer Security) — newer protocol for group chat; scales better than Signal sender-keys.

Device key transparency — cryptographic proof that a user's key wasn't silently swapped.

Disappearing messages — TTL applied on send; receiver deletes on expiry; server expires its copy too.

Voice / video — separate WebRTC stack with DTLS-SRTP; same identity keys used to authenticate the call setup.

Key takeaways

- X3DH = initial handshake; Double Ratchet = ongoing forward-secret messaging.
- Per-device encryption + server-side fan-out = E2E + multi-device.
- Groups use a sender-key to avoid per-message asymmetric ops.
- Server is metadata-aware but blind to message content.

Chapter 27: Design the Uber Driver App (Location-based services)

Difficulty: Hard | **Tags: Location Geohash Background Battery**

Live location streaming with low battery cost, ride matching, offline acceptance, navigation.

1. What's the Problem?

Build the driver app: ping the driver's location to the backend continuously, receive ride requests with rider details, accept/reject within seconds, navigate to pickup, then dropoff, complete the trip. Battery and data must be minimal. Brief network drops on highways are expected.

Scale to assume

- Active drivers concurrently: millions globally.
- Location pings: every 4 seconds while online; 1Hz on a trip.
- Battery target: full 10-hour shift without going below 50%.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Online/offline toggle to start receiving requests?
- Receive ride request with timer to accept?
- Real-time navigation with rerouting?
- Show heatmap of demand?
- Earnings + trip history?
- Offline mid-trip — continue navigation locally?

Non-functional clarifications

- Location accuracy ±10m; refresh under 5s when online, 1Hz on trip.
- Battery: under 5% per hour while online.
- Network: gracefully handle 30s outages.
- Ride request notification within 1s of dispatch.

Why ask these? (the signal)

Asking about Fused Location Provider + GPS vs network shows you've thought about real Android location.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Mobile: Foreground service for location, FusedLocationProvider with adaptive interval, WebSocket to dispatch backend. **Server**: receives location pings, indexes drivers in a geospatial structure (S2 cells / geohash), matches incoming ride requests to nearby drivers. Push ride offer via WebSocket; driver accepts/declines; matched trip enters in-progress state.

Major components

- Foreground LocationService — keeps the process alive while on shift.
- FusedLocationProviderClient — adaptive priority HIGH_ACCURACY on trip, BALANCED idle.
- LocationUploader — coalesces pings; sends batches via WebSocket.
- DispatchChannel — WebSocket; receives offers, sends accepts.

- Server DispatchService — geo-indexed driver lookup, scoring, offering.
- S2 / Geohash index (Redis Geo) — find drivers within radius R.

Low-Level Design (LLD)

Location frequency adapts: 4s when idle online, 1s on trip, 30s when stationary > 1 min. Uber-style power saving.

Foreground service on Android is mandatory for continuous location; sticky notification 'You are online'.

S2 cells (Uber-specific): server indexes drivers by S2 cell ID at level 12 (~1.27 km²). Dispatch query: 'drivers in rider's S2 cell + 8 neighbours sorted by ETA'.

Offer state machine: OFFERED → ACCEPTED / TIMED_OUT. App shows a 10-second countdown; if not accepted, system re-offers to next driver.

Offline tolerance: WebSocket reconnects with exponential backoff; location batches buffered locally; on reconnect, flush. If still on a trip, navigation continues from on-device map tiles.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is real-time location streaming with battery-aware adaptation. Foreground service so Android doesn't kill it; WebSocket so the server can push offers immediately."*

Battery story: *"Naive 1Hz GPS drains a battery in 3 hours. We use FusedLocationProvider with PRIORITY_BALANCED when idle and PRIORITY_HIGH_ACCURACY only on a trip. We also throttle pings server-side — 4s when not actively dispatching, 1s only when near pickup/dropoff."*

Dispatch flow: *"Rider requests ride → backend queries drivers in their S2 cell + neighbours → ranks by ETA + acceptance rate → pushes offer to top driver via WebSocket → driver has 10s to accept. If timeout, next driver."*

Reconnect: *"App buffers pings while offline. On reconnect, server gets a batch with timestamps so the trail isn't lost. Navigation uses pre-downloaded map tiles so the screen never goes blank."*

> ■ ***Tip:*** *Reference* ***Uber's H3 hex grid*** *as an alternative to S2 cells — even more uniform cell sizes. Naming H3 = senior signal.*

5. Actual Code (Reference Snippets)

Foreground service + adaptive FusedLocation + bounded buffer for offline pings

Kotlin

```kotlin
class LocationForegroundService : Service() {
    private lateinit var fused: FusedLocationProviderClient
    private val ws by lazy { DispatchWebSocket(driverId = TokenStore.driverId) }

    override fun onStartCommand(i: Intent?, f: Int, sid: Int): Int {
        startForeground(1, notification())
        fused = LocationServices.getFusedLocationProviderClient(this)
        setLocationMode(LocationMode.IDLE)   // when going online
        return START_STICKY
    }

    private fun setLocationMode(mode: LocationMode) {
        fused.removeLocationUpdates(callback)
        val req = LocationRequest.Builder(
            when (mode) {
                LocationMode.IDLE   -> Priority.PRIORITY_BALANCED_POWER_ACCURACY
                LocationMode.ON_TRIP-> Priority.PRIORITY_HIGH_ACCURACY
            }, /* interval */ if (mode == LocationMode.ON_TRIP) 1_000 else 4_000
        ).build()
        fused.requestLocationUpdates(req, callback, Looper.getMainLooper())
    }
```

```
    private val callback = object : LocationCallback() {
        override fun onLocationResult(r: LocationResult) {
            r.lastLocation?.let { loc ->
                ws.sendPing(Ping(loc.latitude, loc.longitude, loc.accuracy,
                                 System.currentTimeMillis()))
                LocationCache.put(loc)         // in-memory ring buffer
            }
        }
    }
}

class DispatchWebSocket(val driverId: String) {
    private val socket = ResilientWebSocket("/driver/${driverId}/dispatch")
    private val pending = ArrayDeque<Ping>()
    init {
        socket.events.onEach { ev ->
            when (ev) {
                is ServerEvent.RideOffer -> showOfferUI(ev)
                is ServerEvent.OfferTimeout -> dismissOfferUI()
                is ServerEvent.Connected -> drainPings()
            }
        }.launchIn(GlobalScope)
    }
    fun sendPing(p: Ping) {
        if (socket.isOpen) socket.send(p) else pending.addLast(p)
        if (pending.size > 500) pending.pollFirst()  // bounded buffer
    }
    private fun drainPings() {
        while (pending.isNotEmpty() && socket.isOpen) socket.send(pending.pollFirst())
    }
    fun acceptOffer(offerId: String) = socket.send(AcceptOffer(offerId))
}
```

6. Cross-Questions & How to Answer Them

Q: Why a foreground service for location?

Android 8+ aggressively kills background services. Foreground service with sticky notification keeps the process alive. It's also required by Play policy for ongoing location use.

Q: How does the server find nearby drivers efficiently?

Drivers indexed by S2 cell (or H3 hex). Lookup: rider's cell + immediate neighbours = O(1) bounded set. Avoids scanning all drivers.

Q: How do you avoid notification fatigue from offers the driver missed?

Server tracks acceptance rate and offer timeouts per driver; if a driver is unresponsive, deprioritise. Reduces wasted offers.

Q: How would you handle GPS jitter when stationary?

Server-side: if delta-distance < accuracy radius, treat as same point. Client-side: ignore consecutive pings within 5m, drop to lower frequency.

Q: What happens to the user's location during an Android Doze period?

Foreground service + WAKE_LOCK keep the location active. Without these, Android would batch location updates with 15min delay. This is exactly why a Foreground service is non-negotiable.

7. What Better? Alternative Approaches

Snap-to-road on client using Maps SDK — show the marker on the road, not in adjacent buildings.

Predictive dispatch: server pre-offers based on driver's predicted location 30s out.

Cell-tower fallback in tunnels / underground — last known + dead-reckoning.

Driver heatmap uses pre-aggregated demand cells from the last 5min; small payload.

Trip storage: server-side immutable trip log; client just shows current state.

Key takeaways

- Foreground service is mandatory for continuous location.
- Adapt frequency to state (idle vs on-trip).
- S2/H3 cells for spatial indexing.
- Bounded local buffer survives short outages.

Chapter 28: Design Google Photos (Upload, Sync, Gallery)

Difficulty: Hard | **Tags: Upload Background work Gallery Sync**

Continuous background upload of every new photo, deduplication, fast gallery, on-device search.

1. What's the Problem?

Build a Google Photos-like app. Every photo and video taken on the phone is uploaded to the cloud in the background, deduplicated, accessible in a gallery sorted by date, searchable by content (faces, locations, objects). Handles offline (queues), low storage, and works without ever disrupting the user.

Scale to assume

- User has 10,000–100,000 photos.
- Photos: 3–10 MB each. Videos: 50 MB – 5 GB.
- Upload rate must not saturate cellular; WiFi-only by default.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Upload only photos, or videos too?
- Smart albums (Faces, Places, Things)?
- Free up space: delete local copies once safely uploaded?
- Sharing — links, partner accounts?
- On-device search vs server search?
- Pause/resume uploads, choose WiFi-only?

Non-functional clarifications

- Uploads must survive app kills and device reboots.
- Battery-friendly: defer to charge + WiFi by default.
- Gallery loads first page (recent month) under 500ms even on cold start.
- Resumable uploads — a 2GB video shouldn't restart from 0 on connection drop.

Why ask these? (the signal)

Asking about resumable uploads is the senior signal — tcp can drop mid-2GB video.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

MediaStore content observer detects new media → enqueues UploadJob in WorkManager (charging + unmetered network constraints by default). Each upload is chunked (resumable). Server stores blob in S3-class storage; metadata in DB. Client gallery reads from a local index (Room) merged with server's view; on-device decoding via MediaStore for local + thumbnails from CDN for remote-only.

Major components

- ContentObserver on MediaStore — detects new photos/videos.
- UploadWorker (WorkManager) — chunked resumable upload.
- Local Index (Room) — every media item: uri, dateTaken, uploadStatus, serverId, hash.
- Server Upload API — initiate (returns sessionId) + uploadChunk + finalize.

- Server MetadataService — items, albums, search index.
- Gallery (Compose / RecyclerView grid) — virtualised, time-bucketed.
- On-device ML — face/object detection if 'offline search' supported.

Low-Level Design (LLD)

Resumable upload: client requests sessionId; uploads 5MB chunks with chunk index; on failure, queries 'how much did you receive?' and resumes from there. Pattern: tus.io / GCS resumable.

Dedup: client computes SHA-256 of the file; sends hash to server; server says 'already have' → only upload metadata, skip bytes. Massive savings on shared screenshots.

Thumbnail generation: server generates 3 sizes (200/600/2000px). Client picks size by grid density.

Gallery date-bucket layout: items grouped by day; sticky headers; grid count adaptive to screen width.

Free-up-space: only delete local file after server confirms persistence + thumbnail is cached locally. Reversible if user re-enters.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is a continuous background sync of media with resumable chunked uploads and a dedup-aware server. The gallery is a virtualised grid keyed on a local index that merges local + remote items."*

Upload pipeline: *"ContentObserver fires when a photo is added. We enqueue an UploadWorker with constraints. Worker hashes the file → POST /initiate with hash → if dedup hit, done. Else upload 5MB chunks; on each success, persist 'last chunk' so a crash resumes. On finalize, mark uploaded in Room."*

Gallery: *"User opens app. Room query returns the index of all known items, sorted by dateTaken DESC, in pages. The grid shows local URIs for local items (fastest decode) and CDN URLs for remote-only items. Thumbnails cached via the image library."*

> ■ ***Tip:*** *Mention* ***WorkManager with setExpedited*** *for user-triggered uploads (e.g., share immediately). Default background uploads use unmetered + charging constraints.*

5. Actual Code (Reference Snippets)

ContentObserver + resumable chunked UploadWorker with dedup

Kotlin

```kotlin
@Entity(tableName = "media_index",
        indices = [Index("dateTaken"), Index("uploadStatus")])
data class MediaIndex(
    @PrimaryKey val localUri: String,
    val sha256: String,
    val dateTaken: Long,
    val sizeBytes: Long,
    val mediaType: String,           // image / video
    val uploadStatus: UploadStatus,
    val serverId: String?,
    val remoteThumbUrl: String?
)
enum class UploadStatus { PENDING, IN_PROGRESS, UPLOADED, FAILED }

class MediaScannerObserver(
    private val ctx: Context, private val db: AppDatabase
) : ContentObserver(Handler(Looper.getMainLooper())) {
    override fun onChange(self: Boolean, uri: Uri?) {
        scope.launch { syncNew() }
    }
    private suspend fun syncNew() {
        ctx.contentResolver.query(
            MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
```

```
            arrayOf(MediaStore.Images.Media._ID,
                    MediaStore.Images.Media.DATE_TAKEN,
                    MediaStore.Images.Media.SIZE), null, null, null
        )?.use { cursor ->
            while (cursor.moveToNext()) {
                val id = cursor.getLong(0)
                val uri = ContentUris.withAppendedId(
                    MediaStore.Images.Media.EXTERNAL_CONTENT_URI, id).toString()
                if (db.mediaDao().exists(uri)) continue
                val date = cursor.getLong(1); val size = cursor.getLong(2)
                db.mediaDao().insert(MediaIndex(uri, "",  date, size, "image",
                    UploadStatus.PENDING, null, null))
                enqueueUpload(uri)
            }
        }
    }
    private fun enqueueUpload(uri: String) {
        val req = OneTimeWorkRequestBuilder<UploadWorker>()
            .setInputData(workDataOf("uri" to uri))
            .setConstraints(Constraints.Builder()
                .setRequiredNetworkType(NetworkType.UNMETERED)
                .setRequiresCharging(true).build())
            .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 1, TimeUnit.MINUTES)
            .build()
        WorkManager.getInstance(ctx).enqueueUniqueWork(
            "upload:$uri", ExistingWorkPolicy.KEEP, req)
    }
}

class UploadWorker(c: Context, p: WorkerParameters) : CoroutineWorker(c, p) {
    override suspend fun doWork(): Result {
        val uri = inputData.getString("uri") ?: return Result.failure()
        val db = AppModule.db
        val api = AppModule.api
        val item = db.mediaDao().get(uri) ?: return Result.failure()
        try {
            val hash = item.sha256.ifEmpty { computeSha256(uri) }
            val session = api.initiate(hash, item.sizeBytes, item.mediaType)
            if (session.dedupHit) {
                db.mediaDao().markUploaded(uri, session.serverId, session.thumbUrl)
                return Result.success()
            }
            val resume = session.resumeFromByte
            uploadChunks(uri, session.sessionId, fromByte = resume,
                         chunkSize = 5_000_000)
            val final = api.finalize(session.sessionId)
            db.mediaDao().markUploaded(uri, final.serverId, final.thumbUrl)
            return Result.success()
        } catch (e: IOException) {
            return Result.retry()
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why hash-then-upload?

Dedup. The same screenshot might be on 10 phones; we want to upload it once. Hash-first lets the server skip the upload entirely. Even within a single user's library, the same image saved twice doesn't double the bytes.

Q: How do you handle a chunked upload across a process kill?

Each chunk's success is persisted (in Room or the session API). On restart, WorkManager re-runs the worker; worker asks server 'how much do you have' and resumes from there. No bytes wasted.

Q: Cellular spike: WiFi disconnects mid-photo upload.

WorkManager's network constraint fails the worker → reschedule when constraint is met. The session id is preserved; on next attempt, resume.

Q: How do you do on-device face search without uploading every photo to the server?

ML Kit / TensorFlow Lite runs locally; embeddings stored in Room. Search query embeds the term and runs cosine similarity. Photos that are server-only have embeddings fetched on demand.

Q: How does the gallery render 100K items smoothly?

Grid is virtualised (LazyVerticalGrid). Each item is just a thumbnail URL from the index. Image library handles cache. Only ~20–30 items decoded at a time.

7. What Better? Alternative Approaches

Differential sync: server has 'since' watermark; client pulls deletes / changes since last sync.

BLAKE3 hashing on the client — much faster than SHA-256 for large videos.

Smart upload throttling: pause when battery < 20% or storage < 1GB free.

End-to-end encryption as an option — keys held by user only; server stores opaque blobs.

Server-driven gallery layouts: 'memories', 'this day in past years', 'top of trip' — server-driven sections within the gallery.

Key takeaways

- ContentObserver triggers, WorkManager executes — robust to kills.
- Resumable chunked uploads via session IDs.
- Hash-first = dedup + bandwidth savings.
- Local index (Room) is the gallery's source of truth.

Chapter 29: Design Netflix Mobile (Player, ABR, Offline Downloads)

Difficulty: Hard | Tags: Video ExoPlayer ABR DRM

ExoPlayer-based video player with adaptive bitrate, content protection (Widevine DRM), and offline downloads.

1. What's the Problem?

Build a Netflix-style video app: stream movies and shows on demand, adapt quality to network in real time, support pause/resume across devices, allow downloads for offline viewing, and protect content from being copied. The player must be smooth on cellular and recover from network drops without visible buffering for short outages.

Scale to assume

- 200M+ subscribers, peak millions of concurrent streams.
- Catalogue: tens of thousands of titles, each encoded at 5–10 bitrates.
- Mobile bitrate ranges from 200kbps to 4Mbps depending on network.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Streaming + offline download?
- Multi-audio, subtitles?
- Resume position synced across devices (Continue Watching)?
- Quality selection — auto vs manual?
- DRM — Widevine on Android?
- Parental control, mature content?

Non-functional clarifications

- Time-to-first-frame under 2s on 4G.
- Adaptive bitrate switching invisible to user.
- Rebuffer rate < 1% of session time.
- Battery: 90+ min playback on full charge.
- Content protection: cannot be screen-recorded.

Why ask these? (the signal)

Asking about DRM L1 vs L3 separates senior from mid — L1 means secure video path in hardware.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Mobile: ExoPlayer with DASH or HLS source; built-in ABR algorithm picks bitrate. Manifest fetched from CDN; video chunks fetched as needed. **DRM**: Widevine; license server returns content key bound to device. **Downloads**: ExoPlayer's DownloadManager schedules chunk download to local; same media source plays from disk offline. **Resume**: client sends progress every 30s; server records; other devices fetch on play.

Major components

- ExoPlayer + MediaSource (DashMediaSource / HlsMediaSource).
- TrackSelector — picks bitrate from manifest's representations.
- DefaultBandwidthMeter — observes throughput; informs ABR.

- Widevine DRM session — license fetched at play start.
- DownloadManager + DownloadService — for offline content.
- PlaybackProgressUploader — pings server every 30s; persists locally too.
- ContinueWatchingRepository — pull on app open.

Low-Level Design (LLD)

ExoPlayer media source: DASH manifest lists multiple representations (qualities). The TrackSelector chooses based on bandwidth + screen size + user preference.

ABR algorithm: throughput-based — pick highest representation whose bitrate < throughput × safety. Buffer-based — if buffer is low, drop quality. ExoPlayer's default does both.

Widevine: on play, app requests a license — sends DRM challenge to license server, server returns license blob, ExoPlayer's DRM session uses it. L1 = hardware-backed (cannot screen-record HD); L3 = software (only SD).

Download: DownloadManager queues a DownloadRequest with the manifest URL + quality. Constraints: WiFi by default. Downloads continue in foreground service.

Progress sync: 30s heartbeat; on pause, immediate flush. Server returns the highest watermark across devices on play.

4. How to Code & Explain (Win the Interviewer)

Open: *"The heavy lifting is ExoPlayer + a CDN. The mobile app is mostly orchestration: launching playback, handling lifecycle, syncing progress, and managing downloads."*

Show a play flow: *"User taps Play. We fetch the manifest from CDN, hand to ExoPlayer with DRM config. ExoPlayer requests a Widevine license — license server validates the user's subscription and device. License granted; ExoPlayer starts buffering the lowest representation, ramps up via ABR as bandwidth allows."*

Network drop: *"BandwidthMeter notices throughput drops; ABR steps down quality. If a chunk fails, ExoPlayer retries with exponential backoff. Buffer of 30s on the player smooths over outages up to half a minute."*

Downloads: *"User taps download. DownloadManager fetches manifest + chosen quality chunks to internal storage. Same MediaSource API plays them offline — playback code doesn't know the difference."*

> ■ ***Tip:*** *Mention* ***WidevineL1 vs L3****: an app distinguishing them shows it actually integrates DRM, not just talks about it.*

5. Actual Code (Reference Snippets)

ExoPlayer + Widevine + ABR-friendly buffer config + progress sync

Kotlin

```kotlin
class NetflixPlayer(val context: Context, val drmConfig: DrmConfig,
                    val licenseUrl: String) {

    private val player = ExoPlayer.Builder(context)
        .setLoadControl(DefaultLoadControl.Builder()
            .setBufferDurationsMs(30_000, 60_000, 1_500, 5_000)
            .build())
        .setBandwidthMeter(DefaultBandwidthMeter.Builder(context).build())
        .build()

    fun play(manifestUrl: String, startPositionMs: Long = 0) {
        val drmSession = DefaultDrmSessionManager.Builder()
            .setUuidAndExoMediaDrmProvider(C.WIDEVINE_UUID, FrameworkMediaDrm.DEFAULT_PROVIDER)
            .build(WidevineLicenseDataSourceFactory(licenseUrl))
        val source = when {
            manifestUrl.endsWith(".mpd") -> DashMediaSource.Factory(
                DefaultHttpDataSource.Factory())
```

```
                .setDrmSessionManagerProvider { drmSession }
                .createMediaSource(MediaItem.fromUri(manifestUrl))
            manifestUrl.endsWith(".m3u8") -> HlsMediaSource.Factory(
                DefaultHttpDataSource.Factory())
                .setDrmSessionManagerProvider { drmSession }
                .createMediaSource(MediaItem.fromUri(manifestUrl))
            else -> error("unknown manifest")
        }
        player.setMediaSource(source)
        player.prepare()
        player.seekTo(startPositionMs)
        player.play()
    }

    fun release() = player.release()
}

// Progress sync
class ProgressUploader(val player: ExoPlayer, val titleId: String,
                       val api: PlaybackApi, val db: AppDatabase) {
    private var lastSent = 0L
    fun startTicking() = GlobalScope.launch {
        while (player.isPlaying) {
            val pos = withContext(Dispatchers.Main) { player.currentPosition }
            if (pos - lastSent > 30_000) {
                db.progressDao().upsert(Progress(titleId, pos, System.currentTimeMillis()))
                runCatching { api.heartbeat(titleId, pos) }
                lastSent = pos
            }
            delay(5_000)
        }
    }
}

// Offline download — single line on ExoPlayer's DownloadService
fun startDownload(titleId: String, manifestUrl: String, quality: Int) {
    val request = DownloadRequest.Builder(titleId, Uri.parse(manifestUrl))
        .setStreamKeys(listOf(StreamKey(0, 0, quality)))
        .build()
    DownloadService.sendAddDownload(
        context, MyDownloadService::class.java, request,
        /* foreground */ false)
}
```

6. Cross-Questions & How to Answer Them

Q: Why Widevine and not custom encryption?

Widevine is the Android standard, integrated into Chromium, supports L1 (hardware), and content providers (studios) require certified DRM to license premium content. Rolling your own won't get a studio deal.

Q: How does ABR decide bitrate?

ExoPlayer's DefaultTrackSelector uses BandwidthMeter (a rolling average of throughput) + current buffer fullness. Pick highest rep where bitrate × safety_factor < throughput AND buffer can sustain. Implementation: ExoPlayer's `AdaptiveTrackSelection`.

Q: Why a 30s buffer vs 5s?

Tradeoff: bigger buffer absorbs network drops but costs memory and risks data over-fetch if the user closes. 30s is the industry sweet-spot for video. For live streams, 5s to minimise latency.

Q: How is offline content protected?

Downloaded chunks are still encrypted by Widevine. License is fetched at download time + has a TTL; renew when offline. On license expiry, content can't play.

Q: How do you measure rebuffer rate?

ExoPlayer fires events on stall/restart. Aggregate per session: total stall time / total playback time. Send to analytics. Engineering's north-star metric.

7. What Better? Alternative Approaches

Per-title encoding: ladder of bitrates optimised per title (cartoons need less than action films).

Open Connect (Netflix's CDN) — servers physically placed inside ISPs to slash latency.

Predictive prefetch: when on episode 1, prefetch the first 30s of episode 2 in the background.

Audio-only mode on low bandwidth — keep dialogue, drop video.

Subtitles: TTML / WebVTT files separate from video — switchable on the fly.

Key takeaways

- ExoPlayer + DASH/HLS = standard combo.
- Widevine L1 = hardware DRM, required for HD.
- ABR = bandwidth meter + buffer level.
- Downloads use the same MediaSource — offline is mostly free.

Chapter 30: Design Spotify Mobile (Streaming + Offline + Playlists)

Difficulty: Hard | **Tags: Audio Caching Offline Background playback**

Audio streaming with gapless playback, smart caching, offline downloads, background playback through screen lock.

1. What's the Problem?

Build Spotify on Android. Stream music with adaptive bitrate and gapless transitions, cache recently-played for instant replay, support full-album offline downloads, continue playback through screen lock and from Bluetooth headset controls. Synchronise playlists across devices.

Scale to assume

- 500M+ users; tens of millions of tracks; 10K+ playlist items per user.
- Track size: 3-6MB at 160kbps; downloaded library can be GBs.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Stream tracks; gapless transitions?
- Pre-fetch next track in queue?
- Offline downloads — per track / album / playlist?
- Lock-screen + Bluetooth media controls?
- Listening history + Recently Played?
- Sync playlists across devices?
- Lyrics — fetched in sync with playback?

Non-functional clarifications

- Play start under 200 ms (network + cache).
- Gapless: 0 ms silence between tracks.
- Background playback survives task switching, screen off.
- Battery: 8+ hours of playback.
- Offline downloads survive reboot.

Why ask these? (the signal)

Asking about MediaSession + foreground service shows you understand background audio on Android.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

ExoPlayer in a **MediaBrowserServiceCompat / MediaSessionService** (foreground service). The service owns playback; the Activity is a remote control. **Cache**: ExoPlayer's SimpleCache for stream chunks. **Downloads**: DownloadManager per track. **Queue** managed in service; pre-fetches the next track 10s before transition.

Major components

- PlaybackService (MediaSessionService) — owns ExoPlayer, MediaSession.
- QueueManager — current queue, repeat, shuffle, autoplay.
- StreamCache (SimpleCache) — bounded disk cache of stream chunks.

- DownloadManager — for explicit offline tracks.
- MediaSession — exposes controls to OS (lockscreen, BT, Auto).
- PlaylistRepository — local Room + server sync.

Low-Level Design (LLD)

MediaSessionService (newer API) handles foreground service for media automatically and exposes a MediaSession the system uses for lockscreen / BT / Wear OS controls.

Gapless transition: ExoPlayer's ConcatenatingMediaSource holds the current + next; the player primes the next track's decoder during the previous track's final seconds.

Stream cache: ExoPlayer SimpleCache bounded at ~1GB. Recently played tracks remain cached so replay is instant.

Offline download: DownloadManager fetches the full track file; DownloadHelper renders the same MediaItem URI usable for both online and offline.

Playlist sync: same offline-first + outbox pattern from Chapter 15; offline add/remove → PendingOp → sync.

4. How to Code & Explain (Win the Interviewer)

Open: *"Background audio on Android is its own challenge. We use MediaSessionService — a foreground service that owns ExoPlayer and exposes a MediaSession. This is the only way to keep playback alive through screen lock and BT."*

Play one track: *"User taps. Activity sends a play command via MediaController. Service builds a MediaItem with the stream URL, attaches to ExoPlayer with SimpleCache. Playback starts; cache fills."*

Gapless: *"When ~10s remain on the current track, we add the next to the player's queue. ExoPlayer's playback continues without a buffer flush, so the transition is 0 ms silence."*

Lockscreen: *"MediaSession state changes propagate to the OS via setPlaybackState / setMetadata. Lockscreen, BT headset buttons, Wear OS, Android Auto — all use this without app code."*

> ■ ***Tip:*** *Reference* ***MediaSessionService*** *(Media3) explicitly. Most candidates say 'foreground service for music' which is correct but old-fashioned.*

5. Actual Code (Reference Snippets)

MediaSessionService + ExoPlayer + cache + gapless queue + offline download

Kotlin

```kotlin
class MusicPlaybackService : MediaSessionService() {
    private lateinit var player: ExoPlayer
    private lateinit var session: MediaSession

    override fun onCreate() {
        super.onCreate()
        val cacheDataSource = CacheDataSource.Factory()
            .setCache(StreamCache.instance(this))      // 1 GB SimpleCache
            .setUpstreamDataSourceFactory(DefaultHttpDataSource.Factory())
            .setFlags(CacheDataSource.FLAG_IGNORE_CACHE_ON_ERROR)

        player = ExoPlayer.Builder(this)
            .setMediaSourceFactory(DefaultMediaSourceFactory(this)
                .setDataSourceFactory(cacheDataSource))
            .setAudioAttributes(AudioAttributes.Builder()
                .setContentType(C.AUDIO_CONTENT_TYPE_MUSIC)
                .setUsage(C.USAGE_MEDIA).build(),
                /* handleAudioFocus */ true)
            .setWakeMode(C.WAKE_MODE_NETWORK)
            .build()
```

```
        session = MediaSession.Builder(this, player)
            .setCallback(MyCallback())
            .build()
    }

    override fun onGetSession(controllerInfo: MediaSession.ControllerInfo) = session

    override fun onDestroy() {
        player.release(); session.release(); super.onDestroy()
    }
}

class QueueManager(private val service: MusicPlaybackService,
                   private val player: ExoPlayer) {
    fun playTrack(track: Track, queue: List<Track>) {
        val items = queue.map { it.toMediaItem() }
        player.setMediaItems(items, queue.indexOf(track), 0L)
        player.prepare()
        player.playWhenReady = true
    }
    fun enqueueNext(next: Track) {
        // gapless: feed the next item before current ends
        player.addMediaItem(next.toMediaItem())
    }
}

// Offline
class OfflineDownloader(val ctx: Context) {
    fun downloadAlbum(album: Album) {
        for (track in album.tracks) {
            val req = DownloadRequest.Builder(track.id, Uri.parse(track.streamUrl))
                .setData(track.title.toByteArray()).build()
            DownloadService.sendAddDownload(
                ctx, MyDownloadService::class.java, req, false)
        }
    }
    fun localUriIfOffline(track: Track): Uri? {
        val download = DownloadHelper.getDownloadHelperForMediaItem(
            ctx, MediaItem.fromUri(track.streamUrl))
        return /* lookup from DownloadIndex */ null
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why MediaSessionService instead of a regular Service?

MediaSessionService (Media3) handles foreground-service mechanics, lockscreen integration, BT controls, and Auto/Wear connectivity. Vastly less boilerplate than the old MediaBrowserService / MediaSessionCompat. It's the current best practice.

Q: How does gapless work technically?

ExoPlayer pre-warms the next decoder while still playing the previous track. ConcatenatingMediaSource (or now a Timeline window) presents adjacent tracks as one continuous playback. The player's render pipeline doesn't flush between items.

Q: Why cache stream chunks separately from downloads?

Streaming cache is opportunistic (anything you've played) and bounded by size, LRU-evicted. Downloads are explicit, user-managed, and persistent. Two different policies, two different storage areas.

Q: How do you handle headphone unplug while playing?

AudioManager fires an ACTION_AUDIO_BECOMING_NOISY broadcast; ExoPlayer's handleAudioBecomingNoisy = true pauses automatically. User expects this — no more music blasting through phone speakers.

Q: How does playlist sync resolve a conflict (track added twice on two devices)?

Each playlist item has a UUID. Set semantics → adding the same trackId twice is a no-op. Reordering uses a fractional position (e.g. position = midpoint of two neighbours) — concurrent reorders converge.

7. What Better? Alternative Approaches

Per-user audio normalisation — adjust volume so tracks across genres feel consistent.

Predictive downloads: user usually plays Album X every morning → pre-download on charge.

Crossfade via two ExoPlayer instances and a volume ramp.

Spatial audio (Dolby Atmos) — separate codec; pre-encoded streams.

WearOS / Auto projection — MediaBrowserService exposes the catalogue tree; no extra UI code needed.

Key takeaways

- MediaSessionService = the modern way for background audio.
- ExoPlayer's SimpleCache for stream caching.
- Gapless = pre-warming next via ConcatenatingMediaSource.
- DownloadManager handles full offline mode.

Chapter 31: Design a Food Delivery App (Swiggy / Zomato)

Difficulty: Hard | **Tags: Marketplace Real-time tracking State machines Cart**

Three apps in one: customer, restaurant, delivery partner — all coordinated by an order state machine.

1. What's the Problem?

Build a food delivery app. Customer opens app, sees nearby restaurants, picks dishes, places order with payment, watches real-time tracking from restaurant to door. Restaurant accepts order, marks ready. Delivery partner picks up, navigates, delivers. The three sides synchronise via a shared order state.

Scale to assume

- Tens of millions of users, peak 100K+ concurrent orders at meal times.
- Order lifecycle: 30–60 min, real-time tracking last 20 min.
- Three different mobile apps but one backend.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Browse restaurants by location, cuisine, rating?
- Cart, customisations, promo codes?
- Place order, pay (Chapter 23 patterns)?
- Live tracking with ETA?
- Order status updates — push?
- Cancellation policy?
- Reviews + ratings post-delivery?

Non-functional clarifications

- Sub-second order state propagation to all three sides.
- Reliable real-time location during last-mile.
- Payment idempotency.
- Heavy image catalog — restaurant photos, dishes.

Why ask these? (the signal)

Asking 'three apps, one state machine' shows you see the system-wide picture, not just the customer app.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Three mobile clients (Customer, Restaurant, Driver). One central **OrderService** owns the state machine (PLACED → ACCEPTED → COOKING → READY → PICKED_UP → DELIVERED). State changes pushed to all interested parties via WebSocket + FCM fallback. **Tracking** as a separate channel — driver app pings location, customer + restaurant subscribe.

Major components

- Customer app — Browse / Cart / Checkout / Live tracking screens.
- Restaurant app — Order queue, accept/reject, mark ready.
- Driver app — Offers (similar to Chapter 27), navigation.

- OrderService — state machine; event log.
- TrackingService — driver location pings; subscriber routing.
- DispatchService — assigns driver to ready order (chapter 27 patterns).

Low-Level Design (LLD)

Order state machine with explicit transitions; invalid transitions rejected. Each state change emits an event consumed by interested subscribers.

Cart is fully client-side until checkout; server validates prices, stocks (some dishes can run out), and totals before charge.

Live tracking: customer's app subscribes to driver location only after PICKED_UP; before that, show 'Restaurant is preparing'.

ETA: server computes from driver location + traffic + restaurant cook time history. Updates every minute.

Cancellation window: free until ACCEPTED, partial refund until COOKING, no cancel after PICKED_UP.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is a marketplace coordinated by an order state machine. Three different apps subscribe to events on the same order. The mobile side is mostly: render the current state, allow valid actions for the role."*

Walk an order: *"Customer checkouts → POST /order with idempotencyKey → server creates Order(PLACED) → emits 'order.placed' → restaurant app shows it in their queue. Restaurant taps Accept → state=ACCEPTED → customer gets push. Mark ready → state=READY → dispatch finds a driver (chapter 27 patterns)."*

Tracking: *"Driver picks up → state=PICKED_UP → customer app subscribes to driver location channel. Driver pings every 4s; customer sees live marker on map. Dropoff → state=DELIVERED → channels close."*

State machine in code: *"We model transitions explicitly. Server rejects 'mark ready' if state is still PLACED. Saves bugs."*

> ■ ***Tip:*** *Mention **SLAs per restaurant** as a real-world wrinkle: a restaurant that doesn't accept within 90s gets auto-cancelled. Built into the state machine as a timeout.*

5. Actual Code (Reference Snippets)

State machine + customer tracking VM + restaurant queue VM

Kotlin

```kotlin
enum class OrderState { PLACED, ACCEPTED, COOKING, READY, PICKED_UP, DELIVERED, CANCELLED }

object OrderStateMachine {
    private val transitions = mapOf(
        OrderState.PLACED      to setOf(OrderState.ACCEPTED, OrderState.CANCELLED),
        OrderState.ACCEPTED    to setOf(OrderState.COOKING, OrderState.CANCELLED),
        OrderState.COOKING     to setOf(OrderState.READY),
        OrderState.READY       to setOf(OrderState.PICKED_UP),
        OrderState.PICKED_UP   to setOf(OrderState.DELIVERED),
    )
    fun canTransition(from: OrderState, to: OrderState) =
        transitions[from]?.contains(to) == true
}

// Customer-side live tracking
class TrackingViewModel(
    private val orderRepo: OrderRepository,
    private val trackingChannel: TrackingChannel
) : ViewModel() {

    private val _state = MutableStateFlow(TrackingState())
```

```
    val state: StateFlow<TrackingState> = _state

    fun subscribe(orderId: String) {
        viewModelScope.launch {
            // observe order state
            orderRepo.observe(orderId).collect { order ->
                _state.update { it.copy(order = order) }
                if (order.state == OrderState.PICKED_UP && !_state.value.trackingLive) {
                    startLiveTracking(orderId, order.driverId!!)
                }
            }
        }
    }

    private fun startLiveTracking(orderId: String, driverId: String) {
        _state.update { it.copy(trackingLive = true) }
        viewModelScope.launch {
            trackingChannel.subscribe(driverId).collect { p ->
                _state.update { it.copy(driverLocation = LatLng(p.lat, p.lng),
                                        etaMinutes = p.etaMinutes) }
            }
        }
    }
}

// Restaurant-side order queue
class RestaurantQueueViewModel(private val repo: RestaurantRepository) : ViewModel() {
    val pendingOrders: Flow<List<Order>> = repo.pendingOrdersStream()

    fun accept(orderId: String) = viewModelScope.launch {
        repo.transition(orderId, OrderState.ACCEPTED)
    }
    fun reject(orderId: String, reason: String) = viewModelScope.launch {
        repo.transition(orderId, OrderState.CANCELLED, reason)
    }
    fun markReady(orderId: String) = viewModelScope.launch {
        repo.transition(orderId, OrderState.READY)
    }
}
```

6. Cross-Questions & How to Answer Them

Q: What if the restaurant accepts but the driver never picks up?

Server timer: if state hasn't transitioned past READY within N minutes, dispatch escalates — pings another driver, or surfaces a customer notification with refund option.

Q: How is ETA computed?

Server: time_remaining_to_cook + drive_time(restaurant→customer using Maps API + traffic) + base buffer. Refreshed every 60s as driver location updates. Customer app shows a count-down; never reduces faster than wall time.

Q: Same dish: 100 people order at once but stock is 10.

Atomic decrement on the dish's available count at order placement. SQL: UPDATE dishes SET stock = stock - 1 WHERE id = ? AND stock > 0. If 0 rows affected, order is rejected with 'out of stock'.

Q: How is the customer side updated when the restaurant rejects?

OrderService transitions state and emits event. WebSocket pushes to the customer's app; if app is killed, FCM. Auto-initiated refund via payment service.

Q: How do you handle stale offers to drivers?

Each offer has a 10s expiry. If unaccepted, dispatcher moves on. Driver's app filters offers older than expiry on render.

7. What Better? Alternative Approaches

Batch orders: one driver picks up multiple orders from one restaurant in nearby drops. Dispatch logic gets clever.

Pre-cook prediction: ML model predicts upcoming order volume; restaurants pre-cook popular items.

Heatmap on driver app showing high-demand areas (incentives during peak).

Customer pre-checkout location share — let customers add 'I'm at home' before ordering; speeds up routing.

Restaurant tablet/POS integration instead of a phone app — built on the same APIs.

Key takeaways

- Shared state machine across three apps.
- Explicit transitions reject invalid ones.
- Live tracking opens only after PICKED_UP.
- ETA must never reduce faster than wall time.

Chapter 32: Design Dropbox / Drive File Sync (mobile)

Difficulty: Hard | **Tags: File sync Chunking Conflict Offline**

Two-way sync between device and cloud — efficient chunked uploads, conflict resolution, selective sync.

1. What's the Problem?

Build the mobile Dropbox app. The user can upload files, mark folders for offline access (downloaded), get notified when files change from another device, edit files, and resolve conflicts when two devices change the same file. Efficient on bandwidth (delta sync) and storage.

Scale to assume

- Files: KB to GB; libraries of tens of thousands of files.
- Multi-device: phone, tablet, laptop, web — all synced.
- Bandwidth conscious: don't re-upload identical chunks.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Upload files (camera, picker, share intent)?
- Selective sync — entire folders or per-file?
- Conflict resolution — last write wins, both kept?
- Versions / history?
- Sharing — link, permission-controlled?
- Offline preview of synced files?

Non-functional clarifications

- Resumable uploads/downloads (large files).
- Block-level dedup (don't re-upload unchanged chunks).
- Notifications when files change.
- Battery-friendly background sync.

Why ask these? (the signal)

Asking about block-level dedup (rolling hash, content-defined chunking) is the senior signal.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Each file is split into **blocks** (~4MB). Each block has a SHA-256 hash. Server stores blocks once (content-addressed) and a file index (filename → ordered list of block hashes). On upload: hash blocks, ask server which hashes it lacks, upload only those. On download: same, in reverse. Changes pushed via a delta log.

Major components

- BlockStore (server) — content-addressed; deduped storage.
- FileIndex — filename → block hashes + metadata.
- ChangeLog — append-only events for any user's namespace.
- Mobile FileObserver — watches local for changes.
- Mobile SyncEngine — pushes, pulls, applies delta log.

- OfflineMarker — selective sync flag per folder.

Low-Level Design (LLD)

Chunking strategy: fixed 4MB blocks for simplicity (Dropbox); rolling hash content-defined chunks for better dedup (rsync, restic). The latter dedupes across files when only a middle portion changes.

Upload: client splits file → hash each block → POST /uploadStart with hashes → server returns which it doesn't have → client uploads only those + commits a new file version pointing at the full block list.

Download: client fetches file index → requests blocks it lacks → assembles locally.

Conflict: when client tries to commit a new version of file F but server's current version is unexpectedly newer, server rejects with 409 + creates a conflict copy. Client renames its local copy 'name (Alice's conflicting copy)'.

Delta log: server's change log is queried with a cursor; client receives events for added/modified/deleted files since last sync.

4. How to Code & Explain (Win the Interviewer)

Open: *"This is a block-level deduped sync. Each file becomes a list of block hashes. Server stores blocks once; multiple files referencing the same block share storage. Re-uploads of unchanged regions are skipped."*

Walk an upload: *"User adds a 100MB video → 25 × 4MB blocks → SHA-256 each → POST hash list → server says 'I need blocks 3, 7, 9' → client uploads 3 blocks (12MB) → commit. Most users see massive savings — same screenshots in multiple folders, small edits to documents, etc."*

Pull: *"On app open or push, query /changes?cursor=X → server returns events for [file A modified, file B deleted, etc.]. For each modified file, fetch missing blocks. Apply locally. Advance cursor."*

Conflict story: *"User edits document.txt offline; another device also edits. On sync, the server sees a version mismatch and creates a 'conflict copy'. Both versions exist; user picks. We don't try to merge — too error-prone for binary files."*

> ■ ***Tip:*** *Reference **rsync's rolling hash** as the gold standard for block boundaries — even adding a byte at the front of a 1GB file dedupes against the original.*

5. Actual Code (Reference Snippets)

Block-hash sync with server-side dedup + delta-log pull

Kotlin

```
class BlockHasher {
    private val md = MessageDigest.getInstance("SHA-256")
    fun hashBlock(b: ByteArray, off: Int, len: Int): String {
        md.reset(); md.update(b, off, len)
        return md.digest().joinToString("") { "%02x".format(it) }
    }
}

class FileSyncRepository(
    private val api: SyncApi,
    private val db: AppDatabase,
    private val fs: FileSystem
) {
    private val blockSize = 4 * 1024 * 1024

    suspend fun upload(localPath: String): SyncResult {
        val file = fs.open(localPath)
        val hashes = mutableListOf<String>()
        val hasher = BlockHasher()
        val blockBuf = ByteArray(blockSize)
        var idx = 0
```

```
        while (true) {
            val n = file.read(blockBuf)
            if (n <= 0) break
            val hash = hasher.hashBlock(blockBuf, 0, n)
            hashes += hash
            db.blockDao().recordLocal(hash, localPath, idx, n)
            idx++
        }
        // Ask server which blocks are missing
        val missing = api.startUpload(filename = localPath, blocks = hashes)
            .missingBlocks
        for (h in missing) {
            val (path, off, len) = db.blockDao().locate(h)
            api.uploadBlock(h, fs.read(path, off, len))
        }
        return api.commit(localPath, blockHashes = hashes)
    }

    suspend fun pull(sinceCursor: String): String {
        val resp = api.changesSince(sinceCursor)
        for (ev in resp.events) {
            when (ev) {
                is ChangeEvent.Added, is ChangeEvent.Modified -> downloadFile(ev.path, ev.blockHashes)
                is ChangeEvent.Deleted -> fs.delete(ev.path)
            }
        }
        return resp.nextCursor
    }

    private suspend fun downloadFile(path: String, blocks: List<String>) {
        val tmp = fs.tempFile()
        for (h in blocks) {
            val bytes = db.blockDao().localBytes(h)
                ?: api.downloadBlock(h).also { db.blockDao().recordLocal(h, tmp.path, /*off*/ 0, it.size) }
            tmp.append(bytes)
        }
        fs.moveAtomic(tmp, path)
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Block-level vs file-level dedup — why the complexity?

Block-level dedups across files (same screenshot in different folders), across users (everyone has the same OS install file), and across versions (1-byte edit to a 1GB log doesn't re-upload 1GB). File-level can only dedup whole-file duplicates.

Q: How does conflict 'just create a copy' beat 'try to merge'?

Binary files (photos, videos, PDFs) can't be merged automatically. Text files merge well with three-way diff but it's error-prone for users. Creating a conflict copy preserves both intents safely; users can manually merge if they want.

Q: How does push work — every change to every device?

Long-poll or WebSocket subscribed to user's namespace. Server emits change events; client pulls full content on demand. Avoids broadcasting bytes — only metadata.

Q: Selective sync — how is it implemented?

A 'sync_folders' table on the client lists which folders are mirrored locally. Pull only delivers events for those folders. Other files appear as placeholders (icon + size); tapping triggers fetch.

Q: How do you handle a 10GB file on a phone with 8GB free?

Stream-aware fetch: download blocks as needed for the operation (preview, share). Don't auto-fully-download huge files. Or refuse upload from mobile if it would exceed local quota.

7. What Better? Alternative Approaches

Content-defined chunking (rolling hash) instead of fixed blocks — dedups around insertions and deletions, not just overwrites.

Compression before chunking — saves bandwidth for text-heavy content.

End-to-end encryption — client encrypts before chunking; server is blind. Boxcryptor / Tresorit do this.

Live collaborative edit — for docs, switch from file sync to operational-transform / CRDT model.

Server-side preview generation — instant thumbnails for photos / PDFs without downloading the whole file.

Key takeaways

- Files are lists of block hashes — content-addressed storage.
- Hash-first uploads skip redundant bytes.
- Conflicts → both copies kept; manual merge.
- Delta log + cursor = efficient pull.

Chapter 33: Design Google Maps Mobile (Tiles, Search, Navigation)

Difficulty: Hard | **Tags: Tiles Vector maps Search Navigation**

Render map tiles, search POIs, compute driving routes with live re-routing, work offline.

1. What's the Problem?

Build a mobile maps app. Pan/zoom a vector map smoothly; search for places by name or category; tap a POI to see details; request driving directions and follow turn-by-turn navigation that re-routes if the user takes a wrong turn. Support offline maps for downloaded regions.

Scale to assume

- Billions of users, trillions of map renders/day.
- Tiles served from CDN; client caches recent tiles.
- Navigation runs minutes to hours, must survive screen-off.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Vector tiles or raster tiles?
- Search — name, category (restaurants), POI?
- Routing — driving, walking, transit, biking?
- Turn-by-turn voice prompts?
- Offline regions — pre-downloaded packs?
- Live traffic overlay?

Non-functional clarifications

- Map renders at 60fps during pan/zoom.
- Initial map under 500ms from cache.
- Navigation survives screen-off (foreground service).
- Bandwidth-conscious — tile sharing, vector format.

Why ask these? (the signal)

Asking about vector vs raster reveals you know vector tiles are the modern industry standard for mobile.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Vector tiles (MVT format) downloaded from a CDN, decoded and rendered on-device with OpenGL / Vulkan / Mapbox style. Tiles cached in a tile DB on disk. Search and routing are server APIs. Navigation: a route is a list of steps + a GeoJSON polyline; the device tracks GPS, snaps to road, advances steps, re-routes if off.

Major components

- TileRepository — fetches MVT tiles by (z, x, y); cache in disk DB.
- MapView — GL-based; renders style + tiles.
- SearchRepository — server API for place lookup.
- RoutingRepository — server API for directions.
- NavigationEngine — snaps GPS to route; advances steps; voice prompts.

- OfflineRegionManager — bulk tile + routing-graph download.
- Foreground service for active navigation.

Low-Level Design (LLD)

Vector tile: tiny binary blob (~10KB) listing roads, POIs, labels per tile. Decoded on device, rendered via a style (JSON describing colours / widths / icons). Same tile renders day/night/satellite without re-fetch.

Tile cache: bounded LRU on disk; index by (z, x, y). Pre-fetch one zoom level ahead for smooth zoom.

Route step: { polyline, distance, duration, maneuver: TURN_RIGHT, voice: 'Turn right in 100m' }.

Snap-to-road: each GPS sample projected onto the route polyline; advance step when within 30m of next waypoint.

Re-route trigger: if GPS is >50m from polyline for 3 consecutive samples, request a new route from current location.

4. How to Code & Explain (Win the Interviewer)

Open: *"Vector tiles + GL rendering. Tiles are downloaded as compact binary, decoded and styled locally. Style can change (dark mode, traffic colours) without re-downloading data."*

Search: *"Search box → server returns places ranked by distance + relevance + popularity. Tap a result → camera animates to its lat/lng; bottom sheet shows details fetched separately."*

Navigation: *"Start nav → fetch route from /directions → store steps locally → start foreground service → process GPS every second, snap to road, advance step. On step change, fire voice prompt via TTS. Off-route → re-route from current location; show toast."*

Offline: *"User downloads 'home region' → all tiles for zoom 0-15 within bbox + routing graph for the region. Navigation works fully offline using on-device routing."*

> ■ ***Tip:*** *Mention* ***Mapbox GL Native*** *or* ***MapLibre*** *as the open-source rendering engine — naming the engine signals you know the implementation, not just the concept.*

5. Actual Code (Reference Snippets)

Tile cache + navigation engine + foreground service

Kotlin

```kotlin
class TileRepository(private val api: TileApi, private val db: AppDatabase) {
    suspend fun getTile(z: Int, x: Int, y: Int): ByteArray? {
        val cached = db.tileDao().get(z, x, y, expiresAfter = System.currentTimeMillis())
        if (cached != null) return cached.bytes
        val data = try { api.fetchTile(z, x, y) } catch (e: IOException) { return null }
        db.tileDao().put(TileBlob(z, x, y, data, System.currentTimeMillis() + 7.days))
        return data
    }
    suspend fun prefetchAround(z: Int, x: Int, y: Int) {
        // 3x3 around the center
        for (dx in -1..1) for (dy in -1..1)
            launch { getTile(z, x + dx, y + dy) }
    }
}

data class RouteStep(val polyline: List<LatLng>, val distanceM: Int,
                     val durationS: Int, val maneuver: Maneuver,
                     val instruction: String)

class NavigationEngine(
    private val route: List<RouteStep>,
    private val tts: TextToSpeech
```

```
) {
    private var currentStep = 0
    fun onLocation(loc: Location): NavigationState {
        if (currentStep >= route.size) return NavigationState.Arrived
        val step = route[currentStep]
        val nearest = projectOntoPolyline(loc, step.polyline)
        val dToNextWaypoint = distanceM(loc, step.polyline.last())
        if (dToNextWaypoint < 30) {
            currentStep++
            if (currentStep < route.size)
                speakInstruction(route[currentStep].instruction)
        }
        if (nearest.offRouteM > 50) return NavigationState.OffRoute
        return NavigationState.OnRoute(
            distanceToNextManeuverM = dToNextWaypoint,
            remainingStepsCount = route.size - currentStep)
    }
    private fun speakInstruction(t: String) =
        tts.speak(t, TextToSpeech.QUEUE_FLUSH, null, "nav")
}

class NavigationService : Service() {
    override fun onStartCommand(i: Intent?, f: Int, sid: Int): Int {
        startForeground(NAV_ID, navNotification())
        val fused = LocationServices.getFusedLocationProviderClient(this)
        val request = LocationRequest.Builder(
            Priority.PRIORITY_HIGH_ACCURACY, 1000L).build()
        fused.requestLocationUpdates(request, callback, Looper.getMainLooper())
        return START_STICKY
    }
    private val callback = object : LocationCallback() {
        override fun onLocationResult(r: LocationResult) {
            r.lastLocation?.let { engine.onLocation(it) }
        }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why vector tiles over raster?

Smaller payloads (~5–10× less), one-tile-fits-many-styles, smooth re-styling for night mode / traffic overlay without re-download, and crisp at any zoom level. Industry standard since ~2014.

Q: How does live traffic overlay work without huge data?

Server publishes traffic as a separate vector tile layer at coarse zoom (15 → city level). Updated every few minutes. Client merges in the same MapView as a coloured-line layer.

Q: How does offline routing work without the road graph?

Offline region download includes the routing graph: a compressed representation of roads + connectivity + speed limits. On-device Dijkstra (or contraction hierarchies for speed) finds paths.

Q: How is GPS jitter handled?

Kalman filter or simple weighted average over the last 3 samples. Snap to road as discussed. Drop samples with accuracy > 50m.

Q: What if the user turns the screen off during nav?

Foreground service keeps location and the engine alive. Voice prompts continue via the system audio. Wake the screen on major turns.

7. What Better? Alternative Approaches

Predictive prefetch of tiles along the route — preload tiles for the next 5 km so you stay online-quality even on cellular drops.

Lane guidance: extra layer in tiles for highway lanes; gives the 'use the right two lanes' instruction.

Indoor maps via building polygons + floor switcher.

Crowdsourced traffic: anonymous speed reports from your own users (Waze model).

3D rendering: vector tiles + extruded buildings for landmarks.

Key takeaways

- Vector tiles + GL = modern maps.
- Foreground service is mandatory for navigation.
- Snap-to-road + step advance is the core nav loop.
- Offline = tiles + routing graph in a region pack.

Chapter 34: Design YouTube Mobile (Watch + Recommendations)

Difficulty: Hard | Tags: Video Player Recommendations Mini-player

Video-first app with recommendations, mini-player, comments, picture-in-picture — Netflix patterns + UGC complexity.

1. What's the Problem?

Build YouTube on Android. The user opens to a personalised home feed of video thumbnails, taps one, watches with full screen / picture-in-picture toggle, sees comments, follows the recommended-next-up rail, and a swipe down minimises to a mini-player while they keep browsing. Resume position synced across devices.

Scale to assume

- 2B+ users, 1B+ hours watched daily.
- Mixed bandwidth — 240p to 4K.
- Comments and reactions update in near real time on hot videos.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Home feed of recommendations?
- Watch page — player + title + comments + recs?
- Mini-player + PiP?
- Like/dislike, subscribe, share?
- Search?
- Offline downloads (Premium)?
- Live streams?

Non-functional clarifications

- Same as Netflix (Chapter 29) — ABR, low time-to-first-frame.
- Smooth mini-player handoff without restarting playback.
- Comment list with millions of comments — paginated.
- Picture-in-Picture across screens.

Why ask these? (the signal)

Asking about 'one ExoPlayer across mini and full' steers to the correct architectural pattern — Service-owned player.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

One central ExoPlayer owned by a service. Both the Watch screen and the mini-player are views that **attach** to the same underlying player. Switching between full and mini is a view-handoff, not a player restart. **Recommendations** are a personalised feed similar to Instagram (Chapter 25). **Comments** use a paged feed similar to Chapter 16.

Major components

- PlaybackService — owns ExoPlayer + MediaSession.

- WatchActivity / Composable — full-screen player + comments + next-up rail.
- MiniPlayerContainer — global Compose overlay; attaches to the same player surface.
- RecommendationRepository — home + 'next up' rail.
- CommentsRepository — paginated; supports threading.
- DownloadManager — Premium offline.

Low-Level Design (LLD)

Player surface handoff: the SurfaceView (or TextureView) is reparented between watch and mini-player containers. ExoPlayer's setVideoSurface called once on attach. No buffer flush.

Mini-player is a separate Compose layer at the bottom of the activity. Vertical drag-down on watch screen swaps. Tap to expand back.

Picture-in-Picture: enterPictureInPictureMode() on Activity. The OS handles the floating window; we draw the surface the same way.

Recommendations: ML-ranked feed; client-side just renders thumbnails. Tap → start playback in WatchActivity; player switches source without recreating.

Comments: Paging 3 from /comments?cursor=…; new comments via WebSocket on hot videos; old ones loaded on scroll.

4. How to Code & Explain (Win the Interviewer)

Open: *"The signature feel is: tap thumbnail → instantly playing → swipe down → mini-player floats while you browse. The technical key is one ExoPlayer, shared between views via surface handoff."*

Surface handoff: *"WatchActivity has a SurfaceView attached to ExoPlayer. User drags down → animation moves the view into the mini-player container — same SurfaceView, same player. The player keeps decoding; the user perceives zero glitch."*

Recommendations: *"Home feed is server-ranked thumbnails. Tap → POST current-video-state, navigate to Watch. ExoPlayer's MediaItem source changes; new content starts in < 2 seconds. Continue Watching watermark resumes if available."*

Comments: *"Comments page is its own Paging source on /comments?cursor. Threaded replies are children with parentCommentId. Same pattern as Instagram replies."*

> ■ ***Tip:*** *Bring up **PiP** by name and mention* `setAutoEnterEnabled(true)` *on Android 12+ — PiP starts automatically when the user navigates away.*

5. Actual Code (Reference Snippets)

Service-owned player + surface handoff for mini-player + PiP

Kotlin

```kotlin
// Service that owns the player and exposes it
class PlaybackService : MediaSessionService() {
    lateinit var player: ExoPlayer
    private lateinit var session: MediaSession

    override fun onCreate() {
        super.onCreate()
        player = ExoPlayer.Builder(this)
            .setLoadControl(DefaultLoadControl.Builder()
                .setBufferDurationsMs(30_000, 60_000, 1_500, 5_000)
                .build())
            .build()
        session = MediaSession.Builder(this, player).build()
    }
```

```
    override fun onGetSession(c: MediaSession.ControllerInfo) = session
}

// Watch activity binds, attaches its surface
class WatchActivity : ComponentActivity() {
    private var player: ExoPlayer? = null
    private val connection = object : ServiceConnection {
        override fun onServiceConnected(name: ComponentName?, b: IBinder?) {
            player = (b as PlaybackService.LocalBinder).service.player
            playerView.player = player
        }
        override fun onServiceDisconnected(name: ComponentName?) {}
    }
    override fun onStart() { super.onStart(); bindService(...) }

    fun expandToMini() {
        playerView.player = null
        MiniPlayerHost.attach(player!!)   // reparent surface
        finish()                          // or pop watch fragment
    }

    // Picture in picture
    override fun onUserLeaveHint() {
        if (player?.isPlaying == true) {
            enterPictureInPictureMode(
                PictureInPictureParams.Builder()
                    .setAspectRatio(Rational(16, 9))
                    .setAutoEnterEnabled(true)
                    .build()
            )
        }
    }
}

// Mini player overlay
object MiniPlayerHost {
    private var miniSurface: PlayerView? = null
    fun attach(player: ExoPlayer) {
        miniSurface?.player = player
        miniSurface?.visibility = View.VISIBLE
    }
    fun onTap() {
        // pass current player back to a new WatchActivity
        startActivity(WatchActivity.intent(player = miniSurface?.player))
        miniSurface?.visibility = View.GONE
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why one player across mini and full?

Two players would require re-buffering on every transition — visible glitch. One player just swaps which view its surface is rendered to. Smooth, free, the YouTube signature.

Q: How does the player survive WatchActivity finishing?

It lives in the foreground service. Activities bind/unbind; the service stays. Same pattern as music apps.

Q: How is the home feed personalised so quickly?

Server precomputes the user's home batch on push events (new sub uploads, etc.) and caches; cold open fetch is just a read of the cached batch. Heavy ML happens offline.

Q: How does Continue Watching survive across devices?

Server-side progress checkpoint every 30s, same as Netflix. On video open, fetch the highest watermark across the user's devices.

Q: How are comments shown in real time during a live stream?

Live comments stream over WebSocket; client buffers and renders at a throttled rate (so 10 comments/sec don't overwhelm). Old comments paged after stream ends.

7. What Better? Alternative Approaches

Adaptive comments throttling on hot videos — only render every Nth, keep the rest in buffer.

Pre-fetch next video in autoplay queue — buffer the first 30s before the current ends.

Shorts integration — vertical swipe to a different content type with its own player surface and ABR profile.

Multi-audio / dubbing — same Widevine + manifest pattern as Netflix.

Cast to TVs — MediaRouter + Cast SDK; player handoff to the Cast device.

Key takeaways

- One ExoPlayer; multiple views attach via surface.
- Mini-player is a reparenting trick, not a new player.
- PiP via enterPictureInPictureMode() + setAutoEnterEnabled.
- Comments + recs reuse Paging 3 patterns from earlier chapters.

Chapter 35: Design a Live Streaming App (Twitch mobile)

Difficulty: Hard | **Tags: Live video Low latency Chat Real-time**

Sub-3-second-latency live video, parallel chat with thousands of viewers, hearts & reactions, follower notifications.

1. What's the Problem?

Build the viewer side of Twitch on Android. Watch a live stream with low latency (under 3s), see live chat scrolling with thousands of viewers messaging, send messages and reactions, follow channels, get push notifications when followed streamers go live. Recover gracefully from network drops and stream interruptions.

Scale to assume

- Hot streams: 1M+ concurrent viewers, hundreds of chat messages per second.
- Latency target: stream under 3s (low-latency HLS / LL-HLS / WebRTC).
- Chat: high-volume real-time, with moderation.

2. How Should You Think? (Clarifying Questions)

Functional clarifications to ask

- Stream playback with low latency?
- Live chat — send + receive?
- Reactions / hearts overlay?
- Follow / subscribe?
- Notifications when followed channel goes live?
- VOD playback after stream ends?

Non-functional clarifications

- Stream latency under 3 seconds.
- Chat throughput: render hundreds of messages/sec without dropping frames.
- Chat memory bounded — drop old messages.
- ABR for cellular variability.
- Reactions cheap to send and render.

Why ask these? (the signal)

Asking about LL-HLS vs WebRTC vs DASH shows you understand the latency-vs-compatibility tradeoff.

3. How to Start Discussion: HLD & LLD

High-Level Design (HLD)

Video: LL-HLS (or DASH-LL or WebRTC for ultra-low latency) via ExoPlayer with a short buffer (3–5s) and small segment size. **Chat**: dedicated WebSocket for the channel; messages stream in, rendered with coalescing for high rates. **Reactions**: lightweight overlay channel; render small animations. **Notifications**: FCM push on stream-start events for followed channels.

Major components

- VideoPlayer — ExoPlayer with LL-HLS, low-latency LoadControl.
- ChatChannel — per-stream WebSocket; receive+throttle+render.
- ChatViewModel — bounded buffer of latest 200 messages.

- ReactionsOverlay — Compose animations for hearts/emoji.
- FollowRepository — list of followed channels.
- StreamLifeNotifier — server-side cron pushes 'X is live' to followers.

Low-Level Design (LLD)

LL-HLS: low-latency HLS uses 1-2 second segments + partial segment HTTP/2 push. Buffer 3–5s, vs 20–30s for standard HLS.

Stream interruption: on stall, ExoPlayer's BehaviourOnFailure tries to recover from the manifest's edge. On extended failure, show 'Reconnecting…' overlay and retry every 2s.

Chat coalescing: on a 1M-viewer stream, chat may be 1000+ msg/sec. Render at most 30 msg/sec to the visible list; buffer the rest in a ring; OnViewportScrolled to top reveals 'load buffered'.

Chat list rendering: never animate inserts (would jitter); just append. Pin to bottom unless user has scrolled up.

Reactions: send via WebSocket as a 1-byte emoji code; render a particle on screen with random offset. Batch many to one animation if they arrive close together.

4. How to Code & Explain (Win the Interviewer)

Open: *"Two real-time streams in parallel: video and chat. Video needs low latency and adaptive bitrate. Chat needs to throttle at the visible rendering rate while the underlying stream is much faster."*

Latency: *"Standard HLS has 20–30s latency from broadcaster to viewer because of long segments. LL-HLS shrinks segments to 1–2s + uses partial segment delivery → ~3s. WebRTC can push it under 500ms but is harder to scale to a million viewers."*

Chat: *"WebSocket onMessage → ringbuffer.push. A 30Hz throttler drains up to 30 messages per tick to the visible list. If we fall behind, drop oldest from the buffer — viewers don't care about messages they missed during a scroll."*

Reactions: *"Tap heart → send 'reaction:HEART' over chat WebSocket → server fan-outs as light events → all viewers render a particle. We batch incoming particles in 100ms windows so 50 simultaneous taps render as one animation, not 50."*

> ■ ***Tip:*** *Mention **LL-HLS partial segments** — the HLS extension Apple added that lets segments stream while still being encoded. Cuts latency by half.*

5. Actual Code (Reference Snippets)

Low-latency ExoPlayer + chat throttler + Compose pin-to-bottom

Kotlin

```kotlin
class LiveStreamPlayer(ctx: Context, val hlsUrl: String) {
    val player = ExoPlayer.Builder(ctx)
        .setLoadControl(DefaultLoadControl.Builder()
            // Very small buffers for low-latency live
            .setBufferDurationsMs(2_000, 8_000, 500, 1_500)
            .setTargetBufferBytes(C.LENGTH_UNSET)
            .build())
        .setMediaSourceFactory(HlsMediaSource.Factory(DefaultHttpDataSource.Factory())
            .setAllowChunklessPreparation(true)
            .setPlaylistTrackerFactory(
                DefaultHlsPlaylistTracker.FACTORY))
        .build()

    fun start() {
        player.setMediaItem(MediaItem.fromUri(hlsUrl))
        player.prepare()
        player.playWhenReady = true
```

```
    }

    fun seekToLive() {
        // Snap to live edge after a pause/buffer
        player.seekToDefaultPosition()
    }
}

class ChatViewModel(private val socket: ChatWebSocket) : ViewModel() {
    private val buffer = ArrayDeque<ChatMessage>()
    private val _visible = MutableStateFlow<List<ChatMessage>>(emptyList())
    val visible = _visible.asStateFlow()
    private val maxVisible = 200
    private val drainHz = 30

    init {
        socket.events.onEach { msg -> buffer.addLast(msg) }.launchIn(viewModelScope)
        viewModelScope.launch {
            while (true) {
                delay(1000L / drainHz)
                val take = minOf(buffer.size, 5)   // up to 5 per 33ms
                repeat(take) {
                    val m = buffer.pollFirst() ?: return@repeat
                    _visible.update { list ->
                        (list + m).takeLast(maxVisible)
                    }
                }
            }
        }
    }

    fun send(text: String) = viewModelScope.launch { socket.send(text) }
    fun react(emoji: String) = viewModelScope.launch { socket.sendReaction(emoji) }
}

@Composable
fun ChatList(messages: List<ChatMessage>) {
    val listState = rememberLazyListState()
    LaunchedEffect(messages.size) {
        // pin to bottom unless user scrolled up
        if (listState.firstVisibleItemIndex > messages.size - 5) {
            listState.animateScrollToItem(messages.size - 1)
        }
    }
    LazyColumn(state = listState) {
        items(messages, key = { it.id }) { m -> ChatRow(m) }
    }
}
```

6. Cross-Questions & How to Answer Them

Q: Why LL-HLS instead of WebRTC?

Scale. WebRTC scales beautifully to small groups (sub-second) but mesh complexity makes 1M viewers hard. LL-HLS uses the existing CDN infrastructure with ~3s latency — good enough for live streaming, and far cheaper. Twitch uses LL-HLS in production.

Q: How do you handle a 1M-viewer chat without choking the client?

Server-side: chat is bucketed by region / shard; clients connect to a shard. Client-side: ring buffer + throttler renders at most 30 msg/sec to UI; dropping older messages on overflow. Users don't perceive missing messages on hot streams.

Q: How is moderation done?

Server-side: spam filter, slow mode, moderator commands ('/timeout user'). Client-side: blocked users' messages filtered before rendering.

Q: How do you push 'X is live' to a million followers efficiently?

Fan-out via a notification service: server topic 'channel:X:live' with subscribers stored in Redis sets; on stream start, iterate followers in batches, push FCM. Async, throttled (~10K pushes/sec).

Q: How does ABR work on live streams?

Same as Netflix — ExoPlayer's TrackSelector picks among manifest variants by bandwidth. For LL-HLS, variant playlists update frequently; the player follows new segments as they arrive.

7. What Better? Alternative Approaches

Sub-second WebRTC for VIP / esports tournaments where latency is critical.

Chat replay for VODs — store messages with timestamp; replay aligned to playback position.

Channel points / cheers — monetised real-time events on top of chat infra.

Subscriber-only mode, follower-only mode — server gate.

Adaptive chat throttle based on device perf: high-end devices render 60Hz, low-end drop to 15Hz.

Key takeaways

- LL-HLS for ~3s latency at CDN scale.
- Chat = WebSocket + ring buffer + drain throttle.
- Pin-to-bottom is the chat scroll pattern.
- Reactions are 1-byte fan-out + batched animations.